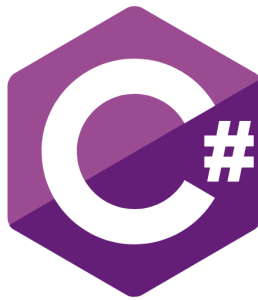


ملخص شامل لـ



دليل تعليمي مختصر و منظم

Amine Mohamed

Telegram: @codeDarija

١ ديسمبر ٢٠٢٥

جدول المحتويات

1. مقدمة إلى #C و بيئة .NET — ما هي #C وما هي منصة .NET (Framework, .NET Core, .NET 8)
2. مقدمة إلى #C و بيئة .NET — الفرق بين CLR و CLI
3. مقدمة إلى #C و بيئة .NET — تنصيب Visual Studio و Visual Studio Code
4. مقدمة إلى #C و بيئة .NET — إنشاء وتشغيل أول برنامج Console
5. مقدمة إلى #C و بيئة .NET — التعليقات وأنواعها XML Documentation
6. مقدمة إلى #C و بيئة .NET — فهم بنية المشروع Project Structure
7. أساسيات #C — المتغيرات وأنواع البيانات (Value Types vs Reference Types)
8. أساسيات #C — المعاملات الحسابية والمنطقية والعلاقاتية
9. أساسيات #C — تحويل الأنواع Convert و Parse و Casting
10. أساسيات #C — التعامل مع Console (الإدخال والإخراج)
11. أساسيات #C — التعامل مع Enumerations
12. أساسيات #C — فهم Nullable Types
13. هياكل التحكم الشرطية والتكرارية — الشروط: if, else if, else
14. هياكل التحكم الشرطية والتكرارية — الاختيار switch و Switch Expressions الحديثة

15. هياكل التحكم الشرطية والتكرارية — الحلقات: for, while, do-while

16. هياكل التحكم الشرطية والتكرارية — الحلقة foreach

17. هياكل التحكم الشرطية والتكرارية — مفاتيح التحكم break, continue, return

18. المصفوفات والسلاسل النصية — إنشاء وتهيئة المصفوفات

19. المصفوفات والسلاسل النصية — المصفوفات متعددة الأبعاد و Jagged Arrays

20. المصفوفات والسلاسل النصية — العمليات الأساسية على النصوص

21. المصفوفات والسلاسل النصية — StringBuilder

22. المصفوفات والسلاسل النصية — Immutable Strings ولماذا هي مهمة

23. الدوال والطرق (Methods) — تعريف الدوال والاستدعاء

24. الدوال والطرق (Methods) — Parameters (By Value, ref, out, in)

25. الدوال والطرق (Methods) — Return Values

26. الدوال والطرق (Methods) — Method Overloading

27. الدوال والطرق (Methods) — Named & Optional Parameters

28. الدوال والطرق (Methods) — Local Functions

29. المجالات (Scopes) وإدارة الذاكرة — نطاق المتغيرات

30. المجالات (Scopes) وإدارة الذاكرة — Garbage Collector

31. المجالات (Scopes) وإدارة الذاكرة — Stack vs Heap

32. مبادئ OOP - الجزء الأول — تعريف الفئات والكائنات

33. مبادئ OOP – الجزء الأول — الحقول والخصائص (Fields vs Properties)

34. مبادئ OOP – الجزء الأول — البواني (Constructors)

35. مبادئ OOP – الجزء الأول — Encapsulation ومعدلات الوصول

36. مبادئ OOP – الجزء الأول — Static Members

37. مبادئ OOP – الجزء الثاني — الوراثة (Inheritance)

38. مبادئ OOP – الجزء الثاني — virtual/override g Polymorphism

39. مبادئ OOP – الجزء الثاني — Interfaces

40. مبادئ OOP – الجزء الثاني — Abstract Classes

41. مبادئ OOP – الجزء الثاني — Sealed Classes and Methods

42. التعامل مع الأخطاء والاستثناءات — ما هي Exceptions

43. التعامل مع الأخطاء والاستثناءات — كتل try-catch-finally

44. التعامل مع الأخطاء والاستثناءات — رمي الأخطاء throw

45. التعامل مع الأخطاء والاستثناءات — إنشاء استثناءات مخصصة
Custom Exceptions

46. التعامل مع الأخطاء والاستثناءات — Global Error Handling

47. المجموعات والأنواع العامة (Collections & Generics) — List,
Dictionary

48. المجموعات والأنواع العامة (Collections & Generics) — Queue g
Stack

49. المجموعات والأنواع العامة (Collections & Generics) — HashSet g
SortedList

50. المجموعات والأنواع العامة (Collections & Generics) — Generic
Generic Constraints g Types
51. المجموعات والأنواع العامة (Collections & Generics) —
IEnumerator g IEnumerable
52. LINQ والاستعلام عن البيانات — مقدمة إلى LINQ
53. LINQ والاستعلام عن البيانات — Query Syntax
54. LINQ والاستعلام عن البيانات — Method Syntax
55. LINQ والاستعلام عن البيانات — التصفية والترتيب والتجميع
56. LINQ والاستعلام عن البيانات — Any g First g Where g Select
57. LINQ والاستعلام عن البيانات — LINQ to Objects vs LINQ to Entities
58. التعامل مع الملفات — القراءة والكتابة للملفات النصية
59. التعامل مع الملفات — التعامل مع المجلدات والمسارات Path &
Directory
60. التعامل مع الملفات — StreamWriter g StreamReader
61. التعامل مع الملفات — استخدام using statement
62. التعامل مع الملفات — التعامل مع JSON باستخدام
System.Text.Json
63. مفاهيم متقدمة — Multicast Delegates g Delegates
64. مفاهيم متقدمة — Event Handlers g Events
65. مفاهيم متقدمة — Lambda Expressions
66. مفاهيم متقدمة — Extension Methods
67. مفاهيم متقدمة — Anonymous Types

68. البرمجة غير المتزامنة (Asynchronous Programming) — مقدمة إلى
await و async

69. البرمجة غير المتزامنة (Asynchronous Programming) Tasks —

70. البرمجة غير المتزامنة (Asynchronous Programming) — إدارة
الأخطاء في Async

71. البرمجة غير المتزامنة (Asynchronous Programming) Parallel —
Programming Basics

72. التعامل مع قواعد البيانات — مقدمة إلى ADO.NET

73. التعامل مع قواعد البيانات — تنفيذ أوامر SQL الأساسية

74. التعامل مع قواعد البيانات — مقدمة إلى Entity Framework Core

75. التعامل مع قواعد البيانات — إنشاء النماذج والعلاقات (& Models
(Relationships

76. التعامل مع قواعد البيانات — Migration وإدارة قاعدة البيانات

77. بناء تطبيقات #C متنوعة — تطبيقات Console

78. بناء تطبيقات #C متنوعة — تطبيقات Windows Forms

79. بناء تطبيقات #C متنوعة — مقدمة إلى WPF

80. بناء تطبيقات #C متنوعة — بناء APIs باستخدام ASP.NET Core

81. بناء تطبيقات #C متنوعة — مقدمة إلى Dependency Injection

مقدمة إلى #C و بيئة .NET — ما هي #C وما هي منصة .NET (Framework, .NET Core, .NET 8).

ما هي #C وما هي منصة .NET (Framework, .NET Core, .NET 8).

في هذا الدرس، سنتعرف على لغة البرمجة #C، ولماذا هي مهمة. كما سنستكشف منصة .NET، وهي البيئة التي تعمل فيها تطبيقات #C، ونفهم تطورها من .NET Framework إلى .NET Core وصولاً إلى .NET 8 الحديثة.

ما هي #C؟

#C (تُنطق "سي شارب") هي لغة برمجة حديثة، متعددة الأنماط، وموجهة للكائنات، طورتها شركة مايكروسوفت. تستخدم على نطاق واسع لبناء أنواع مختلفة من التطبيقات، مثل تطبيقات الويب، تطبيقات سطح المكتب، تطبيقات الأجهزة المحمولة، وحتى الألعاب.

ما هي منصة .NET؟

منصة .NET هي إطار عمل شامل ومفتوح المصدر (SDK) يوفر كل ما تحتاجه لبناء وتشغيل تطبيقات #C. تتضمن مكتبات كبيرة من الأكواد الجاهزة (BCL) وبيئة تشغيل (CLR) تقوم بإدارة تنفيذ الكود. تطورت المنصة من **.NET Framework** الأقدم، إلى **.NET Core** العابرة للمنصات، وصولاً إلى الإصدارات الحديثة مثل **.NET 8** التي تجمع بين أفضل ميزات السابقتين.

مثال بسيط

```
using System;

public class MyFirstProgram
{
    public static void Main(string[] args)
    {
        Console.WriteLine("C#! مرحباً بكم في");
    }
}
```



```
}  
}
```

شرح الكود

هذا الكود يمثل أبسط برنامج في #C. يقوم السطر `Console.WriteLine` ("مرحباً بكم في #C!"); بطباعة النص "مرحباً بكم في #C!" على الشاشة. تستخدم كلمة `Console` للتعامل مع الإدخال والإخراج، و `WriteLine` هي دالة لطباعة سطر نصي.

- **#C** هي لغة برمجة قوية ومرنة من مايكروسوفت لإنشاء أنواع مختلفة من التطبيقات.
- **.NET** هي منصة متكاملة توفر الأدوات والمكتبات اللازمة لبناء وتشغيل تطبيقات #C.
- تطورت منصة **.NET** من Framework إلى Core ثم أصبحت موحدة ومفتوحة المصدر في إصدارات مثل **.NET 8**.

مقدمة إلى #C و بيئة .NET — الفرق بين CLR و CLI

الفرق بين CLR و CLI

فهم الفرق بين CLR و CLI هو أساسي لأي مطور #C و .NET. هذه المفاهيم هي القلب النابض لكيفية عمل تطبيقاتنا، حيث يضمنان التوافقية والتشغيل السلس للكود.

ما هو CLR؟ (Common Language Runtime)

CLR هو الآلة الافتراضية (Virtual Machine) لمكتبة .NET. وظيفته الأساسية هي تنفيذ الكود المكتوب بلغات .NET (مثل #C) بعد تحويله إلى لغة وسيطة (Intermediate Language - IL). يتولى CLR مهام حيوية مثل إدارة الذاكرة (Garbage Collection)، معالجة الاستثناءات، والأمان. إنه المحرك الذي يشغل تطبيقك.

ما هو CLI؟ (Common Language Infrastructure)

CLI ليست أداة، بل هي مواصفة مفتوحة ومعياري. تحدد هذه المواصفة مجموعة من القواعد للمكتبات واللغات وبيئات التشغيل لتكون متوافقة مع .NET. هي التي تحدد كيفية عمل الكود الوسيط (IL) وكيف يجب أن تكون بيئة التشغيل. CLR الخاص بمايكروسوفت هو مجرد تنفيذ واحد لهذه المواصفة.

مثال على الكود

```
using System;

public class MyProgram
{
    public static void Main(string[] args)
    {
        Console.WriteLine("مرحباً بكم في عالم C# و .NET!");
    }
}
```

شرح الكود

يوضح هذا الكود برنامج #C بسيط. عند تجميعه (Compilation)، يتحول إلى لغة وسيطة (IL). هنا يأتي دور CLR، فهو المسؤول عن أخذ هذا الكود الوسيط وتشغيله، وإدارة الموارد اللازمة للتطبيق. CLR يوفر بيئة آمنة ومستقرة لتنفيذ الكود، مستنداً إلى القواعد المحددة في مواصفة CLI.

- **CLR** هو الآلة الافتراضية الفعلية التي تُشغّل كود #C.
- **CLI** هو المعيار والمواصفة المفتوحة التي تحدد كيف يجب أن تعمل بيئة .NET.
- **CLR** من Microsoft هو تطبيق لمواصفة CLI.

مقدمة إلى #C و بيئة .NET — تنصيب Visual Studio و Visual Studio Code

تنصيب Visual Studio و Visual Studio Code

يعد تنصيب بيئات التطوير المتكاملة (IDE) ومحركات الأكواد خطوة أساسية للبدء بتطوير تطبيقات #C. في هذا الدرس، سنتعرف على الأدوات الرئيسية وكيفية تهيئتها لرحلتك مع #C وبيئة .NET.

لماذا نحتاج لهذه البيئات؟

نحتاج لبيئات مثل Visual Studio و Visual Studio Code لكتابة أكواد #C، وتصحيح الأخطاء، وتشغيل المشاريع. توفر هذه الأدوات بيئة متكاملة تسرع عملية التطوير وتجعلها أكثر كفاءة، مع دعم كامل لإطار عمل .NET.

مثال عملي (تهيئة مشروع جديد)

بعد تنصيب .NET SDK (والذي يأتي مع Visual Studio أو يمكن تنصيبه بشكل منفصل)، يمكنك إنشاء مشروع #C بسيط باستخدام سطر الأوامر:

```
dotnet new console -o MyFirstApp
cd MyFirstApp
dotnet run
```

شرح الكود

هذه الأوامر تمكنك من إنشاء وتشغيل تطبيق #C بسيط. الأمر `dotnet new console -o MyFirstApp` ينشئ مشروع تطبيق وحدة تحكم جديد باسم `MyFirstApp`. الأمر `cd MyFirstApp` ينتقل إلى مجلد المشروع الذي تم إنشاؤه حديثًا. وأخيرًا، `dotnet run` يقوم بتشغيل التطبيق المكتوب بلغة #C داخل هذا المشروع.

- **Visual Studio**: بيئة تطوير متكاملة (IDE) قوية وغنية بالميزات، مثالية للمشاريع الكبيرة والمعقدة على أنظمة Windows.
- **Visual Studio Code**: محرر أكواد خفيف الوزن وقابل للتخصيص بدرجة عالية، يدعم #C عبر الإضافات، ويعمل على جميع الأنظمة الأساسية (Windows, macOS, Linux).
- **.NET SDK**: حزمة تطوير البرامج الأساسية التي يجب تنصيبها للعمل مع #C، وتتضمن المترجم والأدوات اللازمة لبناء وتشغيل تطبيقات .NET.

مقدمة إلى #C و بيئة .NET — إنشاء وتشغيل أول برنامج Console

إنشاء وتشغيل أول برنامج Console

في هذا الدرس، سنتعلم الخطوات الأساسية لإنشاء وتشغيل أول تطبيق Console بسيط باستخدام لغة #C. تطبيقات Console هي تطبيقات نصية تعمل من خلال سطر الأوامر، وهي نقطة انطلاق ممتازة لفهم أساسيات البرمجة بلغة #C وبيئة .NET.

البرنامج Console

برنامج Console هو نوع من التطبيقات يتفاعل مع المستخدم عبر إدخال وإخراج نصي في نافذة سطر الأوامر (الشاشة السوداء). يُستخدم لتعلم أساسيات اللغة، اختبار الميزات، أو لإنشاء أدوات مساعدة بسيطة لا تحتاج لواجهة رسومية معقدة.

مثال الكود

```
using System;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("مرحباً بالعالم!");
    }
}
```

شرح الكود

هذا الكود يقوم بإنشاء برنامج بسيط يعرض رسالة "مرحباً بالعالم!" على الشاشة.

- `using System`; يُخبر المترجم بأننا سنستخدم مكتبة `System`، التي تحتوي على وظائف أساسية مثل `Console`.
- `class Program`: كل كود `#C` يجب أن يكون ضمن فئة (class). `Program` هو اسم الفئة الافتراضي.
- `static void Main(string[] args)`: هذه هي الدالة الرئيسية التي يبدأ منها تنفيذ أي برنامج `#C`.
- `Console.WriteLine("مرحباً بالعالم!")`; هذه الأمر يُستخدم لعرض النص الموجود بين القوسين على شاشة `Console`، ثم ينتقل إلى سطر جديد.

ملخص الفكرة

- برامج `Console` تطبيقات نصية بسيطة تُستخدم لتعلم أساسيات `#C`.
- الدالة `Main` هي نقطة البداية لتنفيذ أي برنامج `#C`.
- باستخدام `Console.WriteLine()`، يمكنك إخراج النصوص وعرضها للمستخدم على شاشة سطر الأوامر.

مقدمة إلى C# و بيئة .NET — التعليقات وأنواعها

XML Documentation

التعليقات وأنواعها في C# (XML Documentation)

التعليقات هي نصوص لا يقوم المترجم البرمجي (Compiler) بتنفيذها، ولكنها ضرورية جداً لمساعدة المبرمجين على فهم الكود. إنها بمثابة مذكرات وشروحات داخل الكود تجعل قراءته وصيانته أسهل، سواء لك أو للمطورين الآخرين.

أنواع التعليقات

توجد ثلاثة أنواع رئيسية للتعليقات في C#: **تعليقات السطر الواحد**، **تعليقات الأسطر المتعددة**، و**تعليقات XML Documentation** المخصصة لتوليد التوثيق الاحترافي.

مثال على الكود

```
// هذا تعليق سطر واحد، يستخدم لشرح جزء محدد

/*
 * هذا تعليق متعدد الأسطر.
 * يمكن استخدامه لكتابة شروحات أطول وأكثر تفصيلاً.
 */

/// <summary>
/// هذه دالة بسيطة لجمع عددين صحيحين.
/// </summary>
/// <param name="num1">العدد الصحيح الأول.</param>
/// <param name="num2">العدد الصحيح الثاني.</param>
/// <returns>مجموع العددين الصحيحين.</returns>
public static int AddNumbers(int num1, int num2)
{
    return num1 + num2;
}
```


شرح الكود

يوضح الكود أعلاه أنواع التعليقات المختلفة في C#. يتم استخدام الرمز `//` لتعليق سطر واحد فقط. أما الرمز `/* ... */` فيُستخدم لتعليق عدة أسطر. والرمز `///` يُستخدم لتعليقات XML Documentation، وهي تعليقات خاصة تمكّنك من توثيق عناصر الكود مثل الدوال والكلاسات بتنسيق XML. تقوم بيئة التطوير (IDE) مثل Visual Studio بقراءة هذه التعليقات وعرضها كتلميحات مفيدة عند استخدام هذه العناصر، ويمكن أيضاً استخدامها لتوليد ملفات توثيق خارجية لمشروعك.

- التعليقات ضرورية لجعل الكود واضحاً ومفهوماً للمبرمجين الآخرين ولنفسك مستقبلاً.
- استخدم النوع المناسب من التعليقات حسب الحاجة: سطر واحد، عدة أسطر، أو توثيق XML.
- تعليقات XML تساعد في توليد التوثيق الرسمي لمشروعك وتوفر تلميحات مفيدة في بيئة التطوير.

مقدمة إلى #C و بيئة .NET — فهم بنية المشروع

Project Structure

فهم بنية المشروع Project Structure

فهم بنية مشروع #C أمر حيوي لأي مطور، فهو يساعدك على تنظيم الكود الخاص بك، وإدارة الأجزاء المختلفة من تطبيقك، وفهم كيفية تجميعها لتكوين منتج نهائي قابل للتشغيل.

المكونات الأساسية لمشروع #C

يتكون مشروع #C عادةً من حل (**Solution**) يمكن أن يحتوي على مشروع واحد أو أكثر (**Project**). كل مشروع بدوره يضم مجموعة من الملفات (مثل ملفات الكود #C، الإعدادات، الموارد) التي تعمل معًا لبناء تطبيق واحد. هذا الهيكل الهرمي يسهل إدارة المشاريع الكبيرة والصغيرة.

مثال لبنية مشروع بسيط

```
using System;

namespace MyFirstProject
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("مرحباً بك في عالم C#!");
        }
    }
}
```

شرح الكود

يعرض هذا الكود ملف **Program.cs** النموذجي داخل مشروع تطبيق كونسول بسيط. السطر **using System**; يستورد مكتبات أساسية. **namespace MyFirstProject** ينظم الكود تحت اسم فريد لمنع تضارب الأسماء. داخل **class Program**، الدالة **Main** هي نقطة البداية لتنفيذ أي تطبيق C#، و **Console.WriteLine** تستخدم لعرض نص على الشاشة.

- **الحل (Solution)** هو الحاوية الرئيسية التي تجمع مشروعًا واحدًا أو أكثر ذات صلة.
- **المشروع (Project)** هو مجموعة من ملفات الكود والموارد التي تُبنى معًا لتكوين ملف تنفيذي أو مكتبة.
- **مساحات الأسماء (Namespaces)** تُستخدم لتنظيم الكود ومنع تضارب الأسماء بين الفئات المختلفة، مما يجعل الكود أكثر قابلية للإدارة.

أساسيات #C — المتغيرات وأنواع البيانات (Value Types vs Reference Types)

أساسيات #C: المتغيرات وأنواع البيانات (Value Types vs Reference Types)

في لغة #C، المتغيرات هي أماكن لتخزين البيانات. فهم كيفية تخزين هذه البيانات في الذاكرة أمر حيوي لكتابة برامج فعالة وخالية من الأخطاء. هذه الدرس يوضح الفرق بين أنواع القيمة (Value Types) وأنواع المرجع (Reference Types).

أنواع القيمة وأنواع المرجع

الفرق الأساسي يكمن في كيفية تخزين البيانات في الذاكرة. **أنواع القيمة** تخزن البيانات مباشرة في الموقع الذي تم تعريف المتغير فيه (غالبًا في المكس - Stack). **أنواع المرجع** تخزن مرجعًا (عنوانًا) لموقع البيانات الفعلي في الذاكرة (غالبًا في الكومة - Heap)، وليس البيانات نفسها.

مثال برمجي

```
int num1 = 10;
int num2 = num1; // num2 gets a copy of num1's value
num2 = 20;

Console.WriteLine($"num1: {num1}"); // Output: num1: 10
Console.WriteLine($"num2: {num2}"); // Output: num2: 20

class MyObject
{
    public int Value;
}

MyObject obj1 = new MyObject { Value = 100 };
MyObject obj2 = obj1; // obj2 refers to the same object as obj1
obj2.Value = 200;
```

```
Console.WriteLine($"obj1.Value: {obj1.Value}"); // Output: obj1.Value: 200
Console.WriteLine($"obj2.Value: {obj2.Value}"); // Output: obj2.Value: 200
```

شرح الكود

في المثال، `num1` و `num2` هما متغيرات من نوع قيمة (`int`). عندما أسندنا `num1` إلى `num2`، تم إنشاء **نسخة** من قيمة `num1` (التي هي 10) وأسندت إلى `num2`. لذلك، عندما غيرنا `num2` إلى 20، بقيت قيمة `num1` كما هي (10).

أما `obj1` و `obj2`، فهما متغيرات من نوع مرجع (`MyObject`). عندما أسندنا `obj1` إلى `obj2`، لم يتم نسخ الكائن نفسه، بل تم نسخ **المرجع**. هذا يعني أن كلا المتغيرين `obj1` و `obj2` يشيران الآن إلى نفس الكائن في الذاكرة. وبالتالي، عندما غيّرنا `obj2.Value` إلى 200، تغيرت قيمة `obj1.Value` أيضًا لأننا كنا نعدّل نفس الكائن.

- **أنواع القيمة (Value Types):** مثل `int`, `double`, `structs`. تخزن البيانات مباشرة في المكس. عند النسخ، يتم إنشاء نسخة جديدة من البيانات.
- **أنواع المرجع (Reference Types):** مثل `class`, `string`, `arrays`. تخزن مرجعًا لموقع البيانات في الكومة. عند النسخ، يتم نسخ المرجع وليس البيانات.
- فهم هذا الفرق حيوي لتجنب السلوكيات غير المتوقعة في برنامجك، خاصة عند تمرير المتغيرات بين الدوال أو عند التعامل مع هياكل البيانات المعقدة.

أساسيات C# — المعاملات الحسابية والمنطقية والعلائقية

المعاملات الحسابية والمنطقية والعلائقية في C#

تعتبر المعاملات (Operators) رموزاً أساسية في لغة C# تخبر المترجم بإجراء عمليات معينة على المتغيرات والقيم. فهمها ضروري لكتابة أي برنامج يقوم بمعالجة البيانات واتخاذ القرارات.

ما هي المعاملات؟

تُستخدم المعاملات لأداء عمليات حسابية، مقارنة قيم، ودمج شروط منطقية. وهي تنقسم إلى ثلاثة أنواع رئيسية: المعاملات **الحسابية** للمسائل الرياضية (+، -)، **العلائقية** للمقارنات (<، =، >)، و**المنطقية** لربط الشروط (&&، ||).

مثال توضيحي

```
int num1 = 10;
int num2 = 5;

// معامِل حسابي: الجمع
int sum = num1 + num2;

// معامِل علائقي: أكبر من
bool isNum1Greater = (num1 > num2);

// معامِل منطقي: و (AND)
bool complexCondition = (sum > 10 && isNum1Greater);

Console.WriteLine($"المجموع: {sum}");
Console.WriteLine($"هل {num1} أكبر من {num2}؟ {isNum1Greater}");
Console.WriteLine($"الشرط الكلي صحيح؟ {complexCondition}");
```

شرح الكود

يقوم الكود أولاً بحساب مجموع **num1** و **num2** باستخدام المعامل الحسابي **+**. ثم يتحقق مما إذا كانت **num1** أكبر من **num2** باستخدام المعامل العلائقي **<**. أخيراً، يجمع شرطين (أن يكون المجموع أكبر من 10، وأن يكون **num1** أكبر من **num2**) باستخدام المعامل المنطقي **&&**، ويطبع النتائج.

- المعاملات رموز لأداء مهام محددة (حساب، مقارنة، منطق).
- تساعد المعاملات الحسابية في العمليات الرياضية الأساسية.
- تستخدم المعاملات العلائقية والمنطقية لاتخاذ القرارات والتحكم في سير البرنامج بناءً على الشروط.

أساسيات #C — تحويل الأنواع Casting و Parse و Convert

تحويل الأنواع (Casting, Parse, Convert) في #C

في البرمجة، قد تحتاج غالباً إلى تغيير نوع البيانات من شكل لآخر، مثل تحويل نص يمثل رقماً إلى رقم حقيقي، أو تحويل رقم عشري إلى رقم صحيح. هذا الدرس يشرح الأدوات الأساسية في #C للقيام بهذه العملية بفعالية وأمان.

مفهوم تحويل الأنواع في #C

يشير تحويل الأنواع إلى عملية تغيير نوع بيانات متغير إلى نوع آخر. تتضمن هذه العملية: **Casting** (التحويل الصريح والضمني)، والذي يستخدم بين الأنواع المتوافقة. بالإضافة إلى استخدام الطرق المخصصة مثل **Parse** (لتحويل النصوص إلى أرقام) و **Convert** (لفئة توفر طرقاً شاملة لتحويل أنواع البيانات المختلفة).

مثال توضيحي

```
int numInt = 10;
long numLong = numInt; // Implicit Casting

double numDouble = 9.75;
int numIntExplicit = (int)numDouble; // Explicit Casting

string strNum = "123";
int parsedNum = int.Parse(strNum); // Using Parse

int value = 50;
string convertedStr = Convert.ToString(value); // Using Convert

Console.WriteLine($"Implicit Cast: {numLong}");
Console.WriteLine($"Explicit Cast: {numIntExplicit}");
Console.WriteLine($"Parsed Value: {parsedNum}");
```



```
Console.WriteLine($"Converted String: {convertedStr}");
```

شرح الكود

في هذا المثال، بدأنا بتحويل `numInt` (من نوع `int`) إلى `numLong` (من نوع `long`) بشكل ضمني لأنه آمن ولا يفقد بيانات. ثم قمنا بتحويل `numDouble` (من نوع `double`) إلى `numIntExplicit` (من نوع `int`) بشكل صريح باستخدام `(int)`، مما أدى إلى فقدان الجزء العشري. بعد ذلك، استخدمنا `int.Parse()` لتحويل النص `"123"` إلى رقم صحيح `parsedNum`. وأخيراً، حولنا الرقم `value` إلى نص `convertedStr` باستخدام `Convert.ToString()`.

- تحويل الأنواع ضروري للتعامل مع البيانات بأشكال مختلفة في تطبيقات `#C`.
- **Casting** يُستخدم للتحويل بين الأنواع المتوافقة، سواء ضمناً (آمن) أو صريحاً (قد يفقد بيانات).
- الطرق مثل **Parse** (للتحويل من نص إلى رقم) و **Convert** (لفئة تحويل شاملة) توفر مرونة وقوة إضافية.

أساسيات #C — التعامل مع Console (الإدخال والإخراج)

التعامل مع Console (الإدخال والإخراج)

في هذا الدرس، سنتعلم كيف تتفاعل برامج #C مع المستخدم عبر نافذة الأوامر (Console). سنكتشف كيفية عرض النصوص والمعلومات على الشاشة، وكيفية قراءة المدخلات التي يكتبها المستخدم.

مفتاح الفكرة: الإدخال والإخراج القياسي

ال Console هو أداة أساسية للتواصل المباشر بين برنامجك والمستخدم. نستخدم دوال الإخراج مثل `Console.WriteLine()` لعرض النصوص مع الانتقال لسطر جديد، و `Console.Write()` للعرض دون الانتقال. أما لاستقبال البيانات من المستخدم، فنستخدم دوال الإدخال مثل `Console.ReadLine()`.

مثال عملي

```
using System;

public class ConsoleInteraction
{
    public static void Main(string[] args)
    {
        Console.Write("من فضلك أدخل اسمك ");
        string userName = Console.ReadLine();
        Console.WriteLine($"{userName}! مرحباً بك");
        Console.WriteLine("...اضغط أي مفتاح للمتابعة");
        Console.ReadKey();
    }
}
```

شرح الكود

يبدأ البرنامج بسؤال المستخدم عن اسمه باستخدام `Console.WriteLine()`. ثم يقوم `Console.ReadLine()` بقراءة النص الذي يدخله المستخدم ويخزنه في متغير `userName`. بعد ذلك، تعرض `Console.WriteLine()` رسالة ترحيب مخصصة باستخدام الاسم المدخل. أخيراً، `Console.ReadKey()` توقف الشاشة مؤقتاً حتى يضغط المستخدم على أي مفتاح، مما يمنع النافذة من الإغلاق فوراً.

- `Console.WriteLine()`: لعرض نص على الشاشة والانتقال تلقائياً إلى سطر جديد بعده.
- `Console.Write()`: لعرض نص على الشاشة والبقاء في نفس السطر.
- `Console.ReadLine()`: لقراءة سطر كامل من إدخال المستخدم كنص (string).

أساسيات #C — التعامل مع Enumerations

التعامل مع Enumerations في #C

في هذا الدرس، سنتعلم عن ال Enumerations (أو "Enums" اختصارًا) في #C. تُعد ال Enums طريقة قوية لتحديد مجموعة من الثوابت المسماة، مما يجعل الكود أكثر وضوحًا وسهولة في القراءة والصيانة.

المفهوم الأساسي

ال Enum هو نوع بيانات خاص يسمح لك بتعريف مجموعة من الثوابت المرتبطة ببعضها البعض تحت اسم واحد. بدلاً من استخدام الأرقام السحرية أو السلاسل النصية، يمكنك استخدام أسماء معبرة تجعل كودك أسهل للفهم.

مثال على الكود

```
public enum OrderStatus
{
    Pending,
    Processing,
    Shipped,
    Delivered,
    Cancelled
}

public class Program
{
    public static void Main(string[] args)
    {
        OrderStatus currentStatus = OrderStatus.Processing;
        Console.WriteLine("حالة الطلب الحالية: " + currentStatus);

        if (currentStatus == OrderStatus.Delivered)
        {
            Console.WriteLine("تم تسليم الطلب بنجاح");
        }
        else
```

```
{  
    Console.WriteLine("الطلب لم يُسَلَّم بعد");  
}  
}  
}
```

شرح الكود

يُعرّف هذا الكود تعدادًا (enum) باسم `OrderStatus` يمثل حالات طلب مختلفة. كل حالة (مثل `Pending`، `Processing`) هي ثابت مُسمى. في الدالة `Main`، تُعلن عن متغير من نوع `OrderStatus` وتُسند إليه قيمة `Processing`. ثم نطبع هذه القيمة ونستخدمها في شرط للتحقق من حالة الطلب، مما يوضح كيف يمكن استخدام الـ Enums لجعل الشروط أكثر قابلية للقراءة.

نقاط أساسية

- تُستخدم الـ Enums لتعريف مجموعة من الثوابت المرتبطة ذاتيًا.
- تُحسن الـ Enums من وضوح الكود وقابلية صيانتها بجعل القيم أكثر تعبيرًا.
- يُمكن الوصول إلى أعضاء الـ Enum باستخدام اسمها متبوعًا بنقطة ثم اسم العضو (مثل `OrderStatus.Processing`).

أساسيات #C — فهم Nullable Types

فهم Nullable Types

في لغة #C، أنواع البيانات القيمة (Value Types) مثل `int` و `double` لا يمكن أن تكون `null` بشكل افتراضي. ولكن في بعض الحالات، قد نحتاج إلى تمثيل قيمة غائبة أو غير محددة لهذه الأنواع. هنا يأتي دور Nullable Types.

ما هي Nullable Types؟

هي بنية خاصة تسمح لأنواع البيانات القيمة بأن تحمل قيمة `null` بالإضافة إلى قيمتها المعتادة. يمكنك تعريف نوع قابل للقيم الخالية بإضافة علامة الاستفهام `?` بعد اسم النوع (مثل `int?`) أو باستخدام البنية `<T>Nullable`.

مثال عملي

```
using System;

public class Program
{
    public static void Main(string[] args)
    {
        int? age = null;
        double? temperature = 25.5;

        Console.WriteLine($"Temperature: {temperature}");

        if (age.HasValue)
        {
            Console.WriteLine($"Age is: {age.Value}");
        }
        else
        {
            Console.WriteLine("Age is null.");
        }
    }
}
```

```
// Using null-coalescing operator (??)
int actualAge = age ?? 18;
Console.WriteLine($"Actual age (defaulted): {actualAge}");
}
}
```

شرح الكود

في هذا المثال، قمنا بتعريف `age` ك `int?` مما يسمح له بأن يأخذ القيمة `null`. بينما `temperature` أخذ قيمة عادية. نستخدم الخاصية `HasValue` للتحقق مما إذا كان `age` يحمل قيمة أم `null`. إذا كان يحمل قيمة، يمكننا الوصول إليها عبر `.Value`. وفي النهاية، استخدمنا عامل التشغيل `??` (`null-` coalescing operator) لتوفير قيمة افتراضية لـ `actualAge` إذا كان `age` هو `null`.

نقاط رئيسية

- الـ `Nullable Types` تسمح لأنواع البيانات القيمة بحمل القيمة `null`.
- يتم تعريفها باستخدام علامة الاستفهام `?` (مثال: `int?`) أو `<Nullable>T`.
- للتعامل معها بأمان، استخدم `HasValue` للتحقق من وجود قيمة، و `.Value` للوصول للقيمة، أو عامل `??` لتوفير قيمة افتراضية.

هياكل التحكم الشرطية والتكرارية — الشروط: if, else if, else

الشروط: if, else if, else

تُعد هياكل التحكم الشرطية أدوات أساسية في البرمجة تمكن برامجك من اتخاذ قرارات بناءً على شروط معينة. في C#, تسمح لك عبارات `if`, `else if`, و `else` بتوجيه سير البرنامج ليتصرف بشكل مختلف حسب قيمة المتغيرات أو صحة تعبيرات منطقية.

المفهوم الأساسي: اتخاذ القرارات باستخدام if, else if, else

تتيح لك هذه الهياكل تنفيذ كتل مختلفة من الكود بناءً على ما إذا كانت شروط معينة صحيحة أم خاطئة:

- `if`: تنفذ كتلة من الكود فقط إذا كان الشرط المحدد صحيحًا (True).
- `else if`: تُستخدم لفحص شرط آخر إذا كان الشرط في `if` السابق خاطئًا. يمكنك استخدام عدة `else if` متتالية.
- `else`: يُنفذ هذا الجزء من الكود إذا كانت جميع الشروط في `if` و `else if` السابقة خاطئة. يمثل الخيار الافتراضي.

مثال برمجي

```
int درجة = 85;

if (درجة <= 90)
{
    Console.WriteLine("ممتاز");
}
else if (درجة <= 75)
{
    Console.WriteLine("جيد جداً");
}
else if (درجة <= 50)
{

```



```
Console.WriteLine("ناجح");  
}  
else  
{  
    Console.WriteLine("راسب");  
}
```

شرح الكود

في هذا المثال، نحدد متغيرًا اسمه `درجة` بقيمة 85. أولاً، يتم فحص الشرط الأول: هل `درجة` أكبر من أو تساوي 90؟ (الإجابة خاطئة لأن 85 ليست أكبر من 90). ثم، يتم فحص الشرط التالي: هل `درجة` أكبر من أو تساوي 75؟ (الإجابة صحيحة لأن 85 أكبر من 75). بما أن هذا الشرط صحيح، سيتم طباعة "جيد جداً". على الشاشة، ويتوقف البرنامج عن فحص الشروط المتبقية. لو كانت `درجة` أقل من 50، لتم طباعة "راسب".

نقاط رئيسية

- تبدأ سلسلة الشروط دائماً بـ `if`.
- يمكنك إضافة `else if` متعددة لفحص شروط بديلة على التوالي.
- يتم تنفيذ `else` فقط عندما تكون جميع الشروط في `if` و `else if` السابقة غير صحيحة.

هياكل التحكم الشرطية والتكرارية — الاختيار switch و Switch Expressions الحديثة

الاختيار switch و Switch Expressions الحديثة

تُعد هياكل التحكم الشرطية مثل **switch** أدوات أساسية لتنفيذ أكواد مختلفة بناءً على قيمة متغير معين. ومع C# 8.0، ظهرت **Switch Expressions** لتقديم طريقة أكثر إيجازًا ومرونة لتحقيق نفس الهدف، خصوصًا عند الحاجة لإرجاع قيمة.

مفهوم الاختيار switch و Switch Expressions

يسمح لك بيان **switch** بتوجيه سير البرنامج إلى مسارات متعددة بناءً على مطابقة قيمة مع حالات محددة. أما **Switch Expressions** فهي تُبسّط هذه العملية، حيث تُرجع قيمة مباشرة بناءً على النتيجة، مما يقلل من الأكواد المطلوبة ويحسن القراءة.

مثال على الكود

```
int dayNumber = 3;
string dayNameTraditional;

// Traditional switch statement
switch (dayNumber)
{
    case 1:
        dayNameTraditional = "الأحد";
        break;
    case 2:
        dayNameTraditional = "الاثنين";
        break;
    case 3:
        dayNameTraditional = "الثلاثاء";
        break;
    default:
        dayNameTraditional = "يوم غير معروف";
        break;
}
```

```

}

// Modern Switch Expression (C# 8.0+)
string dayNameModern = dayNumber switch
{
    1 => "الأحد",
    2 => "الاثنين",
    3 => "الثلاثاء",
    _ => "يوم غير معروف"
};

// Console.WriteLine(dayNameTraditional); // Output: الثلاثاء
// Console.WriteLine(dayNameModern);      // Output: الثلاثاء

```

شرح الكود

في هذا المثال، لدينا رقم يمثل يوماً. يوضح الجزء الأول كيفية استخدام بيان **switch** التقليدي لتعيين اسم اليوم إلى المتغير `dayNameTraditional`. يقوم كل **case** بتحديد قيمة معينة وعند المطابقة، يتم تنفيذ الكود الخاص به ثم يتم الخروج بـ **break**. أما الجزء الثاني، فيُظهر **Switch Expression** التي تقوم بنفس المهمة، ولكن بطريقة أكثر اختصارًا، حيث تُرجع اسم اليوم مباشرة إلى المتغير `dayNameModern` باستخدام عامل `<=`.

- **switch** التقليدي: لتنفيذ كتل أكواد مختلفة بناءً على قيمة المتغير، يتطلب **break** بعد كل حالة.
- **Switch Expressions**: لتعيين قيمة لمتغير بشكل مباشر واختصاري بناءً على تطابق القيمة، مما يقلل من حجم الكود.
- يُعد استخدام **Switch Expressions** خيارًا أفضل عندما تحتاج إلى إرجاع قيمة بناءً على حالات متعددة، لتحسين وضوح الكود وإيجازه.

هياكل التحكم الشرطية والتكرارية — الحلقات: `for`, `while`, `do-while`

الحلقات: `for`, `while`, `do-while`

تسمح لنا هياكل التحكم التكرارية بتنفيذ مجموعة من التعليمات عدة مرات بناءً على شرط معين أو عدد محدد من المرات. هذا يوفر الوقت ويجعل الكود أكثر كفاءة، وهو أساسي في البرمجة.

مفهوم الحلقات

الحلقات (Loops) هي أدوات برمجية لتكرار تنفيذ جزء من الكود. حلقة `for` مناسبة عندما نعرف عدد التكرارات مسبقًا. حلقة `while` تكرر طالما كان الشرط صحيحًا، وقد لا تنفذ أبدًا إذا كان الشرط خاطئًا من البداية. حلقة `do-while` تضمن تنفيذ الكود مرة واحدة على الأقل قبل التحقق من الشرط.

مثال الكود

```
for (int i = 0; i < 3; i++)
{
    Console.WriteLine("أهلاً بك في البرمجة " + i);
}

int count = 0;
while (count < 2)
{
    Console.WriteLine("رقم While تكرار: " + count);
    count++;
}
```

شرح الكود

المثال يوضح استخدام حلقتي `for` و `while`. حلقة `for` تبدأ المتغير `i` من 0، وتستمر طالما أن `i` أقل من 3، وتزيد `i` بواحد في كل تكرار، لتطبع رسالة ترحيب ثلاث مرات. أما حلقة `while`، فتبدأ بالمتغير `count`

وتكرر طالما أن `count` أقل من 2، وتطبع رسالة ثم تزيد `count`، مما يؤدي لتكرارين.

- الحلقات توفر طريقة فعالة لتكرار تنفيذ الكود، مما يقلل من تكرار كتابة الأوامر.
- اختر نوع الحلقة (for, while, do-while) بناءً على حاجتك لمعرفة عدد التكرارات أو لضمان التنفيذ الأول.
- يجب دائمًا التأكد من وجود شرط للتوقف لتجنب الحلقات اللانهائية التي قد تتسبب في تعليق البرنامج.

هياكل التحكم الشرطية والتكرارية — الحلقة

foreach

الحلقة foreach

تُعد هياكل التحكم التكرارية أدوات أساسية في البرمجة لتنفيذ كود معين عدة مرات. من بين هذه الهياكل، تبرز حلقة **foreach** كطريقة سهلة وفعالة للتكرار عبر مجموعات من البيانات في لغة C#.

المفهوم الأساسي

تُستخدم حلقة **foreach** للتكرار على جميع العناصر داخل مجموعة (مثل مصفوفة أو قائمة) بشكل تسلسلي، دون الحاجة للتعامل مع الفهارس. تقوم هذه الحلقة بمعالجة كل عنصر بدوره حتى يتم الانتهاء من جميع العناصر في المجموعة تلقائيًا.

مثال برمجي

```
string[] fruits = { "تفاح", "برتقال", "موز" };

Console.WriteLine("قائمة الفواكه");
foreach (string fruit in fruits)
{
    Console.WriteLine(fruit);
}
```

شرح الكود

في هذا المثال، قمنا بإنشاء مصفوفة (array) تسمى **fruits** تحتوي على أسماء ثلاث فواكه. تبدأ حلقة **foreach** بالتكرار، حيث تقوم في كل دورة بأخذ عنصر واحد من مصفوفة **fruits** وتخزينه مؤقتًا في المتغير **fruit** (من نوع **string**). يتم طباعة قيمة هذا المتغير إلى الشاشة، ثم تنتقل الحلقة إلى العنصر التالي حتى تنتهي من جميع الفواكه في المصفوفة.

نقاط رئيسية

- تُسهل حلقة `foreach` عملية التكرار على المجموعات مثل المصفوفات والقوائم بشكل كبير.
- لا تتطلب منك إدارة الفهارس يدويًا، مما يقلل من الأخطاء المحتملة ويزيد من وضوح الكود.
- تُعد خيارًا ممتازًا عندما تحتاج إلى معالجة كل عنصر في مجموعة دون الحاجة إلى ترتيب معين أو فحص شروط التوقف.

هياكل التحكم الشرطية والتكرارية — مفاتيح التحكم break, continue, return

مفاتيح التحكم break, continue, return في C#

تُعد مفاتيح التحكم **break**، **continue**، و**return** أدوات قوية لضبط مسار تنفيذ البرامج داخل الحلقات والدوال (المتودات). تساعد هذه الكلمات المفتاحية المبرمج على التحكم بمرونة في تدفق الكود لضمان سلوك البرنامج المطلوب.

break: لكسر الحلقة أو جملة switch

يُستخدم **break** للخروج الفوري من الحلقة الحالية (مثل **for**، **while**، **do-while**) أو من جملة **switch**. عند تنفيذ **break**، يتوقف تنفيذ الحلقة أو جملة **switch** وينتقل التحكم إلى السطر التالي بعد الهيكل مباشرة.

continue: لتخطي الدورة الحالية

يُستخدم **continue** لتخطي الجزء المتبقي من الدورة الحالية للحلقة والانتقال مباشرة إلى الدورة التالية. هذا يعني أن الكود الموجود بعد **continue** داخل نفس الدورة لن يتم تنفيذه، وتبدأ الدورة الجديدة فوراً.

return: للخروج من الدالة وإعادة قيمة

يُستخدم **return** لإنهاء تنفيذ الدالة (المتود) الحالية والعودة إلى الجزء من الكود الذي استدعى هذه الدالة. إذا كانت الدالة ذات نوع إرجاع (ليست **void**)، فيجب أن يعيد **return** قيمة من النوع المحدد.

مثال توضيحي

```
using System;  
  
public class Program
```



```

{
    public static void Main(string[] args)
    {
        Console.WriteLine("بدء العد");
        for (int i = 1; i <= 10; i++)
        {
            if (i % 2 == 0) // إذا كان العدد زوجياً
            {
                continue; // تخطي هذه الدورة
            }

            if (i > 7) // إذا وصل العدد إلى 8
            {
                break; // اخرج من الحلقة
            }
            Console.WriteLine($"العدد الفردي: {i}");
        }

        Console.WriteLine(GetMessage(true)); // استدعاء دالة تستخدم return
        Console.WriteLine(GetMessage(false));
    }

    public static string GetMessage(bool success)
    {
        if (success)
        {
            return "العملية تمت بنجاح."; // ينهي الدالة ويعيد القيمة
        }
        return "العملية فشلت."; // ينهي الدالة ويعيد القيمة
    }
}

```

شرح الكود

في هذا المثال، تستخدم الحلقة `for` كلمة `continue` لتخطي طباعة الأعداد الزوجية. وعندما يصل العداد `i` إلى قيمة أكبر من 7 (أي 8)، تستخدم الكلمة `break` للخروج من الحلقة تماماً. أما الدالة `GetMessage`، فتستخدم `return` لإنهاء تنفيذها وإعادة رسالة نصية بناءً على القيمة المنطقية المدخلة.

- **break**: يوقف تنفيذ الحلقة أو جملة `switch` فوراً.
- **continue**: يتخطى ما تبقى من الدورة الحالية للحلقة وينتقل للدورة التالية.

- **return**: ينهي تنفيذ الدالة الحالية ويعيد قيمة إذا كانت الدالة غير **void**.

المصفوفات والسلاسل النصية — إنشاء وتهيئة المصفوفات

إنشاء وتهيئة المصفوفات في C#

المصفوفات (Arrays) هي هياكل بيانات أساسية في C# تسمح لك بتخزين مجموعة من العناصر من نفس النوع. هذا الدرس سيشرح كيف يمكنك تعريف المصفوفات وتحديد حجمها، بالإضافة إلى كيفية إعطائها قيمًا ابتدائية.

المفهوم الأساسي

الإنشاء يعني تخصيص مساحة في الذاكرة للمصفوفة وتحديد عدد العناصر التي ستحتويها. أما **التهيئة** فهي عملية إعطاء قيم أولية لهذه العناصر عند الإنشاء أو بعده. يمكن دمج هاتين العمليتين لتسهيل العمل.

مثال الكود

```
// تعريف مصفوفة من الأعداد الصحيحة بحجم 3
int[] ages = new int[3];

// تهيئة عناصر المصفوفة بعد إنشائها
ages[0] = 25;
ages[1] = 30;
ages[2] = 22;

// تعريف وتهيئة مصفوفة من السلاسل النصية مباشرة
string[] fruits = { "تفاح", "برتقال", "موز" };
```

شرح الكود

يبين الجزء الأول من الكود كيفية تعريف مصفوفة اسمها `ages` تستطيع حفظ 3 أعداد صحيحة، مع تهيئة قيمها الافتراضية إلى صفر. ثم يتم تعيين قيم محددة لكل عنصر من عناصر المصفوفة بشكل فردي. أما الجزء الثاني فيوضح طريقة مختصرة لتعريف مصفوفة اسمها `fruits` من السلاسل النصية وتهيئة عناصرها مباشرة بقيم محددة عند الإنشاء.

- **المصفوفات** تخزن مجموعة من القيم المتجانسة (من نفس النوع).
- يمكنك **إنشاء** المصفوفة أولاً ثم **تهيئة** عناصرها لاحقاً بشكل منفصل.
- يمكنك **تعريف وتهيئة** المصفوفة بقيم ابتدائية في سطر واحد لتوفير الوقت.

المصفوفات والسلاسل النصية — المصفوفات متعددة الأبعاد و Jagged Arrays

المصفوفات متعددة الأبعاد و Jagged Arrays في C#

يستعرض هذا الدرس نوعين مهمين من المصفوفات في C#: **المصفوفات متعددة الأبعاد** (Multidimensional Arrays)، التي تسمح بتخزين البيانات في شكل شبكة أو جدول، و**المصفوفات المتعرجة** (Jagged Arrays)، وهي مصفوفات من المصفوفات يمكن أن تحتوي على صفوف بأطوال مختلفة.

المصفوفات متعددة الأبعاد (Multidimensional Arrays)

هي مصفوفات تحتوي على أكثر من بُعد واحد (مثل الصفوف والأعمدة). تُستخدم لتمثيل البيانات في شكل شبكة، مثل مصفوفة أو خريطة، حيث يتم تحديد كل عنصر بزوج من الفهارس أو أكثر.

المصفوفات المتعرجة (Jagged Arrays)

هي مصفوفات من المصفوفات (array of arrays). الفرق الرئيسي هو أن كل مصفوفة داخلية يمكن أن يكون لها حجم أو طول مختلف، مما يجعلها مرنة جدًا لتخزين مجموعات بيانات ذات أحجام غير متساوية.

مثال برمجي

```
// تعريف وتهيئة مصفوفة ثنائية الأبعاد (2D Array)
int[,] grid = { {10, 20}, {30, 40}, {50, 60} };
Console.WriteLine($"عنصر في المصفوفة متعددة الأبعاد: {grid[1, 0]}"); // 30

// تعريف وتهيئة مصفوفة متعرجة (Jagged Array)
int[][] studentsScores = new int[3][];
studentsScores[0] = new int[] {85, 90, 78};
studentsScores[1] = new int[] {92, 88};
studentsScores[2] = new int[] {70, 75, 80, 85};
```

```
Console.WriteLine($"عنصر في المصفوفة المتعرجة: {studentsScores[0][2]}"); /
```

شرح الكود

يقوم الكود أولاً بتعريف مصفوفة ثنائية الأبعاد **grid** بثلاثة صفوف وعمودين وتخصيص قيم لها. ثم يقوم بالوصول إلى العنصر الموجود في الصف الثاني والعمود الأول (قيمة 30) وطباعته. بعد ذلك، يعرف مصفوفة متعرجة **studentsScores**، حيث لكل صف (مصفوفة داخلية) عدد مختلف من العناصر، ثم يصل إلى العنصر الثالث في الصف الأول (قيمة 78) ويطبعه.

نقاط رئيسية

- **المصفوفات متعددة الأبعاد** (Multidimensional) منظمة كشبكة (صفوف وأعمدة) وتحتاج لفهارس متعددة للوصول للعناصر.
- **المصفوفات المتعرجة** (Jagged) هي مصفوفات من المصفوفات، وكل مصفوفة داخلية يمكن أن يكون لها طول مختلف.
- المرونة هي الميزة الأساسية للمصفوفات المتعرجة، حيث توفر استهلاكاً أفضل للذاكرة عندما تكون أحجام البيانات غير متساوية.

المصفوفات والسلاسل النصية — العمليات الأساسية على النصوص

العمليات الأساسية على النصوص في #C

تُعد النصوص (Strings) جزءًا لا يتجزأ من أي تطبيق برمجي، فهي تمثل البيانات النصية التي نتعامل معها يوميًا. في هذا الدرس، سنتعرف على كيفية تعريف النصوص، دمجها، ومعرفة طولها في لغة #C.

مفاهيم أساسية

النصوص في #C هي كائنات لا يمكن تغييرها (immutable). يمكن تعريفها باستخدام الكلمة المفتاحية `string`، وتُكتب قيمتها بين علامتي اقتباس مزدوجة. من العمليات الشائعة عليها: دمج النصوص، الحصول على طولها، وتحويل حالة الأحرف.

مثال عملي

```
string firstName = "أحمد";
string lastName = "علي";

// دمج النصوص باستخدام عامل +
string fullName = firstName + " " + lastName;

// الحصول على طول النص
int nameLength = fullName.Length;

// تحويل النص إلى أحرف صغيرة
string lowerCaseName = fullName.ToLower();

Console.WriteLine($"الاسم الكامل: {fullName}");
Console.WriteLine($"حرف {nameLength} طول الاسم");
Console.WriteLine($"الاسم بأحرف صغيرة: {lowerCaseName}");
```

شرح الكود

يبدأ الكود بتعريف نصين هما `firstName` و `lastName`. ثم نستخدم عامل الدمج `+` لربط الاسمين مع إضافة مسافة بينهما لتكوين `fullName`. بعد ذلك، نستخدم الخاصية `Length` للحصول على عدد الأحرف في الاسم الكامل، ونخزنها في المتغير `nameLength`. أخيرًا، نظهر كيفية تحويل النص إلى أحرف صغيرة باستخدام الدالة `ToLower()`، ثم نطبع النتائج باستخدام `Console.WriteLine`.

- النصوص (Strings) في #C تُعرّف بـ `string` وقيمتها بين علامتي اقتباس مزدوجة.
- يمكن دمج النصوص بسهولة باستخدام العامل `+`.
- تُستخدم الخاصية `Length` للحصول على عدد الأحرف في النص، بينما `ToLower()` تحول النص إلى أحرف صغيرة.

المصفوفات والسلاسل النصية — StringBuilder

StringBuilder

في #C، عندما تتعامل مع السلاسل النصية التي تحتاج إلى تعديلات متكررة (مثل الإضافة أو الحذف)، فإن استخدام نوع `string` العادي قد يكون غير فعال. هنا يأتي دور `StringBuilder` ليقدم حلاً أفضل وأكثر كفاءة.

مفهوم StringBuilder

السلاسل النصية (`string`) في #C **غير قابلة للتغيير** (immutable)، وهذا يعني أن كل عملية تعديل (مثل الإضافة أو الدمج) تنشئ سلسلة نصية جديدة في الذاكرة. على عكس ذلك، فإن `StringBuilder` **قابل للتغيير** (mutable)، مما يسمح لك بتعديل السلسلة النصية في مكانها دون الحاجة إلى إنشاء كائنات جديدة باستمرار، مما يحسن الأداء ويقلل استهلاك الذاكرة بشكل ملحوظ عند إجراء عمليات كثيرة.

مثال برمجي

```
using System;
using System.Text;

public class Program
{
    public static void Main(string[] args)
    {
        StringBuilder sb = new StringBuilder();
        sb.Append("مرحبا");
        sb.Append(" بكم");
        sb.Append(" في");
        sb.Append(" عالم");
        sb.Append(" C#!");

        Console.WriteLine(sb.ToString());
    }
}
```

شرح الكود

يبدأ الكود بإنشاء كائن جديد من `StringBuilder`. ثم يستخدم الدالة `Append` لإضافة أجزاء من النص تدريجياً إلى الكائن. بدلاً من إنشاء سلاسل نصية جديدة لكل عملية إضافة، يقوم `StringBuilder` بتعديل السلسلة النصية الداخلية بكفاءة. في النهاية، يتم تحويل محتوى `StringBuilder` إلى سلسلة نصية عادية باستخدام `ToString()` وعرضها على الشاشة.

- **تحسين الأداء:** يعتبر `StringBuilder` الخيار الأمثل لتعديل السلاسل النصية المتكرر أو بناء سلاسل نصية كبيرة.
- **قابلية التغيير:** يسمح بتعديل السلسلة النصية في مكانها، مما يقلل من النفقات العامة لإنشاء كائنات جديدة.
- **كفاءة الذاكرة:** يقلل بشكل فعال من استهلاك الذاكرة مقارنة بالعمليات المتعددة على نوع `string`.

المصفوفات والسلاسل النصية — Immutable Strings ولماذا هي مهمة

السلاسل النصية غير القابلة للتغيير (Immutable Strings) ولماذا هي مهمة في #C

في #C، تُعد السلاسل النصية (strings) من الأنواع الأساسية التي نستخدمها باستمرار. لكن ما قد لا يدركه الكثيرون هو أن هذه السلاسل تتمتع بخاصية مهمة جداً تسمى "عدم القابلية للتغيير" (Immutability). فهم هذه الخاصية ضروري لكتابة كود فعال وخالي من الأخطاء.

ما هي السلاسل النصية غير القابلة للتغيير؟

تعني السلاسل النصية غير القابلة للتغيير أن كائن الـ `string` بمجرد إنشائه لا يمكن تعديل محتواه الأصلي. أي عملية تبدو وكأنها تعديل (مثل الإضافة أو الاستبدال) لا تقوم بتغيير السلسلة الأصلية، بل تنشئ سلسلة نصية جديدة تماماً في الذاكرة.

مثال برمجي

```
string greeting = "مرحبا";
Console.WriteLine(greeting); // Output: مرحبا

greeting = greeting + " !بك يا عالم";
Console.WriteLine(greeting); // Output: !مرحبا بك يا عالم

string name = "ahmed";
name = name.ToUpper();
Console.WriteLine(name); // Output: AHMED
```

شرح الكود

في المثال الأول، قمنا بتعيين القيمة "مرحبا" للمتغير `greeting`. عندما أضفنا إليه " بك يا عالم!"، لم يتغير النص الأصلي "مرحبا". بدلاً من ذلك، تم إنشاء سلسلة نصية جديدة "مرحبا بك يا عالم!"، وأصبح المتغير `greeting` يشير إليها. كذلك الحال مع المتغير `name`، حيث تقوم الدالة `ToUpper()` بإنشاء سلسلة نصية جديدة بأحرف كبيرة بدلاً من تعديل السلسلة الأصلية.

- **الثبات:** تعني السلسلة النصية غير القابلة للتغيير أن قيمتها لا تتغير أبداً بعد إنشائها.
- **إنشاء كائنات جديدة:** أي عملية تعديل ظاهرية على السلسلة النصية تنشئ فعلياً كائناً جديداً في الذاكرة.
- **الأهمية:** هذه الخاصية تجعل السلاسل النصية آمنة للاستخدام في البيئات متعددة المهام (thread-safe) وتضمن سلوكاً يمكن التنبؤ به.

الدوال والطرق (Methods) — تعريف الدوال والاستدعاء

الدوال والطرق (Methods) في C#: تعريف الدوال والاستدعاء

تُعد الدوال (أو الطرق Methods) من المفاهيم الأساسية في البرمجة، فهي تُمكنك من تنظيم الكود الخاص بك في كتل قابلة لإعادة الاستخدام. في هذا الدرس، سنتعلم كيفية تعريف الدالة وإنشائها، ثم كيفية استدعائها لتنفيذ المهمة المخصصة لها.

تعريف الدالة (Defining a Method)

تعريف الدالة يعني إخبار C# باسم الدالة، وما نوع القيمة التي سترجعها (إن وُجدت)، وأي معلومات تحتاجها للعمل (المعاملات)، وما هو الكود الذي ستنفذه. هذا يُشبه كتابة وصفة طبخ لخطوات معينة.

استدعاء الدالة (Calling a Method)

استدعاء الدالة يعني تشغيل الكود الموجود بداخلها. بعد تعريف الدالة، يمكنك استدعاؤها في أي مكان تحتاج فيه إلى تنفيذ مهمتها، مما يوفر عليك كتابة نفس الكود مراراً وتكراراً ويجعل برنامجك أكثر تنظيماً.

مثال الكود

```
using System;

public class Program
{
    // تعريف دالة بسيطة لطباعة رسالة ترحيب
    public static void SayHello(string name)
    {
        Console.WriteLine($"مرحباً، {name}!");
    }

    public static void Main(string[] args)
```

```

{
    // استدعاء الدالة SayHello
    SayHello("عالم البرمجة");
    SayHello("يا صديقي");
}
}

```

شرح الكود

في هذا المثال، قمنا أولاً **بتعريف** دالة اسمها `SayHello`. هذه الدالة لا تُرجع أي قيمة (`void`) وتستقبل معاملاً واحدًا من نوع نصي (`string name`). مهمتها هي طباعة رسالة ترحيب مخصصة باستخدام الاسم الذي يتم تمريره إليها.

في الدالة الرئيسية `Main`، قمنا **باستدعاء** الدالة `SayHello` مرتين، ومررنا في كل مرة اسمًا مختلفًا ("عالم البرمجة" ثم "يا صديقي"). عند استدعائها، يتم تنفيذ الكود بداخلها وطباعة رسائل الترحيب في شاشة الكونسول.

- **تعريف الدالة:** هو بناء الدالة بتحديد اسمها، نوع إرجاعها، ومعاملاتها، والتعليمات التي تنفذها.
- **استدعاء الدالة:** هو تنفيذ الكود الموجود داخل الدالة باستخدام اسمها وتمرير القيم المطلوبة (إن وُجدت).
- الدوال تجعل الكود منظمًا، سهل القراءة، وقابلًا لإعادة الاستخدام، مما يقلل من تكرار الكود ويحسن صيانتة.

الدوال والطرق (Methods) — Parameters (Value, ref, out, in)

الدوال والطرق: المعاملات (Parameters) بقيمة، in، out، ref

عند استدعاء دالة (Method)، غالبًا ما تحتاج هذه الدالة إلى بيانات للعمل عليها. هذه البيانات تسمى "معاملات" (Parameters). في C#، توجد طرق مختلفة لتمرير هذه المعاملات، تؤثر على كيفية تعامل الدالة مع البيانات الأصلية خارج نطاقها.

أنواع تمرير المعاملات

تحدد طريقة تمرير المعامل ما إذا كانت الدالة تعمل على نسخة من البيانات أو على البيانات الأصلية مباشرة: **By Value (تلقائي)**: تمرر نسخة من القيمة، ولا تؤثر التغييرات على المتغير الأصلي. **ref**: تمرر مرجعًا للمتغير الأصلي. التغييرات داخل الدالة تؤثر على المتغير الأصلي. يجب تهيئة المتغير قبل تمريره. **out**: تمرر مرجعًا للمتغير الأصلي. لا يتطلب تهيئة قبل التمرير، ولكن يجب على الدالة تعيين قيمة له قبل انتهائها. **in**: تمرر مرجعًا للقراءة فقط للمتغير الأصلي. لا يمكن للدالة تعديل قيمة هذا المتغير.

مثال الكود

```
using System;

public class Program
{
    static void ModifyValues(int valParam, ref int refParam, out int outParam)
    {
        valParam = 100;
        refParam = 200;
        outParam = 300;
        // inParam = 400; // هذا السطر سيسبب خطأ في الترجمة (compile error)
        Console.WriteLine($"Inside method: valParam={valParam}, refParam={refParam}, outParam={outParam}");
    }
}
```

```

public static void Main(string[] args)
{
    int x = 1;
    int y = 2;
    int z; // 'out' غير مهيأة، سيتم تهيئتها بواسطة
    int w = 4;

    Console.WriteLine($"Before call: x={x}, y={y}, w={w}");
    ModifyValues(x, ref y, out z, w);
    Console.WriteLine($"After call: x={x}, y={y}, z={z}, w={w}");
}
}

```

شرح الكود

قبل استدعاء الدالة `ModifyValues`، كانت قيم `y=2`، `x=1`، و `w=4`. المتغير `z` لم يكن مهيئاً. داخل الدالة:

- `valParam` (يمثل `x`) تغير إلى 100، لكن هذا التغير لم يؤثر على `x` الأصلية.
- `refParam` (يمثل `y`) تغير إلى 200، مما أثر على `y` الأصلية.
- `outParam` (يمثل `z`) تم تعيين قيمته إلى 300، مما أثر على `z` الأصلية.
- `inParam` (يمثل `w`) احتفظ بقيمته 4، ولا يمكن تعديله داخل الدالة.

بعد انتهاء الدالة، نجد أن `x` بقيت 1، بينما `y` أصبحت 200، و `z` أصبحت 300، و `w` بقيت 4.

- تمرير المعاملات بالقيمة (By Value) يرسل نسخة، فلا تتأثر القيمة الأصلية بالمتغيرات داخل الدالة.
- `ref` و `out` يمرران مرجعاً للمتغير الأصلي، مما يسمح بتعديله داخل الدالة. `out` لا يتطلب تهيئة مسبقة.
- `in` يضمن أن المعامل لا يمكن تغييره داخل الدالة، مما يحمي البيانات الأصلية ويزيد من وضوح الكود.

الدوال والطرق (Methods) — Return Values

قيم الإرجاع (Return Values)

في C#، لا تقوم الدوال والطرق (Methods) بتنفيذ الأوامر فحسب، بل يمكنها أيضاً معالجة البيانات ثم **إرجاع** نتيجة هذه المعالجة إلى الجزء الذي استدعاها. هذه النتيجة تسمى "قيمة الإرجاع".

مفهوم قيم الإرجاع

عندما تقوم دالة بعملية حسابية أو تجلب معلومة، يمكنها أن "تعيد" هذه المعلومة كنتاج. نوع هذه القيمة المرتجعة يجب أن يتم تحديده قبل اسم الدالة في تعريفها. إذا لم ترجع الدالة أي قيمة، فإن نوع الإرجاع يكون `void`.

مثال برمجي

```
public class Calculator
{
    public int Add(int num1, int num2)
    {
        int sum = num1 + num2;
        return sum; // إرجاع القيمة المجمعة
    }

    public string GetGreeting(string name)
    {
        return "مرحباً بك يا " + name + "!"; // إرجاع نص ترحيب
    }
}

// مثال على كيفية استخدامها:
// Calculator calc = new Calculator();
// int result = calc.Add(5, 3); // result ستكون 8
// string greeting = calc.GetGreeting("أحمد"); // greeting سيكون مرحباً بك يا أحمد
```

شرح الكود

في المثال أعلاه، الدالة `Add` من نوع `int` قبل اسمها، مما يعني أنها سترجع قيمة عدد صحيح (integer). داخل الدالة، يتم حساب المجموع ثم استخدام الكلمة المفتاحية `return` لإعادة هذا المجموع. أما الدالة `GetGreeting`، فترجع قيمة نصية (`string`) وهي رسالة ترحيب بعد دمجها بالاسم المُرر.

- تستخدم الكلمة المفتاحية `return` لإرسال قيمة من الدالة إلى الجزء الذي استدعاها.
- يحدد نوع البيانات المكتوب قبل اسم الدالة نوع القيمة التي سترجعها الدالة (مثل `int`, `string`, `bool`).
- إذا لم ترجع الدالة أي قيمة، فإن نوعها يُكتب `void`.

الدوال والطرق (Methods) — Method Overloading

الدوال والطرق: تحميل الزائد للدوال (Method Overloading)

في #C، تحميل الزائد للدوال (Method Overloading) هو ميزة قوية تسمح لنا بتعريف عدة دوال تحمل نفس الاسم داخل نفس الفئة (class)، ولكن كل دالة تؤدي وظيفة مختلفة أو تتعامل مع أنواع مختلفة من البيانات. هذا يجعل الكود أكثر مرونة ووضوحًا.

المفهوم الأساسي

يحدث تحميل الزائد عندما يكون لدينا أكثر من دالة بنفس الاسم في نفس الفئة (class). للتمييز بينها، يجب أن تختلف هذه الدوال في توقيعها (signature)، أي في عدد المعاملات (parameters)، أو أنواع هذه المعاملات، أو ترتيبها. بناءً على القيم التي تمررها عند استدعاء الدالة، يختار المترجم (compiler) الدالة الصحيحة لتنفيذها.

مثال برمجي

```
using System;

public class Calculator
{
    public void Add(int a, int b)
    {
        Console.WriteLine($"Sum of two integers: {a + b}");
    }

    public void Add(double a, double b)
    {
        Console.WriteLine($"Sum of two doubles: {a + b}");
    }

    public void Add(int a, int b, int c)
    {
        Console.WriteLine($"Sum of three integers: {a + b + c}");
    }
}
```

```
public static void Main(string[] args)
{
    Calculator calc = new Calculator();
    calc.Add(5, 10);
    calc.Add(3.5, 2.5);
    calc.Add(1, 2, 3);
}
}
```

شرح الكود

في هذا المثال، لدينا ثلاث دوال كلها تحمل نفس الاسم `Add`. الدالة الأولى تجمع عددين صحيحين، والثانية تجمع عددين عشريين (`double`)، بينما الثالثة تجمع ثلاثة أعداد صحيحة. على الرغم من أن أسماء الدوال متطابقة، فإن توقيع كل منها مختلف (عدد المعاملات أو أنواعها). عند استدعاء `calc.Add(...)`، يحدد مترجم `#C` تلقائيًا أي دالة `Add` يجب استخدامها بناءً على أنواع وعدد المعاملات التي تم تمريرها.

نقاط رئيسية

- يمكن لعدة دوال أن تشترك في نفس الاسم داخل الفئة (`class`) الواحدة.
- يجب أن تختلف الدوال المحملة زائدًا (`overloaded methods`) في توقيعها (عدد أو أنواع أو ترتيب المعاملات).
- يساعد تحميل الزائد على زيادة مرونة الكود وقابليته للقراءة، مما يسمح باستخدام نفس الاسم لأداء مهام متشابهة ولكن مع بيانات مختلفة.

الدوال والطرق (Methods) — Named & Optional Parameters

المعاملات المسماة والاختيارية في #C

عند التعامل مع الدوال والطرق في لغة #C، يمكننا استخدام المعاملات المسماة والاختيارية لجعل استدعاء الطرق أكثر وضوحًا ومرونة. تساعد هذه الميزات في تحسين قراءة الكود وتقليل الحاجة إلى تعريف طرق متعددة للحالات المختلفة.

المعاملات المسماة (Named Parameters)

تسمح لك المعاملات المسماة بتمرير قيم الوسائط إلى الدالة باستخدام اسمها بدلاً من ترتيبها. هذا يحسن من وضوح الكود، خاصةً عند وجود عدد كبير من المعاملات أو عندما تكون أنواع البيانات متشابهة، مما يجعل الكود أسهل للقراءة والفهم.

المعاملات الاختيارية (Optional Parameters)

يمكنك تحديد قيمة افتراضية للمعاملات في تعريف الدالة، مما يجعلها اختيارية عند استدعاء الدالة. إذا لم يتم تمرير قيمة لهذا المعامل، فسيتم استخدام قيمته الافتراضية، مما يوفر مرونة كبيرة ويقلل من عدد الدوال الزائدة.

مثال الكود

```
public class Greet
{
    public void SayHello(string name, string message = "أهلاً بك", int repeatCount)
    {
        for (int i = 0; i < repeatCount; i++)
        {
            Console.WriteLine($"{message}, {name}!");
        }
    }
}
```

```

    }

    public static void Main(string[] args)
    {
        Greet greeter = new Greet();

        // 1. استخدام عادي (المعاملات الاختيارية بقيمتها الافتراضية)
        greeter.SayHello("أحمد");

        // 2. استخدام المعاملات المسماة (ترتيب مرن)
        greeter.SayHello(message: "مرحباً", name: "ليلي");

        // 3. استخدام المسماة والاختيارية (تجاوز القيم الافتراضية)
        greeter.SayHello("سارة", repeatCount: 3, message: "صباح الخير");
    }
}

```

شرح الكود

في هذا المثال، لدينا دالة `SayHello` التي تأخذ اسماً، ورسالة (اختيارية بقيمة افتراضية "أهلاً بك")، وعدد مرات تكرار (اختياري بقيمة افتراضية 1).

1. الاستدعاء الأول يمرر الاسم فقط، وتستخدم القيم الافتراضية للرسالة وعدد التكرار.
2. الاستدعاء الثاني يستخدم **المعاملات المسماة** لتمرير الرسالة والاسم بترتيب مختلف.
3. الاستدعاء الثالث يمرر الاسم بشكل مباشر، ثم يستخدم **المعاملات المسماة** لتجاوز القيم الافتراضية لكل من `repeatCount` و `message`.

- **الوضوح:** تجعل المعاملات المسماة استدعاء الدالة أسهل في القراءة والفهم، خصوصاً مع المعاملات المتعددة.
- **المرونة:** تسمح المعاملات الاختيارية باستخدام الدالة بأشكال مختلفة دون الحاجة لتعريف دالات متعددة أو تمرير قيم وهمية.
- **الدمج:** يمكنك دمج المعاملات المسماة والاختيارية لإنشاء استدعاءات دالة قوية ومرنة، مع الحفاظ على الكود نظيفاً وسهل الصيانة.

الدوال والطرق (Methods) — Local Functions

الدوال المحلية (Local Functions)

الدوال المحلية هي ميزة قوية في #C تسمح لك بتعريف دوال داخل دوال أخرى. هذه الميزة تساعد في تنظيم الكود وتحسين قراءته، خاصة عندما تحتاج لدالة مساعدة تُستخدم في سياق محدد جداً.

ما هي الدوال المحلية؟

هي دوال تُعلن داخل جسم عضو تعبيرى آخر (مثل دالة عادية أو مُنشئ). وظيفتها الأساسية هي المساعدة في تنظيم الكود وجعله أكثر قابلية للقراءة، خصوصاً عندما تحتاج لدالة مساعدة تستخدم مرة واحدة فقط ضمن نطاق معين ولا ترغب في جعلها متاحة لباقي أجزاء الكلاس.

مثال عملي

```
public class DataProcessor
{
    public string ProcessText(string text)
    {
        // Local function to standardize text
        string Standardize(string input)
        {
            return input.Trim().ToLower();
        }

        if (string.IsNullOrEmpty(text))
        {
            return "النص فارغ";
        }

        string standardizedText = Standardize(text);
        return $"تم معالجة النص: {standardizedText}";
    }
}
```

شرح الكود

في هذا المثال، لدينا الدالة الرئيسية `ProcessText` التي تعالج نصاً معيناً. بداخلها، قمنا بتعريف دالة محلية تسمى `Standardize`. مهمة هذه الدالة المحلية هي إزالة المسافات البيضاء من بداية ونهاية النص وتحويله إلى أحرف صغيرة. لاحظ أن الدالة `Standardize` لا يمكن استدعاؤها إلا من داخل الدالة `ProcessText` التي تحتويها، مما يجعل الكود منظماً ومحدداً.

- تُعرّف الدوال المحلية داخل دالة أخرى.
- لا يمكن الوصول إليها إلا من الدالة التي تحتويها.
- تساعد في تنظيم الكود وتحسين قراءته من خلال تجميع المهام ذات الصلة.

المجالات (Scopes) وإدارة الذاكرة — نطاق المتغيرات

نطاق المتغيرات (Variable Scope)

في C#, يشير **نطاق المتغير** إلى المنطقة في برنامجك حيث يمكن الوصول إلى المتغير أو استخدامه. فهم النطاق أمر أساسي لكتابة كود منظم وفعال وإدارة الذاكرة بشكل صحيح.

المفهوم الأساسي

نطاق المتغير يحدد "صلاحية" المتغير. المتغيرات يمكن أن يكون لها نطاق محلي (داخل دالة أو كتلة كود) أو نطاق حقل (داخل صنف). عندما يخرج المتغير من نطاقه، لا يمكن الوصول إليه وتُفقد قيمته، مما يسمح لنظام جمع القمامة (Garbage Collector) بتحرير الذاكرة التي كان يشغلها.

مثال برمجي

```
using System;

public class ScopeExample
{
    public static void Main(string[] args)
    {
        int x = 10; // Main هو داخل الدالة x نطاق

        if (true)
        {
            int y = 20; // if هو داخل كتلة y نطاق
            Console.WriteLine($"if داخل {y}");
        }

        Console.WriteLine($"Main داخل {x}");
        // Console.WriteLine($"if خارج {y}"); // سيسبب خطأ y: هذا النطاق
    }
}
```

شرح الكود

في هذا المثال، المتغير `x` تم تعريفه داخل الدالة `Main`، لذا يمكن الوصول إليه في أي مكان داخل هذه الدالة. أما المتغير `y`، فقد تم تعريفه داخل كتلة `if`. هذا يعني أنه يمكن الوصول إليه واستخدامه فقط داخل تلك الكتلة. إذا حاولت الوصول إلى `y` خارج كتلة `if` (كما يوضح السطر المعلق)، فسيحدث خطأ في الترجمة لأن `y` ليس في نطاق الوصول.

- نطاق المتغير يحدد أين يمكن استخدام المتغير في الكود.
- المتغيرات المعرفة داخل كتلة كود (مثل دالة، حلقة، أو جملة `if`) تكون مرئية فقط داخل تلك الكتلة.
- عندما يخرج المتغير من نطاقه، لا يمكن الوصول إليه ويتم إدارته بواسطة نظام جمع القمامة لتحرير الذاكرة.

المجالات (Scopes) وإدارة الذاكرة — Garbage Collector

جامع القمامة (Garbage Collector)

في لغة #C، يعد **جامع القمامة (Garbage Collector)** جزءًا أساسيًا من نظام إدارة الذاكرة الأوتوماتيكي. وظيفته هي تحرير الذاكرة التي لم تعد الكائنات تستخدمها، مما يساعد المبرمجين على التركيز على منطق التطبيق بدلاً من إدارة الذاكرة يدويًا وتقليل الأخطاء.

كيف يعمل جامع القمامة؟

يعمل جامع القمامة عن طريق تحديد الكائنات التي لم يعد بالإمكان الوصول إليها أو استخدامها في البرنامج. بمجرد تحديد هذه الكائنات "الميتة"، يقوم بتحرير الذاكرة التي كانت تشغلها، مما يجعلها متاحة للاستخدام من قبل كائنات جديدة عند الحاجة.

مثال توضيحي

```
class LargeObject
{
    public int[] data;
    public LargeObject(int size)
    {
        data = new int[size];
        Console.WriteLine($"LargeObject created with size {size}.");
    }
    // A finalizer method to demonstrate when the object is collected
    ~LargeObject()
    {
        Console.WriteLine($"LargeObject collected by GC.");
    }
}

class Program
{
```

```

static void CreateAndForget()
{
    LargeObject obj = new LargeObject(1000000); // Creates a large object
    // 'obj' goes out of scope when this method finishes.
}

static void Main(string[] args)
{
    Console.WriteLine("Program starts...");
    CreateAndForget();
    Console.WriteLine("CreateAndForget method finished.");
    // The Garbage Collector will eventually clean up 'obj'.
    // This usually happens automatically without explicit calls.
    Console.WriteLine("Program ends.");
}
}

```

شرح الكود

في هذا المثال، تقوم الدالة `CreateAndForget` بإنشاء كائن من نوع `LargeObject`. بمجرد انتهاء هذه الدالة، يصبح الكائن `obj` خارج نطاق الوصول (out of scope)، ولا توجد أي مراجع أخرى تشير إليه. في هذه المرحلة، يصبح الكائن مؤهلاً للجمع بواسطة جامع القمامة، الذي سيقوم بتحرير الذاكرة المخصصة له تلقائيًا في وقت لاحق عندما يقرر ذلك.

- **إدارة ذاكرة تلقائية:** يحرر جامع القمامة الذاكرة بشكل تلقائي دون تدخل المبرمج.
- **الكشف عن الكائنات غير المستخدمة:** يحدد الكائنات التي لم تعد مرجعًا لها ويستعيد مساحتها.
- **تقليل تسرب الذاكرة:** يمنع تسرب الذاكرة ويحسن أداء التطبيق واستقراره على المدى الطويل.

المجالات (Scopes) وإدارة الذاكرة — Stack vs Heap

المجالات (Scopes) وإدارة الذاكرة: Stack vs Heap

في لغة #C، يعد فهم كيفية إدارة الذاكرة أمرًا حيويًا لكتابة برامج فعالة وخالية من الأخطاء. الذاكرة الرئيسية مقسمة إلى منطقتين أساسيتين: **المجال التكدسي (Stack)** و**المجال الكومي (Heap)**. معرفة الفرق بينهما يساعد على فهم سلوك المتغيرات والكائنات.

المجال التكدسي (Stack) والمجال الكومي (Heap)

الـ **Stack** هي منطقة ذاكرة سريعة ومنظمة (Last-In, First-Out - LIFO) تُستخدم لتخزين الأنواع ذات القيمة (Value Types) مثل الأعداد الصحيحة (int)، والمتغيرات المحلية (local variables)، وعناوين الرجوع للدوال. أما الـ **Heap** فهي منطقة ذاكرة أكبر وأكثر مرونة ولكنها أبطأ، وتُستخدم لتخزين الأنواع المرجعية (Reference Types) مثل الكائنات (objects) ومثيلات الفئات (class instances). تتم إدارة الـ Heap بواسطة جامع القمامة (Garbage Collector).

مثال على الكود (Code Example)

```
public struct Point
{
    public int X;
    public int Y;
}

public class MyObject
{
    public int Value;
}

public class MemoryDemo
{
    public static void Main()
    {
        // Value Type (Stack)
```

```

int number = 10;
Point p = new Point { X = 5, Y = 10 };

// Reference Type (Heap)
MyObject obj1 = new MyObject { Value = 100 };
MyObject obj2 = obj1; // obj2 refers to the same object on Heap
obj2.Value = 200; // Changes obj1.Value too
}
}

```

شرح الكود

في هذا المثال: المتغير `number` من نوع `int` والمتغير `p` من نوع `struct Point` يتم تخزينهما مباشرة في الـ **Stack**، لأن كلاهما من الأنواع ذات القيمة (Value Types). أما المتغير `obj1` فهو متغير مرجعي (reference variable) يتم تخزينه في الـ **Stack**، ولكنه يشير إلى كائن من نوع `MyObject` تم إنشاؤه وتخزينه في الـ **Heap**. عندما نقوم بتعيين `obj1` إلى `obj2`، فإن `obj2` يصبح مرجعًا آخر يشير إلى نفس الكائن الموجود في الـ **Heap**، ولذلك فإن تغيير `obj2.Value` يؤثر أيضًا على `obj1.Value`.

- **الـ Stack:** تخزن الأنواع ذات القيمة (Value Types) والمتغيرات المحلية وعناوين الدوال، وهي سريعة ومحدودة الحجم.
- **الـ Heap:** تخزن الأنواع المرجعية (Reference Types) مثل الكائنات، وهي أكبر وأبطأ وتتم إدارتها بواسطة جامع القمامة (GC).
- فهم Stack vs Heap ضروري لتحسين أداء الذاكرة وتجنب الأخطاء المتعلقة بالتحكم في الكائنات.

مبادئ OOP – الجزء الأول — تعريف الفئات والكائنات

تعريف الفئات والكائنات (Classes and Objects)

مرحبًا بك في مبادئ البرمجة الشيئية (OOP)! في هذا الدرس الأول، سنتعلم عن مفهومين أساسيين يشكلان حجر الزاوية في بناء البرامج الحديثة: الفئات (Classes) والكائنات (Objects). فهما هو مفتاح لفهم كيفية تنظيم وهيكلة الكود.

ما هي الفئات والكائنات؟

الفئة (Class): هي قالب أو مخطط (Blueprint) يُستخدم لإنشاء الكائنات. تحدد الفئة الخصائص (مثل: اللون، الحجم) والسلوكيات (مثل: التحرك، التحدث) التي ستمتلكها الكائنات التي تنشأ منها. تخيلها كقالب كعكة.

الكائن (Object): هو نسخة حقيقية ومحددة (Instance) من الفئة. هو تجسيد ملموس للمخطط. الكائن يمتلك قيمًا فعلية لخصائصه ويمكنه أداء السلوكيات المحددة في الفئة. تخيلها كالكعكة الفعلية المصنوعة من القالب.

مثال الكود

```
public class Car
{
    public string Model;
    public int Year;

    public void StartEngine()
    {
        // Logic to start the car engine
    }
}

// How to create an object from the Car class
/*
```

```
public static void Main(string[] args)
{
    Car myCar = new Car(); // Creating an object
    myCar.Model = "Toyota Camry"; // Setting a property
    myCar.Year = 2023; // Setting another property
    myCar.StartEngine(); // Calling a method (behavior)
}
*/
```

شرح الكود

في هذا المثال، قمنا بتعريف **فئة** اسمها **Car**. هذه الفئة تمثل "مخطط" لكل السيارات، وتحدد أن كل سيارة سيكون لها **Model** (نموذج) و **Year** (سنة الصنع) كخصائص.

كما أن لديها **StartEngine()** وهو سلوك (Method) يشير إلى إمكانية تشغيل محرك السيارة.

الجزء المحاط بالتعليقات يوضح كيف ننشئ **كائنًا** حقيقيًا اسمه **myCar** من هذه الفئة. الكائن **myCar** هو سيارة فعلية يمكننا تحديد خصائصها مثل `myCar.Model`` واستدعاء سلوكياتها مثل `myCar.StartEngine`()`.

- **الفئة (Class)** هي قالب يصف ما سيكون عليه الكائن (الخصائص والسلوكيات).
- **الكائن (Object)** هو نسخة واقعية وحية من الفئة، بخصائص وقيم محددة.
- يمكنك إنشاء العديد من الكائنات المختلفة والمستقلة من نفس الفئة الواحدة.

مبادئ OOP – الجزء الأول — الحقول والخصائص

(Fields vs Properties)

الحقول والخصائص (Fields vs Properties)

في هذا الدرس، سنتعمق في فهم مفهومين أساسيين في لغة #C وهما **الحقول (Fields)** و**الخصائص (Properties)**. كلاهما يستخدم لتخزين البيانات داخل الفئات (classes)، ولكن الفارق الجوهرى يكمن في طريقة الوصول والتحكم في هذه البيانات، وهو ما يعتبر جزءًا حيويًا من مبادئ البرمجة الكائنية التوجه (OOP).

المفهوم الأساسي

الحقول هي متغيرات تُعلن مباشرة داخل الفئة لتخزين حالتها الداخلية، ويمكن الوصول إليها وتعديلها بشكل مباشر. أما **الخصائص**، فهي توفر آلية آمنة ومرنة للوصول إلى الحقول (الخاصة عادةً) وتعديلها عبر دالتي **get** (للقراءة) و**set** (للكتابة)، مما يتيح لنا تطبيق منطق إضافي مثل التحقق من الصحة.

مثال توضيحي

```
public class Student
{
    // وصول مباشر - (Field) حقل
    public string Name;

    // وصول متحكم به - (Property) خاصية
    private int _grade; // حقل خاص (backing field)
    public int Grade
    {
        get { return _grade; }
        set
        {
            if (value >= 0 && value <= 100)
            {
                _grade = value;
            }
        }
    }
}
```

```
    }  
    else  
    {  
        System.Console.WriteLine("الدرجة يجب أن تكون بين 0 و 100");  
    }  
}  
}  
}
```

شرح الكود

في الكود أعلاه، `Name` هو **حقل** عام يمكن لأي جزء من الكود الوصول إليه وتغيير قيمته مباشرة دون قيود. أما `Grade` فهي **خاصية** تتحكم في قيمة الحقل الخاص `grade_`. من خلال الدالة `set`، نضمن أن تكون قيمة الدرجة دائمًا بين 0 و 100 قبل تعيينها، مما يحمي البيانات من القيم غير الصالحة. تسمح الدالة `get` بقراءة قيمة الدرجة المخزنة بأمان.

نقاط رئيسية

- الحقول توفر وصولاً مباشرًا للبيانات، بينما الخصائص توفر وصولاً متحكمًا به عبر دوال القراءة والكتابة.
- استخدم الخصائص عندما تحتاج إلى تطبيق منطق إضافي (مثل التحقق من الصحة أو التنسيق) عند قراءة أو كتابة البيانات.
- تساعد الخصائص في تحقيق مبدأ التغليف (Encapsulation)، وهو أساسي في البرمجة الكائنية، عن طريق إخفاء تفاصيل التنفيذ الداخلي.

مبادئ OOP – الجزء الأول — البواني (Constructors)

البواني (Constructors)

في البرمجة كائنية التوجه (OOP)، تُعد **البواني** جزءاً أساسياً لإنشاء الكائنات وتهيئتها. تخيلها كبوابة الدخول التي تُعد الكائن لاستخدامه فور إنشائه.

ما هي البواني؟

الباني (Constructor) هي دالة خاصة داخل الفئة (Class) تُستدعى تلقائياً عند إنشاء كائن جديد من تلك الفئة. وظيفتها الرئيسية هي تهيئة حالة الكائن، أي تعيين القيم الأولية لخصائصه (Properties).

مثال برمجي

```
public class Car
{
    public string Model { get; set; }
    public int Year { get; set; }

    // هذا هو الباني (Constructor)
    public Car(string model, int year)
    {
        Model = model;
        Year = year;
    }

    public void DisplayInfo()
    {
        Console.WriteLine($"Model: {Model}, Year: {Year}");
    }
}

// طريقة استخدام الباني
public class Program
{
    public static void Main(string[] args)
```

```
{
    Car myCar = new Car("BMW", 2023); // إنشاء الكائن
    myCar.DisplayInfo(); // Output: Model: BMW, Year: 2023
}
}
```

شرح الكود

في هذا المثال، لدينا فئة اسمها `Car`. الباني `public Car(string model, int year)` يأخذ معلمات (parameters) لنموذج السيارة وسنة الصنع. عند إنشاء كائن جديد مثل `myCar` باستخدام `new` (`Car("BMW", 2023)`، يتم استدعاء هذا الباني تلقائياً لتعيين قيمتي `Model` و `Year` مباشرة، مما يضمن أن الكائن يبدأ بحالة صحيحة ومُهَيَّأَة.

- **تهيئة الكائنات:** البواني هي دوال خاصة تُستخدم لتهيئة الكائنات عند إنشائها، لضمان حالتها الأولية الصحيحة.
- **نفس اسم الفئة:** يجب أن يحمل الباني نفس اسم الفئة التي ينتمي إليها تماماً.
- **لا نوع إرجاع:** لا تحتوي البواني على أي نوع إرجاع (حتى `void`) لأنها لا تُرجع قيمة، بل مهمتها بناء الكائن.

مبادئ OOP – الجزء الأول — Encapsulation ومعدلات الوصول

Encapsulation ومعدلات الوصول

في هذا الدرس، سنتعمق في مفهومين أساسيين من مبادئ البرمجة كائنية التوجه (OOP) في C#: التغليف (Encapsulation) ومعدلات الوصول (Access Modifiers). فهما ضروري لبناء برمجيات قوية ومنظمة.

التغليف (Encapsulation)

التغليف هو مبدأ يجمع البيانات (الخصائص) والوظائف (الأساليب) التي تعمل على هذه البيانات ضمن وحدة واحدة (كائن أو صنف). يهدف إلى إخفاء التفاصيل الداخلية لتطبيق الكائن عن العالم الخارجي، والتحكم في كيفية الوصول إلى هذه البيانات أو تعديلها. فكر فيها كـ "صندوق أسود" حيث لا تحتاج لمعرفة كيف يعمل داخليًا لتستخدمه.

معدلات الوصول (Access Modifiers)

معدلات الوصول هي كلمات مفتاحية في C# تحدد مستوى رؤية وإمكانية الوصول إلى الأصناف، الأعضاء (مثل الخصائص والأساليب)، وغيرها. هي أدواتنا لتحقيق مبدأ التغليف. أهمها لتحقيق التغليف هي:

- **private**: العضو لا يمكن الوصول إليه إلا من داخل الصنف الذي تم تعريفه فيه فقط.
- **public**: العضو يمكن الوصول إليه من أي مكان دون قيود.

مثال عملي

```
public class BankAccount
{
    private string _accountNumber;
```

```

private decimal _balance;

public BankAccount(string accountNumber, decimal initialBalance)
{
    _accountNumber = accountNumber;
    _balance = initialBalance;
}

public string AccountNumber
{
    get { return _accountNumber; }
}

public decimal Balance
{
    get { return _balance; }
}

public void Deposit(decimal amount)
{
    if (amount > 0)
    {
        _balance += amount;
    }
}

public bool Withdraw(decimal amount)
{
    if (amount > 0 && _balance >= amount)
    {
        _balance -= amount;
        return true;
    }
    return false;
}
}

```

شرح الكود

في المثال أعلاه، لدينا صنف `BankAccount`. استخدمنا معدل الوصول `private` للخصائص `accountNumber` و `balance` لإخفاء بيانات الحساب الحساسة. لا يمكن تغيير الرصيد مباشرة من خارج الصنف. بدلاً من ذلك، وفرنا خصائص `public` للقراءة فقط (`Balance` و `AccountNumber`)

وأساليب عامة (Deposit و Withdraw) تسمح بالتفاعل مع الرصيد بطريقة آمنة ومتحكم بها، مع إضافة منطق للتحقق من صحة العمليات.

- التغليف يحمي البيانات الداخلية للكائن من الوصول والتعديل غير المباشر.
- معدلات الوصول مثل `private` و `public` هي الأدوات الأساسية لتطبيق التغليف.
- يساهم التغليف في بناء كود أكثر أمانًا، سهولة في الصيانة، وأقل عرضة للأخطاء.

مبادئ OOP – الجزء الأول — Static Members

أعضاء ال Static (Static Members)

في C#، تُعد الأعضاء ال Static جزءًا أساسيًا من فهم كيفية عمل الكلاسات والكائنات. على عكس الأعضاء العادية التي تنتمي لكائن محدد، تنتمي الأعضاء ال Static إلى الكلاس نفسه مباشرةً.

الفكرة الأساسية

الأعضاء ال Static هي حقول (fields) أو دوال (methods) تنتمي إلى الكلاس نفسه، وليست إلى أي كائن (instance) تم إنشاؤه منه. هذا يعني أن قيمتها أو سلوكها يكون مشتركًا بين جميع كائنات الكلاس، ويمكن الوصول إليها دون الحاجة لإنشاء كائن.

مثال برمجي

```
public class MathUtility
{
    public static double PI = 3.14159; // حقل Static

    public static double CalculateCircleArea(double radius) // دالة Static
    {
        return PI * radius * radius;
    }
}

// طريقة استخدام الأعضاء ال Static
// double area = MathUtility.CalculateCircleArea(5);
// double piValue = MathUtility.PI;
```

شرح الكود

في هذا المثال، لدينا كلاس اسمه **MathUtility**. يحتوي هذا الكلاس على حقل **PI** ودالة **CalculateCircleArea** وكلاهما مُعرف بكلمة **static**. هذا يعني أننا لا نحتاج لإنشاء كائن من

MathUtility لاستخدامهما. يمكننا الوصول إلى قيمة **PI** أو استدعاء الدالة **CalculateCircleArea** مباشرةً باستخدام اسم الكلاس، مثل **MathUtility.PI** أو **MathUtility.CalculateCircleArea(5)**.

- الأعضاء الـ **Static** (حقول أو دوال) تنتمي للكلاس وليس لكائن محدد.
- يمكن الوصول إليها مباشرةً باستخدام اسم الكلاس (مثل **ClassName.MemberName**) دون الحاجة لإنشاء كائن.
- تُستخدم غالبًا للوظائف المساعدة العامة أو للبيانات التي يجب أن تكون مشتركة عبر جميع كائنات الكلاس أو عند عدم الحاجة لكائن.

مبادئ OOP – الجزء الثاني — الوراثة (Inheritance)

الوراثة (Inheritance) في C#

تُعد الوراثة من الركائز الأساسية للبرمجة كائنية التوجه (OOP)، وهي تسمح لك بإنشاء فئات جديدة بناءً على فئات موجودة، مما يعزز إعادة استخدام الكود ويُبسط هيكله البرامجي.

المفهوم الأساسي

الوراثة هي آلية تسمح لفئة (الفئة المشتقة أو الابن) بالحصول على خصائص وسلوكيات فئة أخرى (الفئة الأساسية أو الأب). هذا يعني أن الفئة الابن ترث الحقول والدوال من الفئة الأب، مع إمكانية إضافة أو تجاوز سلوكياتها.

مثال عملي

```
public class Vehicle
{
    public string Brand { get; set; }

    public void DisplayInfo()
    {
        Console.WriteLine($"Brand: {Brand}");
    }
}

public class Car : Vehicle
{
    public string Model { get; set; }

    public void DisplayCarInfo()
    {
        DisplayInfo(); // Calls method from base class
        Console.WriteLine($"Model: {Model}");
    }
}
```

شرح الكود

في هذا المثال، لدينا الفئة الأساسية **Vehicle** التي تحتوي على خاصية **Brand** ودالة **DisplayInfo**. ثم لدينا الفئة المشتقة **Car** التي **ترث** من **Vehicle** باستخدام علامة **:**. الفئة **Car** تضيف خاصية **Model** خاصة بها، وفي دالة **DisplayCarInfo** تقوم باستدعاء الدالة **DisplayInfo** من الفئة الأب **Vehicle**، ثم تعرض معلوماتها الخاصة.

- **إعادة استخدام الكود:** تتيح لك الوراثة إعادة استخدام الخصائص والدوال المحددة في الفئة الأساسية بدلاً من كتابتها مرة أخرى.
- **إنشاء تسلسل هرمي:** تُمكنك من بناء علاقات منطقية (مثل "هو نوع من") بين الفئات، مما يجعل تصميم البرنامج أكثر تنظيماً.
- **التوسيع والتخصص:** يمكن للفئة الابن إضافة وظائف جديدة أو تجاوز (Override) وظائف الفئة الأب لتوفير سلوك خاص بها.

مبادئ OOP – الجزء الثاني — Polymorphism و virtual/override

مبادئ OOP – الجزء الثاني: التعددية الشكلية (Polymorphism) و virtual/override

نستكمل في هذا الجزء رحلتنا مع مبادئ البرمجة الشيئية (OOP) لتتعرف على أحد أهم الركائز: **التعددية الشكلية (Polymorphism)**. هذا المفهوم يمنح برامجك مرونة وقابلية للتوسع لا مثيل لها.

التعددية الشكلية (Polymorphism)

التعددية الشكلية تعني "أشكال متعددة". تسمح هذه الخاصية بمعاملة الكائنات من فئات مختلفة، والتي تشترك في فئة أساسية مشتركة، كما لو كانت من الفئة الأساسية. هذا يمكننا من استدعاء نفس الدالة (الميثود) على كائنات مختلفة والحصول على سلوكيات فريدة لكل كائن. يتحقق ذلك في C# باستخدام الكلمات المفتاحية `virtual` و `override`.

مثال عملي

```
public class Shape
{
    public virtual void Draw()
    {
        Console.WriteLine("Drawing a generic shape.");
    }
}

public class Circle : Shape
{
    public override void Draw()
    {
        Console.WriteLine("Drawing a circle.");
    }
}
```

```

public class Rectangle : Shape
{
    public override void Draw()
    {
        Console.WriteLine("Drawing a rectangle.");
    }
}

public class Program
{
    public static void Main(string[] args)
    {
        List<Shape> shapes = new List<Shape>();
        shapes.Add(new Circle());
        shapes.Add(new Rectangle());
        shapes.Add(new Shape());

        foreach (Shape s in shapes)
        {
            s.Draw();
        }
    }
}

```

شرح الكود

لدينا فئة أساسية `Shape` تحتوي على دالة `Draw()` مُعرفة بكلمة `virtual`، مما يعني أن الفئات المشتقة يمكنها تغيير سلوك هذه الدالة. فئتا `Circle` و `Rectangle` هما فئتان مشتقتان من `Shape`، وكل منهما تستخدم كلمة `override` لتقديم تعريفها الخاص لدالة `Draw()`. في دالة `Main`، نقوم بإنشاء قائمة من الكائنات من نوع `Shape` ونضيف إليها كائنات من `Circle` و `Rectangle` و `Shape`. عند المرور على هذه القائمة واستدعاء `Draw()` لكل كائن، سيتم تنفيذ الدالة المناسبة لنوعه الفعلي في وقت التشغيل، مما يظهر التعددية الشكلية.

خلاصة

- **التعددية الشكلية (Polymorphism):** تسمح بمعاملة كائنات من فئات مختلفة ككائنات من فئة أساسية مشتركة.
- تُستخدم الكلمة المفتاحية `virtual` في الفئة الأساسية للسماح للفئات المشتقة بتغيير سلوك دالة معينة.

- تُستخدم الكلمة المفتاحية `override` في الفئة المشتقة لإعادة تعريف (تجاوز) سلوك الدالة الافتراضية للفئة الأساسية.

مبادئ OOP – الجزء الثاني — Interfaces

مبادئ OOP – الجزء الثاني: الواجهات (Interfaces)

في هذا الدرس، سنتعرف على مفهوم الواجهات (Interfaces) في #C، وهي أداة قوية في البرمجة كائنية التوجه (OOP) تتيح لنا تعريف مجموعة من السلوكيات التي يجب على الكلاسات المختلفة أن تلتزم بتنفيذها.

المفهوم الأساسي: الواجهات (Interfaces)

الواجهة هي **عقد** أو **مخطط** يحدد مجموعة من الدوال والخصائص التي يجب على أي كلاس يطبق هذه الواجهة أن يقوم بتنفيذها. هي لا تحتوي على تطبيق لهذه الأعضاء، بل مجرد تعريف لها. الواجهات تضمن أن الكلاسات التي تطبقها ستوفر سلوكيات محددة.

مثال الكود

```
interface IMovable
{
    void Move();
    int Speed { get; set; }
}

class Car : IMovable
{
    public int Speed { get; set; } = 0;

    public void Move()
    {
        Console.WriteLine($"The car is moving at {Speed} km/h.");
    }
}

class Program
{
    static void Main(string[] args)
    {
    }
```

```
Car myCar = new Car();  
myCar.Speed = 60;  
myCar.Move();  
}  
}
```

شرح الكود

في هذا المثال، قمنا بتعريف واجهة اسمها **IMovable**. هذه الواجهة تحدد أن أي كلاس يطبقها يجب أن يحتوي على دالة **(Move)** وخصيصة **Speed**. الكلاس **Car** يقوم بتطبيق الواجهة **IMovable**، وبالتالي يلتزم بتوفير تنفيذ لهذين العضوين. هذا يضمن أن كل كائن من نوع **Car** سيكون لديه القدرة على **(Move)** وسيكون لديه **Speed**، مما يجعل الكود منظمًا وسهل التوسع.

- الواجهات تُعرف "عقدًا" للسلوكيات التي يجب على الكلاسات تنفيذها.
- الكلاس يمكنه تطبيق أكثر من واجهة، مما يوفر مرونة في التصميم.
- تساعد الواجهات في تحقيق التجريد (Abstraction) وتصميم كود مرن وقابل للتوسع.

مبادئ OOP – الجزء الثاني — Abstract Classes

مبادئ OOP – الجزء الثاني: الفئات المجردة (Abstract Classes)

تُعتبر الفئات المجردة (Abstract Classes) ركيزة أساسية في البرمجة كائنية التوجه (OOP)، وتُستخدم لوضع "مخطط" عام للفئات الأخرى. إنها تسمح لك بتعريف سلوك مشترك ودوال يجب على الفئات الفرعية تطبيقها، دون الحاجة لتقديم تطبيق كامل لها في الفئة الأساسية نفسها.

المفهوم الأساسي

الفئة المجردة هي فئة لا يمكن إنشاء كائن مباشر منها. يمكن أن تحتوي على دوال مجردة (Abstract Methods) ليس لها تطبيق، ودوال عادية (Concrete Methods) لها تطبيق كامل. يجب على أي فئة ترث من فئة مجردة أن توفر تطبيقًا لجميع الدوال المجردة الموجودة فيها.

مثال على الكود

```
abstract class Shape
{
    public abstract double CalculateArea();

    public void DisplayMessage()
    {
        Console.WriteLine("This is a shape.");
    }
}

class Circle : Shape
{
    public double Radius { get; set; }

    public Circle(double radius)
    {
        Radius = radius;
    }

    public override double CalculateArea()
```

```
{  
    return Math.PI * Radius * Radius;  
}
```

شرح الكود

في هذا المثال، قمنا بتعريف الفئة المجردة **Shape**، والتي تحتوي على دالة مجردة **CalculateArea()**. هذا يعني أن كل شكل (سواء كان دائرة، مربع، إلخ) يجب أن يعرف كيف يحسب مساحته. الفئة **Circle** ترث من **Shape** وتوفر تطبيقًا محددًا لدالة **CalculateArea()** لحساب مساحة الدائرة. بينما **DisplayMessage()** هي دالة عادية يتم توريثها كما هي.

نقاط أساسية

- لا يمكن إنشاء كائنات مباشرة من الفئات المجردة.
- يجب على الفئات الفرعية (المشتقة) توفير تطبيق لجميع الدوال المجردة فيها.
- تُستخدم لتحديد واجهة مشتركة وسلوك أساسي للفئات المشتقة.

مبادئ OOP – الجزء الثاني — Sealed Classes and Methods

الفئات والأساليب المختومة (Sealed Classes and Methods)

في رحلتنا مع مبادئ البرمجة كائنية التوجه (OOP) في #C، نكتشف أدوات تسمح لنا بالتحكم في كيفية تفاعل الفئات مع بعضها البعض. أحيانًا نرغب في منع فئات أخرى من الوراثة من فئة معينة، أو منع الأساليب الفرعية من تجاوز تنفيذ أسلوب معين. هنا يأتي دور الكلمة المفتاحية `sealed`.

المفهوم الرئيسي: الفئات والأساليب المختومة

الكلمة المفتاحية `sealed` تُستخدم لمنع الوراثة أو التجاوز. عندما تُطبق على **فئة (Class)**، فهذا يعني أنه لا يمكن لأي فئة أخرى أن ترث منها، مما يوقف تسلسل الوراثة عند هذه الفئة. وعندما تُطبق على **أسلوب (Method)** (يجب أن يكون الأسلوب أصلاً `override` لأسلوب `virtual` أو `abstract`)، فهذا يمنع الفئات المشتقة اللاحقة من تجاوز هذا الأسلوب مرة أخرى.

مثال توضيحي

```
class Animal
{
    public virtual string MakeSound()
    {
        return "Animal sound";
    }
}

class Dog : Animal
{
    public override string MakeSound()
    {
        return "Woof!";
    }
}
```

```
sealed class Poodle : Dog
{
    // لا يمكن لأي فئة أخرى أن ترث منها Poodle فئة
    public override string MakeSound()
    {
        return "Yip yip!";
    }
}

// ستؤدي إلى خطأ برمجي Poodle ترث من TinyPoodle محاولة تعريف فئة
// class TinyPoodle : Poodle { }
```

شرح الكود

لدينا الفئة الأساسية `Animal` التي تحتوي على أسلوب `MakeSound` قابل للتجاوز (`virtual`). الفئة `Dog` ترث من `Animal` وتتجاوز أسلوب `MakeSound`. أما الفئة `Poodle`، فترث من `Dog` ولكنها مُعلّمة بالكلمة المفتاحية `sealed`. هذا يعني أنه على الرغم من أنها ترث وتتجاوز الأسلوب، إلا أنه لا يمكن لأي فئة أخرى (مثل `TinyPoodle`) أن ترث من `Poodle`، مما يوقف توسع التسلسل الوراثي عند هذه النقطة.

- **منع الوراثة:** استخدام `sealed` على الفئة يمنع أي فئة أخرى من الوراثة منها.
- **منع التجاوز:** استخدام `sealed override` على أسلوب يمنع الفئات المشتقة اللاحقة من تجاوز هذا الأسلوب مرة أخرى.
- **التحكم في التصميم:** تتيح لك `sealed` التحكم في هيكل التصميم، مما يضمن استقرار أجزاء معينة من تسلسل الوراثة ويمنع التغييرات غير المقصودة.

التعامل مع الأخطاء والاستثناءات — ما هي Exceptions

ما هي Exceptions (الاستثناءات)؟

في عالم البرمجة، لا تسير الأمور دائماً كما هو مخطط لها. قد تحدث أخطاء غير متوقعة أثناء تشغيل البرنامج. هنا يأتي دور "التعامل مع الأخطاء والاستثناءات" لضمان استمرارية عمل برامجنا.

ما هي الاستثناءات (Exceptions)؟

الاستثناء (Exception) هو **حدث غير متوقع** يحدث أثناء تنفيذ البرنامج، يقطع التدفق الطبيعي للعمليات. يمكن أن يكون سببه مشكلة في الكود، أو مشكلة في المدخلات، أو ظروف بيئية خارجة عن سيطرة البرنامج.

مثال عملي

```
int numerator = 10;
int denominator = 0;
// هذه العملية ستثير استثناء DivideByZeroException
int result = numerator / denominator;
Console.WriteLine("Result: " + result);
```

شرح الكود

في هذا المثال، نحاول قسمة عدد (10) على صفر (0)، وهي عملية غير ممكنة رياضياً في البرمجة. سيؤدي هذا السطر إلى إثارة استثناء من نوع `DivideByZeroException`. إذا لم يتم التعامل مع هذا الاستثناء، سيتوقف البرنامج عن العمل بشكل مفاجئ ويظهر رسالة خطأ.

- **أحداث غير متوقعة:** الاستثناءات هي مشكلات تحدث أثناء تشغيل البرنامج وليست أخطاء برمجية تقليدية (مثل أخطاء بناء الجملة).

- **تعطيل التدفق:** عند حدوث استثناء، يتوقف التدفق الطبيعي للبرنامج ما لم يتم "التقاطه" والتعامل معه.
- **أهمية التعامل معها:** تعلم كيفية التعامل مع الاستثناءات يجعل برامجك أكثر قوة، استقراراً، وقدرة على التعافي من المشاكل دون تعطل كامل.

التعامل مع الأخطاء والاستثناءات — كتل try-catch-finally

كتل try-catch-finally

في عالم البرمجة، غالبًا ما تواجه مواقف غير متوقعة قد تؤدي إلى توقف البرنامج عن العمل. لغة C# توفر آلية قوية للتحكم في هذه الأخطاء، أو ما يسمى بـ "الاستثناءات" (Exceptions)، من خلال استخدام كتل **try-catch-finally**.

مفهوم كتل try-catch-finally

تُعد كتل **try-catch-finally** أدوات أساسية لمعالجة الاستثناءات. تُستخدم كتلة **try** لاحتواء الكود الذي قد يسبب خطأ. إذا حدث استثناء، تنتقل السيطرة إلى كتلة **catch** لمعالجته. أما كتلة **finally**، فهي تُنفذ دائمًا، سواء حدث استثناء أم لا، مما يجعلها مثالية لعمليات التنظيف النهائية.

مثال الكود

```
using System;

public class Program
{
    public static void Main(string[] args)
    {
        try
        {
            int num1 = 10;
            int num2 = 0;
            int result = num1 / num2; // This line will throw a DivideByZeroException
            Console.WriteLine($"النتيجة: {result}");
        }
        catch (DivideByZeroException ex)
        {
            Console.WriteLine("حدث خطأ: لا يمكن القسمة على صفر");
            Console.WriteLine($"تفاصيل الخطأ: {ex.Message}");
        }
    }
}
```

```

    }
    catch (Exception ex)
    {
        Console.WriteLine("حدث خطأ عام غير متوقع.");
        Console.WriteLine($"تفاصيل الخطأ: {ex.Message}");
    }
    finally
    {
        Console.WriteLine("انتهى تنفيذ جزء معالجة الاستثناءات");
    }
    Console.WriteLine("البرنامج يكمل عمله بعد الكتل");
}
}

```

شرح الكود

في هذا المثال، وضعنا عملية القسمة التي قد تسبب خطأ (القسمة على صفر) داخل كتلة **try**. عندما يحدث الاستثناء **DivideByZeroException**، تنتقل السيطرة إلى كتلة **catch** الأولى المخصصة لهذا النوع من الأخطاء، وتُطبع رسالة تنبيهية. لو حدث نوع آخر من الاستثناءات، لالتقطته كتلة **catch** العامة. أخيرًا، تُنفذ كتلة **finally** دائمًا، في هذا المثال لطباعة رسالة تأكيد انتهاء العملية.

- **try**: تحدد جزء الكود الذي من المحتمل أن يثير استثناءً (خطأً).
- **catch**: تلتقط وتعالج الاستثناءات التي تحدث داخل كتلة **try** المطابقة لها.
- **finally**: تُنفذ دائمًا، بغض النظر عما إذا حدث استثناء أم لا، وتُستخدم لإنجاز مهام التنظيف الضرورية.

التعامل مع الأخطاء والاستثناءات — رمي الأخطاء

throw

رمي الأخطاء (throw)

عندما يواجه برنامجك موقفًا غير طبيعي أو خطأ لا يمكنه التعامل معه بشكل مباشر، يمكنك استخدام آلية "رمي الأخطاء" لإعلام الأجزاء الأخرى من البرنامج بوجود مشكلة تتطلب اهتمامًا خاصًا.

مفهوم رمي الأخطاء

الكلمة المفتاحية `throw` في C# تُستخدم لإنشاء و"رمي" استثناء (exception). عند رمي استثناء، يتوقف التنفيذ الطبيعي للبرنامج وينتقل التحكم فورًا إلى كتلة `catch` الأقرب التي يمكنها التعامل مع هذا النوع المحدد من الاستثناء.

مثال برمجي

```
public class Calculator
{
    public int Divide(int numerator, int denominator)
    {
        if (denominator == 0)
        {
            throw new ArgumentException("Cannot divide by zero.", nameof(denominator));
        }
        return numerator / denominator;
    }
}
```

شرح الكود

في هذا المثال، لدينا دالة `Divide` التي تحاول إجراء عملية قسمة. قبل إجراء القسمة، نتحقق الدالة إذا كان المقام (`denominator`) يساوي صفرًا. إذا كان كذلك، فإنها تنشئ استثناءً جديدًا من نوع `ArgumentException` باستخدام الكلمة `new` وتقوم برميها باستخدام `throw`. هذا يمنع القسمة على صفر التي قد تسبب خطأ في وقت التشغيل، ويشير بوضوح إلى أن هناك مشكلة في المدخلات.

- تستخدم الكلمة `throw` للإشارة إلى حدوث حالة استثنائية أو خطأ في البرنامج.
- عند رمي استثناء، يتوقف تدفق البرنامج الطبيعي ويتم البحث عن معالج استثناء مناسب.
- يمكنك رمي استثناءات مدمجة (مثل `ArgumentException`) أو تعريف استثناءاتك المخصصة.

التعامل مع الأخطاء والاستثناءات — إنشاء استثناءات مخصصة Custom Exceptions

إنشاء استثناءات مخصصة Custom Exceptions

عندما لا تكون الاستثناءات المدمجة في C# كافية لوصف الأخطاء الخاصة بمنطق عمل تطبيقك، يمكنك إنشاء استثناءاتك المخصصة. هذا يتيح لك التعامل مع المشكلات بطريقة أكثر وضوحًا وتنظيمًا.

مفهوم الاستثناءات المخصصة

الاستثناءات المخصصة هي فئات تقوم بتعريفها لتمثيل أنواع محددة من الأخطاء التي قد تحدث داخل تطبيقك. هي ترث من الفئة **Exception** أو إحدى فئاتها المشتقة، مما يسمح لها بالاندماج مع نظام معالجة الاستثناءات في C#.

مثال الكود

```
public class InvalidAgeException : Exception
{
    public InvalidAgeException() { }
    public InvalidAgeException(string message) : base(message) { }
}

public class User
{
    private int _age;

    public void SetAge(int age)
    {
        if (age < 0 || age > 120)
        {
            throw new InvalidAgeException("العمر المدخل غير صالح");
        }
        _age = age;
        // هنا يمكن إضافة منطق آخر بعد التحقق من العمر
    }
}
```

```
}  
}
```

شرح الكود

في هذا المثال، قمنا بإنشاء فئة استثناء مخصصة باسم `InvalidAgeException` ترث من `Exception`. هذا يجعلها استثناءً صالحًا في #C. توفر الفئة مُنشئين (constructors) يسمحان بتمرير رسالة الخطأ. داخل الفئة `User`، تقوم الدالة `SetAge` بالتحقق من قيمة العمر، وإذا كانت غير صالحة، يتم رمي (throw) كائن من `InvalidAgeException` مع رسالة توضيحية. هذا الاستثناء يمكن بعد ذلك التقاطه (catch) في أجزاء أخرى من الكود باستخدام كتلة `try-catch` للتعامل مع هذا النوع المحدد من الأخطاء.

- تُستخدم الاستثناءات المخصصة لتعريف أخطاء خاصة بمنطق عمل التطبيق وتحسين وضوح الكود.
- يجب أن ترث الاستثناءات المخصصة من الفئة `Exception` أو فئة مشتقة منها.
- تساعد في تنظيم معالجة الأخطاء وتسهيل تحديد المشكلات في الأنظمة المعقدة.

التعامل مع الأخطاء والاستثناءات — Global Error Handling

التعامل الشامل مع الأخطاء (Global Error Handling)

عندما تحدث أخطاء غير معالجة (unhandled exceptions) في تطبيق #C، يمكن أن يتوقف التطبيق عن العمل فجأة. يتيح لنا التعامل الشامل مع الأخطاء التقاط هذه الاستثناءات في مكان واحد مركزي لمنع الأعطال وتحسين استقرار التطبيق.

مفهوم التعامل الشامل مع الأخطاء

هو آلية لضمان أن جميع الاستثناءات التي لم يتم التقاطها بواسطة كتل `try-catch` محددة، يتم التعامل معها على مستوى التطبيق بأكمله. يساعد هذا في تسجيل الأخطاء، عرض رسالة للمستخدم، أو اتخاذ إجراءات أخرى قبل أن ينهار التطبيق.

مثال الكود

```
using System;

class Program
{
    static void Main(string[] args)
    {
        // تسجيل معالج الأخطاء الشامل
        AppDomain.CurrentDomain.UnhandledException += GlobalExceptionHandler;

        Console.WriteLine("بدء التطبيق");
        ThrowAnError(); // هذه الدالة ستلقي خطأ غير معالج
        Console.WriteLine("تم الانتهاء من التطبيق"); // وصول إلى هنا غالباً
    }

    static void ThrowAnError()
    {
        throw new InvalidOperationException("!حدث خطأ غير متوقع في العملية");
    }
}
```

```

static void GlobalExceptionHandler(object sender, UnhandledExceptionEvent
{
    Exception ex = (Exception)e.ExceptionObject;
    Console.WriteLine("\n!!! تم التقاط خطأ شامل غير معالج");
    Console.WriteLine($"رسالة الخطأ: {ex.Message}");
    Console.WriteLine($"هل التطبيق على وشك الإنهاء? {e.IsTerminating}");
    // هنا يمكن تسجيل تفاصيل الخطأ في ملف أو قاعدة بيانات
}
}

```

شرح الكود

يقوم الكود بتسجيل دالة `GlobalExceptionHandler` كمعالج لحدث `UnhandledException` الخاص بالنطاق الحالي للتطبيق. عندما تقوم دالة `ThrowAnError()` بإلقاء استثناء `InvalidOperationException` دون وجود معالج `try-catch` محدد لها، سيتم التقاط هذا الاستثناء بواسطة معالجنا الشامل. داخل `GlobalExceptionHandler`، يمكننا فحص تفاصيل الخطأ واتخاذ إجراءات مثل طباعته أو تسجيله.

- يمنع تعطل التطبيق بالكامل بسبب الأخطاء غير المعالجة.
- يوفر مكانًا مركزيًا وفعالًا لتسجيل جميع الأخطاء غير المتوقعة.
- يعزز من استقرار وموثوقية التطبيق وتجربة المستخدم.

المجموعات والأنواع العامة (Collections & List, Dictionary — (Generics

المجموعات والأنواع العامة في C#: List<T> و Dictionary<TKey,TValue>

في C#, تُعد المجموعات (Collections) أساسية لتخزين البيانات والتعامل معها بكفاءة. سنتعرف هنا على نوعين شائعين ومهمين: **List<T>** و **Dictionary<TKey,TValue>**، وكلاهما يستخدم مفهوم الأنواع العامة (Generics) لمرونة أكبر وسلامة للأنواع.

مفهوم List<T> و Dictionary<TKey,TValue>

List<T> هي قائمة ديناميكية يمكن تغيير حجمها وتخزن عناصر من نفس النوع (T) بترتيب معين، ويمكن الوصول للعناصر بالرقم الفهردي (index). أما **Dictionary<TKey,TValue>** فتخزن أزواجًا من المفتاح والقيمة، حيث يكون كل مفتاح (TKey) فريدًا ويُستخدم للوصول إلى القيمة المقابلة (TValue).

مثال عملي

```
using System;
using System.Collections.Generic;

public class Program
{
    public static void Main(string[] args)
    {
        // قائمة أسماء List<T> مثال على
        List<string> الطلاب = new List<string>();
        الطلاب.Add("أحمد");
        الطلاب.Add("ليلى");
        Console.WriteLine($"{{0}} الطالب الأول: {اسماء الطلاب[0]}");

        // قاموس درجات Dictionary<TKey,TValue> مثال على
        Dictionary<int, string> درجات الطلاب = new Dictionary<int, string>();
```

```
        Add(101, "ممتاز");  
        Add(102, "جيد جداً");  
        Console.WriteLine($"درجات الطالب 102: {درجات الطالب[102]}");  
    }  
}
```

شرح الكود

يوضح الكود كيفية إنشاء قائمة من الأسماء باستخدام **List<string>** وإضافة عناصر نصية إليها ثم الوصول لأحدها باستخدام رقمه الفهري. كما يبين كيفية إنشاء قاموس لدرجات الطلاب باستخدام **Dictionary<int, string>**، حيث المفتاح هو رقم الطالب (عدد صحيح) والقيمة هي تقديره (نص)، وكيفية استرجاع القيمة باستخدام المفتاح الفريد.

خلاصة

- **List<T>**: مناسبة لتخزين مجموعة مرتبة من العناصر من نفس النوع ويمكن الوصول إليها بالفهرس.
- **Dictionary<TKey,TValue>**: مثالية لتخزين البيانات كأزواج مفتاح-قيمة، حيث المفاتيح فريدة وتستخدم للوصول السريع للقيم.
- الأنواع العامة (Generics) مثل **T** تضمن سلامة الأنواع وتجعل المجموعات مرنة وقابلة لإعادة الاستخدام مع أنواع بيانات مختلفة.

المجموعات والأنواع العامة (Collections & Generics)

Stack و Queue

المجموعات والأنواع العامة: Stack و Queue

في C#, تعتبر **Queue** و **Stack** من هياكل البيانات الأساسية التي تُستخدم لتنظيم العناصر بطرق محددة. تتيح لنا هذه الهياكل إدارة البيانات بكفاءة بناءً على مبادئ الترتيب، وهي جزء من مفهوم المجموعات والأنواع العامة (Generics).

مفهوم Queue (قائمة الانتظار)

تعمل **Queue** على مبدأ "من يدخل أولاً يخرج أولاً" (FIFO - First-In, First-Out). تخيل قائمة انتظار في متجر؛ أول شخص يدخل هو أول من يُخدم. تُستخدم لعمليات المعالجة المتسلسلة للمهام.

مفهوم Stack (المكدس)

تعمل **Stack** على مبدأ "من يدخل أخيراً يخرج أولاً" (LIFO - Last-In, First-Out). تخيل كومة من الأطباق؛ الطبق الأخير الذي تضعه هو أول طبق ستأخذه. تُستخدم لمهام مثل التراجع (undo) أو تتبع مسار تنفيذ الدوال.

مثال عملي

```
using System;
using System.Collections.Generic;

public class Program
{
    public static void Main(string[] args)
    {
        // استخدام Queue (FIFO)
        Queue<string> tasks = new Queue<string>();
        tasks.Enqueue("1 مهمة");
        tasks.Enqueue("2 مهمة");
```

```

        Console.WriteLine($"أول مهمة: {tasks.Dequeue()}"); // 1 مهمة: الناتج

        // استخدام Stack (LIFO)
        Stack<int> history = new Stack<int>();
        history.Push(10);
        history.Push(20);
        Console.WriteLine($"آخر إضافة: {history.Pop()}"); // 20: الناتج
    }
}

```

شرح الكود

في هذا المثال، قمنا أولاً بإنشاء **Queue** من نوع **string** لتخزين المهام. أضفنا "مهمة 1" ثم "مهمة 2" باستخدام **Enqueue**. عند استدعاء **Dequeue**، تم سحب "مهمة 1" لأنها الأقدم. ثم أنشأنا **Stack** من نوع **int** لتخزين الأرقام. أضفنا 10 ثم 20 باستخدام **Push**. عند استدعاء **Pop**، تم سحب 20 لأنها الأحدث.

الخلاصة

- **Queue** تعمل بمبدأ **FIFO** (من يدخل أولاً يخرج أولاً)، ومناسبة للمهام التي تتطلب معالجة متسلسلة.
- **Stack** تعمل بمبدأ **LIFO** (من يدخل أخيراً يخرج أولاً)، ومفيدة لعمليات التراجع أو تتبع مسار الدوال.
- كلاهما يستخدم الأنواع العامة **<T>** لتحديد نوع العناصر، مما يوفر أماناً قوياً للأنواع ويمنع الأخطاء.

المجموعات والأنواع العامة (Collections & SortedList و HashSet — (Generics

المجموعات والأنواع العامة: SortedList و HashSet

في هذا الدرس، سنتعرف على نوعين مهمين من المجموعات (Collections) في **HashSet** C#: لتخزين العناصر الفريدة، و**SortedList** لتخزين أزواج المفتاح-القيمة المرتبة. تساعدنا هذه الأنواع على تنظيم البيانات بكفاءة.

مفهوم HashSet

HashSet هي مجموعة تخزن عناصر فريدة فقط، مما يعني أنها لا تسمح بتكرار نفس العنصر. تتميز بسرعة البحث والإضافة والحذف، ولكنها لا تضمن ترتيب العناصر.

مفهوم SortedList

SortedList هي مجموعة تخزن أزواج من المفاتيح والقيم (Key-Value Pairs). تتميز بترتيب عناصرها تلقائياً بناءً على المفتاح، مما يسهل البحث عن طريق المفتاح ويسرع عملية الترتيب.

مثال عملي

```
using System;
using System.Collections.Generic;

public class Program
{
    public static void Main(string[] args)
    {
        // تخزين أسماء فريدة باستخدام HashSet
        HashSet<string> uniqueNames = new HashSet<string>();
        uniqueNames.Add("أحمد");
        uniqueNames.Add("سارة");
    }
}
```

```

uniqueNames.Add("أحمد"); // لن تتم إضافته لأنه مكرر
Console.WriteLine("أسماء فريدة");
foreach (var name in uniqueNames)
{
    Console.WriteLine(name);
}

Console.WriteLine("\n-----\n");

// لتخزين طلاب مرتبين حسب رقم الطالب SortedList استخدام
SortedList<int, string> sortedStudents = new SortedList<int, string>
sortedStudents.Add(102, "فاطمة");
sortedStudents.Add(101, "خالد");
sortedStudents.Add(103, "ليلى");
Console.WriteLine("طلاب مرتبون حسب الرقم");
foreach (var student in sortedStudents)
{
    Console.WriteLine($"الاسم : {student.Value}, الرقم : {student.Key}");
}
}
}

```

شرح الكود

يقوم الكود أولاً بإنشاء **HashSet** لتخزين أسماء فريدة، ويضيف "أحمد" مرتين ليوضح أن الإضافة الثانية لن تُقبل. ثم يقوم بإنشاء **SortedList** لتخزين أرقام الطلاب وأسمائهم. لاحظ كيف يتم ترتيب الطلاب تلقائياً حسب أرقامهم (المفاتيح) عند عرضهم، بغض النظر عن ترتيب الإضافة الأولي.

نقاط رئيسية

- **HashSet**: لتخزين مجموعة من العناصر الفريدة، ويوفر أداة سريعة للبحث والإضافة والحذف.
- **SortedList**: لتخزين أزواج المفتاح-القيمة مع ضمان ترتيبها تلقائياً بناءً على المفتاح.
- كلا النوعين يعتبران جزءاً من المجموعات العامة (Generics) في C#, مما يضمن أمان النوع (Type Safety).

المجموعات والأنواع العامة (Collections & Generic Constraints و Generic Types — (Generics

المجموعات والأنواع العامة: Generic و Generic Types Constraints

تُعد الأنواع العامة (Generics) في #C أداة قوية لكتابة تعليمات برمجية مرنة وقابلة لإعادة الاستخدام، مع الحفاظ على الأمان من الأخطاء المتعلقة بالأنواع. سنتعلم هنا كيف تعمل وكيف يمكننا تقييدها.

الأنواع العامة (Generic Types)

هي فئات أو واجهات أو طرق تسمح لك بتحديد عنصر نائب لنوع بيانات (مثل T). هذا يعني أنك تكتب الكود مرة واحدة ويمكنك استخدامه مع أي نوع بيانات، مما يزيد من قابلية إعادة الاستخدام ويحقق أمان الأنواع عن طريق التحقق من الأنواع في وقت الترجمة.

قيود الأنواع العامة (Generic Constraints)

تُستخدم القيود لتحديد المتطلبات التي يجب أن يستوفها النوع الذي يتم استخدامه كمعامل نوع عام (T). على سبيل المثال، يمكنك المطالبة بأن يكون T فئة، أو بنية، أو أن يطبق واجهة معينة، أو أن يحتوي على مُنشئ افتراضي، مما يسمح بإجراء عمليات محددة على هذا النوع.

مثال على الكود

```
public class GenericComparer
{
    public T GetMaximum<T>(T value1, T value2) where T : IComparable<T>
    {
        if (value1.CompareTo(value2) > 0)
        {
            return value1;
        }
        return value2;
    }
}
```

```
}  
}
```

شرح الكود

يعرض هذا الكود فئة `GenericComparer` التي تحتوي على دالة عامة `GetMaximum`. تستقبل هذه الدالة قيمتين من النوع `T` وتُرجع القيمة الأكبر بينهما.

الشرط `where T : IComparable<T>` يفرض على النوع `T` أن يطبق واجهة `IComparable<T>`، مما يضمن أننا نستطيع استخدام الدالة `CompareTo` لمقارنة القيم من النوع `T` بأمان.

- تسمح الأنواع العامة بكتابة كود مرّن يعمل مع أنواع بيانات مختلفة دون تكرار.
- توفر القيود طريقة لتحديد المتطلبات على الأنواع المستخدمة مع الأنواع العامة، لضمان عمليات آمنة وموثوقة.
- تزيد الأنواع العامة من أمان الأنواع وتُقلل من أخطاء التحويل في وقت التشغيل.

المجموعات والأنواع العامة (Collections & Generics)

IEnumerator و IEnumerable

IEnumerator و IEnumerable

في هذا الدرس، سنتعرف على واجهتي **IEnumerator** و **IEnumerable**، وهما أساسيتان لفهم كيفية عمل التكرار (Iteration) على المجموعات (Collections) في C#. إنهما يمثلان جوهر حلقة **foreach**.

المفهوم الأساسي

تُتيح واجهة **IEnumerable** للكائنات أن تكون "قابلة للتعداد"، أي يمكن التكرار عليها. أما واجهة **IEnumerator**، فهي الواجهة التي تقوم بالفعل بالتنقل بين عناصر المجموعة، وتوفر العنصر الحالي، وتُشير إلى ما إذا كانت هناك عناصر أخرى للتكرار. تستخدم حلقة **foreach** هاتين الواجهتين ضمناً للتكرار بسلسلة.

مثال برمجي

```
using System;
using System.Collections.Generic;

public class Program
{
    public static void Main(string[] args)
    {
        List<string> fruits = new List<string> { "موز", "برتقال", "تفاح" };

        Console.WriteLine("قائمة الفواكه");
        foreach (string fruit in fruits)
        {
            Console.WriteLine(fruit);
        }
    }
}
```

```
}
```

شرح الكود

في المثال أعلاه، قمنا بإنشاء قائمة من الفواكه `List<string>`. هذه القائمة تُنفَّذ واجهة `IEnumerable<string>`، مما يجعلها قابلة للتكرار. تستخدم حلقة `foreach` تلقائيًا `IEnumerator<string>` الخاص بالقائمة للانتقال من عنصر لآخر (تفاح، ثم برتقال، ثم موز) وطباعة كل عنصر على حدة.

- تُشير **IEnumerator** إلى أن الكائن يمكن التكرار عليه.
- تُنفَّذ **IEnumerator** عملية التكرار الفعلية (الانتقال للعنصر التالي والحصول على العنصر الحالي).
- تبسط حلقة `foreach` استخدام هاتين الواجهتين، مما يُسهل التكرار على المجموعات.

LINQ والاستعلام عن البيانات — مقدمة إلى LINQ

مقدمة إلى LINQ

تُعد LINQ (اختصار لـ Language Integrated Query) ميزة قوية في لغة #C تتيح لك كتابة استعلامات البيانات مباشرةً ضمن كود #C. سواء كنت تتعامل مع القوائم، قواعد البيانات، أو ملفات XML، فإن LINQ توفر طريقة موحدة وبسيطة للاستعلام والتلاعب بالبيانات.

ما هو LINQ؟

LINQ هي تقنية تُمكنك من الاستعلام عن البيانات من مصادر مختلفة باستخدام صيغة متجانسة ومفهومة تشبه SQL، ولكنها مدمجة بالكامل في لغة #C. هذا يجعل معالجة البيانات أسهل وأكثر قابلية للقراءة.

مثال عملي

```
using System;
using System.Collections.Generic;
using System.Linq;

public class Program
{
    public static void Main(string[] args)
    {
        List<int> numbers = new List<int> { 1, 5, 8, 12, 15, 20 };

        var evenNumbers = from num in numbers
                           where num % 2 == 0
                           select num;

        Console.WriteLine("الأرقام الزوجية:");
        foreach (var n in evenNumbers)
        {
            Console.WriteLine(n);
        }
    }
}
```

```
}  
}
```

شرح الكود

يقوم هذا الكود بإنشاء قائمة من الأعداد الصحيحة. ثم يستخدم استعلام LINQ بسيطًا لاختيار جميع الأرقام الزوجية فقط من هذه القائمة. يُحدد الجزء `where num % 2 == 0` الشرط الذي يجب أن يستوفيه كل رقم، وهو أن يكون قابلاً للقسمة على 2 بدون باقي. بعد ذلك، يتم طباعة الأرقام الزوجية التي تم العثور عليها إلى وحدة التحكم.

نقاط رئيسية

- **تكامل اللغة:** تدمج LINQ قدرات الاستعلام مباشرة في لغة C#، مما يجعل الكود أكثر تناسقًا.
- **مصادر بيانات متعددة:** يمكن استخدام LINQ للاستعلام عن أنواع مختلفة من البيانات مثل القوائم والمجموعات (LINQ to Objects)، قواعد البيانات (LINQ to SQL / Entity Framework)، و XML (LINQ to XML).
- **صيغة موحدة:** توفر LINQ صيغة استعلام موحدة، مما يقلل من تعقيد التعامل مع مصادر بيانات مختلفة بأساليب مختلفة.

LINQ والاستعلام عن البيانات — Query Syntax

بناء الاستعلام (Query Syntax) في LINQ

في #C، يوفر LINQ طريقتين رئيسيتين لكتابة الاستعلامات. أحد هذه الأساليب هو "بناء الاستعلام" (Query Syntax)، والذي يقدم طريقة تشبه لغة SQL لاستكشاف البيانات والتعامل معها، مما يجعله سهل القراءة والفهم.

ما هو بناء الاستعلام (Query Syntax)؟

هو نمط لكتابة استعلامات LINQ مباشرة داخل #C باستخدام كلمات مفتاحية مثل `from`، `where`، و `select`. يوفر هذا البناء بنية تشبه لغة الاستعلامات الهيكلية (SQL) للتعامل مع أي مجموعة بيانات تدعم واجهة `IEnumerable`.

مثال على الكود

```
using System;
using System.Collections.Generic;
using System.Linq;

public class Program
{
    public static void Main(string[] args)
    {
        List<int> numbers = new List<int> { 1, 5, 8, 12, 15, 20 };

        // Query Syntax
        var evenNumbers = from num in numbers
                          where num % 2 == 0
                          select num;

        Console.WriteLine("الأعداد الزوجية:");
        foreach (var n in evenNumbers)
        {
            Console.WriteLine(n);
        }
    }
}
```

```
}  
}
```

شرح الكود

يبدأ الاستعلام بكلمة `from` لتحديد مصدر البيانات (`numbers`) والمتغير المؤقت لكل عنصر (`num`). يستخدم الشرط `where` لتصفية الأعداد واختيار الزوجية فقط منها (التي يكون باقي قسمتها على 2 يساوي صفرًا). أخيرًا، يقوم `select` باختيار الأعداد التي استوفت الشرط. يتم تخزين النتيجة في المتغير `evenNumbers` ثم طباعتها.

- **سهولة القراءة:** يشبه بناء الاستعلام لغة SQL، مما يجعله بديهيًا للمطورين المعتادين عليها.
- **البداية والنهاية:** يبدأ كل استعلام بـ `from` وينتهي بـ `select` أو `group by`.
- **المرونة:** يمكن استخدامه مع أنواع مختلفة من مصادر البيانات مثل القوائم، قواعد البيانات، وXML.

LINQ والاستعلام عن البيانات — Method Syntax

LINQ: صيغة الدوال (Method Syntax)

تُعد صيغة الدوال (Method Syntax) إحدى الطرق الرئيسية لكتابة استعلامات LINQ في #C. وهي تعتمد على استخدام دوال التوسيع (Extension Methods) مباشرةً على المجموعات، مما يوفر طريقة مرنة وقوية للاستعلام عن البيانات ومعالجتها.

مفهوم أساسي: صيغة الدوال (Method Syntax)

تستخدم صيغة الدوال مجموعة من دوال التوسيع التي يوفرها LINQ لتصفية البيانات، ترتيبها، وتحويلها. يتم ربط هذه الدوال معًا باستخدام نقطة (.) لتشكيل سلسلة استعلامات متتابعة وواضحة، تُنفذ خطوة بخطوة على البيانات.

مثال عملي

```
using System;
using System.Collections.Generic;
using System.Linq;

public class Program
{
    public static void Main()
    {
        List<int> numbers = new List<int> { 1, 5, 8, 2, 7, 3, 10 };

        // تصفية وترتيب الأرقام الزوجية باستخدام Method Syntax
        var evenNumbers = numbers.Where(n => n % 2 == 0)
                                .OrderBy(n => n)
                                .ToList();

        Console.WriteLine("الأرقام الزوجية المرتبة");
        foreach (var num in evenNumbers)
        {
            Console.WriteLine(num); // الناتج: 2, 8, 10
        }
    }
}
```

```
}  
}  
}
```

شرح الكود

يبدأ الكود بتعريف قائمة من الأرقام الصحيحة. ثم نستخدم دالة التوسيع **Where** لتصفية هذه القائمة، حيث يتم اختيار الأرقام الزوجية فقط باستخدام تعبير لامدا `(n => n % 2 == 0)`. بعد ذلك، نطبق دالة التوسيع **OrderBy** لترتيب الأرقام الزوجية التي تم الحصول عليها تصاعديًا. أخيرًا، تقوم دالة **ToList()** بتحويل نتيجة الاستعلام إلى قائمة جديدة، يمكننا بعد ذلك طباعتها.

- تعتمد صيغة الدوال على **دوال التوسيع المتسلسلة** (chained extension methods).
- تُكتب هذه الاستعلامات مباشرةً على الكائنات التي تدعم واجهة **IEnumerable<T>**.
- توفر **مرونة قوية** في بناء الاستعلامات المعقدة بشكل تدريجي وواضح.

LINQ والاستعلام عن البيانات — التصفية والترتيب والتجميع

LINQ: التصفية والترتيب والتجميع

تُعد تقنية LINQ (Language Integrated Query) أداة قوية في #C لتمكينك من الاستعلام عن البيانات بأسلوب موحد، سواء كانت من قواعد بيانات، كائنات، أو XML. في هذا الدرس، سنتعلم كيفية استخدام LINQ لتصفية البيانات، ترتيبها، وتجميعها بسهولة وفعالية.

التصفية (Filtering)

تسمح لك التصفية باختيار العناصر التي تستوفي شروطاً معينة من مجموعة بيانات. نستخدم لذلك الدالة `Where` التي تُرجع مجموعة فرعية من العناصر المطابقة للمعيار.

الترتيب (Ordering)

يُستخدم الترتيب لإعادة ترتيب العناصر في مجموعة البيانات بناءً على خاصية معينة، تصاعدياً أو تنازلياً. نستخدم الدالتين `OrderBy` للترتيب التصاعدي و `OrderByDescending` للترتيب التنازلي.

التجميع (Grouping)

يساعد التجميع على تنظيم العناصر المتشابهة في مجموعات بناءً على مفتاح مشترك. نستخدم الدالة `GroupBy` لذلك، حيث تُرجع مجموعات من العناصر التي تشترك في نفس الخاصية.

مثال عملي

```
using System;  
using System.Collections.Generic;  
using System.Linq;
```

```

public class Product
{
    public string Name { get; set; }
    public string Category { get; set; }
    public decimal Price { get; set; }
}

public class LinqExample
{
    public static void Main()
    {
        var products = new List<Product>
        {
            new Product { Name = "لابتوب", Category = "إلكترونيات", Price = 800M },
            new Product { Name = "هاتف", Category = "إلكترونيات", Price = 500M },
            new Product { Name = "قميص", Category = "ملابس", Price = 50M },
            new Product { Name = "بنطال", Category = "ملابس", Price = 70M },
            new Product { Name = "شاشة", Category = "إلكترونيات", Price = 300M }
        };

        // ب: المنتجات الإلكترونية بسعر < 500، مرتبة تنازلياً حسب السعر 1.
        var highValueElectronics = products
            .Where(p => p.Category == "إلكترونيات" && p.Price > 500M)
            .OrderByDescending(p => p.Price)
            .Select(p => p.Name); // لعرض الأسماء فقط

        Console.WriteLine("المنتجات الإلكترونية قيمة (مرتبة تنازلياً):");
        foreach (var name in highValueElectronics)
        {
            Console.WriteLine($"- {name}");
        }

        // 2. تجميع: المنتجات مجمعة حسب الفئة
        var productsByCategory = products
            .GroupBy(p => p.Category);

        Console.WriteLine("\nالمنتجات مجمعة حسب الفئة:");
        foreach (var group in productsByCategory)
        {
            Console.WriteLine($"> {group.Key} ({group.Count()} منتجات)");
            foreach (var product in group)
            {
                Console.WriteLine($"  - {product.Name}");
            }
        }
    }
}

```



```
}
```

شرح الكود

يُظهر المثال أعلاه كيف نستخدم LINQ بثلاث طرق: أولاً، لتصفية المنتجات (Where) التي تنتمي لفئة "إلكترونيات" وسعرها أكبر من 500، ثم ترتيب النتائج (OrderByDescending) تنازلياً حسب السعر. ثانياً، يوضح كيفية تجميع (GroupBy) جميع المنتجات ضمن فئات بناءً على خاصية Category، مما يسهل عرضها كقوائم فرعية.

- **التصفية بـ Where:** تساعدك على استخراج البيانات التي تُلبي شروطاً معينة فقط.
- **الترتيب بـ OrderBy/OrderByDescending:** يمنحك القدرة على تنظيم بياناتك ترتيباً تصاعدياً أو تنازلياً.
- **التجميع بـ GroupBy:** يتيح لك تجميع العناصر المتشابهة معاً ليسهل تحليلها وعرضها.

LINQ والاستعلام عن البيانات — Where و Select و Any و First

LINQ: Select, Where, First, Any

في هذا الدرس، سنتعلم أساسيات التعامل مع LINQ في C# لاستعلام البيانات، وذلك بالتركيز على أربع عمليات رئيسية: **Select** لاختيار وتحويل البيانات، **Where** لتصفيته، **First** لاسترجاع أول عنصر، و**Any** للتحقق من وجود عناصر معينة.

Select لتحويل البيانات

تستخدم **Select** لتحويل عناصر المجموعة إلى شكل جديد أو لاختيار خصائص محددة منها. كأن تحوّل قائمة أرقام إلى قائمة مربعاتها.

Where لتصفية البيانات

تستخدم **Where** لتصفية العناصر من مجموعة بناءً على شرط محدد. فقط العناصر التي تحقق الشرط ستُضمن في النتيجة النهائية.

First / FirstOrDefault لاسترجاع العنصر الأول

تستخدم **First** لاسترجاع أول عنصر يطابق شرطًا معينًا. إذا لم يوجد، فستُحدث خطأ. أما **FirstOrDefault** فتعيد القيمة الافتراضية للنوع إذا لم تجد العنصر، مما يجعلها أكثر أمانًا.

Any للتحقق من وجود عناصر

تستخدم **Any** للتحقق مما إذا كان هناك أي عنصر في المجموعة يطابق شرطًا معينًا. تعيد قيمة منطقية (صحيح/خطأ) دون الحاجة لاسترجاع العناصر نفسها.

مثال عملي

```

using System;
using System.Collections.Generic;
using System.Linq;

public class LinqOperations
{
    public static void Main(string[] args)
    {
        List<int> numbers = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

        // Select: تحويل كل رقم إلى مربعه
        var squaredNumbers = numbers.Select(n => n * n).ToList();
        Console.WriteLine("الأرقام المربعة: " + string.Join(", ", squaredNumbers));

        // Where: تصفية الأرقام الزوجية
        var evenNumbers = numbers.Where(n => n % 2 == 0).ToList();
        Console.WriteLine("الأرقام الزوجية: " + string.Join(", ", evenNumbers));

        // FirstOrDefault: استرجاع أول رقم أكبر من 5
        int? firstNumberGreaterThanFive = numbers.FirstOrDefault(n => n > 5);
        Console.WriteLine("أول رقم أكبر من 5: " + (firstNumberGreaterThanFive ?? 0));

        // Any: التحقق مما إذا كان هناك أي رقم أكبر من 10
        bool anyNumberGreaterThanTen = numbers.Any(n => n > 10);
        Console.WriteLine("هل يوجد أي رقم أكبر من 10؟ " + anyNumberGreaterThanTen);
    }
}

```

شرح الكود

يبدأ الكود بتعريف قائمة أرقام صحيحة. يوضح **Select** كيف يمكن تحويل كل رقم في القائمة إلى مربعه. ثم يستخدم **Where** لاختيار الأرقام الزوجية فقط من القائمة الأصلية. بعد ذلك، يستعرض **FirstOrDefault** كيفية البحث عن أول رقم أكبر من خمسة، مع إمكانية التعامل مع عدم وجود نتيجة. أخيرًا، يوضح **Any** كيفية التحقق بسرعة مما إذا كان هناك أي رقم في القائمة يفي بشرط معين (أن يكون أكبر من 10).

- تُعد **Select** أداة قوية لتغيير شكل البيانات أو استخراج أجزاء معينة منها.
- تستخدم **Where** لإنشاء مجموعات فرعية من البيانات بناءً على شروط منطقية محددة.
- تساعد **First/FirstOrDefault** في العثور على العنصر الأول المطابق لشرط، مع توفير مرونة في التعامل مع الحالات التي لا يوجد فيها تطابق.

- تُمكن **Any** من إجراء عمليات تحقق سريعة لوجود بيانات معينة دون الحاجة لاسترجاعها كلها.

LINQ to Objects vs — LINQ to Entities

LINQ to Objects vs LINQ to Entities

في هذا الدرس، سنتعلم عن نوعين أساسيين من LINQ وكيف يختلفان في طريقة معالجتهما للبيانات: **LINQ to Objects** و **LINQ to Entities**. فهمهما يساعدك على اختيار الأداة المناسبة لاستعلاماتك.

المفهوم الأساسي

يستخدم **LINQ to Objects** للاستعلام عن مجموعات البيانات (collections) الموجودة في ذاكرة التطبيق (مثل List، Array). أما **LINQ to Entities** فيستخدم للاستعلام عن البيانات المخزنة في قواعد البيانات، حيث يقوم بتحويل استعلام LINQ إلى لغة استعلام قاعدة البيانات (مثل SQL) ليتم تنفيذه على الخادم مباشرة.

مثال على الكود (LINQ to Objects)

```
using System;
using System.Collections.Generic;
using System.Linq;

public class Program
{
    public static void Main()
    {
        List<int> numbers = new List<int> { 1, 5, 10, 15, 20, 25 };

        // مثال على LINQ to Objects
        var evenNumbers = from num in numbers
                          where num % 2 == 0
                          select num;

        Console.WriteLine("الأرقام الزوجية:");
        foreach (var n in evenNumbers)
        {
```

```
        Console.WriteLine(n);  
    }  
}  
}
```

شرح الكود

في هذا المثال، نستخدم **LINQ to Objects** للاستعلام عن قائمة الأرقام `numbers`، وهي مجموعة بيانات موجودة في ذاكرة التطبيق. يقوم الاستعلام `evenNumbers` باختيار الأرقام الزوجية فقط من هذه القائمة. لو كنا نستخدم **LINQ to Entities**، لكان الاستعلام مشابهاً لكنه سيعمل على جدول في قاعدة بيانات، وسيتم تحويله إلى استعلام SQL وتنفيذه على خادم قاعدة البيانات.

- **LINQ to Objects**: يستعلم عن البيانات الموجودة في ذاكرة التطبيق (كائنات).
- **LINQ to Entities**: يستعلم عن البيانات المخزنة خارج التطبيق، مثل قواعد البيانات.
- الفرق الرئيسي هو مكان تنفيذ الاستعلام: في الذاكرة لـ `Objects`، وعلى خادم قاعدة البيانات لـ `Entities`.

التعامل مع الملفات — القراءة والكتابة للملفات النصية

القراءة والكتابة للملفات النصية في #C

تعد القدرة على التعامل مع الملفات جزءًا أساسيًا في أي تطبيق، حيث تسمح لنا بتخزين البيانات وقراءتها بشكل دائم. في #C، يمكننا بسهولة قراءة وكتابة الملفات النصية باستخدام فئات مخصصة لهذا الغرض، مما يفتح آفاقًا واسعة لحفظ إعدادات المستخدم أو سجلات التطبيقات.

المفهوم الأساسي

نستخدم في #C فئتي **StreamWriter** للكتابة إلى الملفات النصية و **StreamReader** للقراءة منها. يفضل دائمًا استخدام هاتين الفئتين ضمن جملة **using** لضمان إغلاق الملفات بشكل صحيح وتحرير الموارد تلقائيًا، حتى لو حدث خطأ.

مثال عملي

```
string filePath = "MyNotes.txt"; // تحديد اسم الملف

// 1. الكتابة إلى الملف
using (StreamWriter writer = new StreamWriter(filePath))
{
    writer.WriteLine("مرحباً، هذا السطر الأول");
    writer.WriteLine("وهذا السطر الثاني من الملف");
}

// 2. القراءة من الملف
string fileContent;
using (StreamReader reader = new StreamReader(filePath))
{
    fileContent = reader.ReadToEnd(); // قراءة كل محتوى الملف
}
// يحتوي على نص الملف fileContent الآن متغير
```

شرح الكود

يبدأ الكود بتحديد مسار الملف النصي. في الجزء الأول، يتم إنشاء كائن `StreamWriter` للكتابة إلى الملف. تستخدم دالة `WriteLine` لإضافة أسطر جديدة. يضمن استخدام `using` إغلاق الكاتب تلقائيًا. في الجزء الثاني، يتم إنشاء `StreamReader` لقراءة المحتوى. تقوم دالة `ReadToEnd` بقراءة النص بأكمله وتخزينه في المتغير `fileContent`.

- `StreamWriter` للكتابة و `StreamReader` للقراءة من الملفات النصية.
- استخدم جملة `using` دائمًا لضمان إغلاق الملفات وتحرير الموارد بشكل سليم.
- التعامل مع الملفات يتيح للتطبيق حفظ واسترجاع البيانات بشكل دائم.

التعامل مع الملفات — التعامل مع المجلدات

Path & Directory والمسارات

التعامل مع المجلدات والمسارات Path & Directory

في #C، يعد التعامل مع المجلدات والمسارات جزءًا أساسيًا لإدارة الملفات وتنظيمها بشكل فعال. يركز هذا الدرس على كيفية استخدام كلاسات **Path** و **Directory** لإجراء عمليات مختلفة مثل إنشاء المجلدات، والتحقق من وجودها، ودمج أجزاء المسار بطريقة آمنة وموثوقة.

مفهوم الكلاسات Path و Directory

الكلاس **Path** يوفر أدوات ثابتة للتعامل مع سلاسل المسارات (مثل دمجها أو استخراج أجزاء منها) دون الحاجة لوجود المسار فعليًا. بينما الكلاس **Directory** يسمح لك بإجراء عمليات مباشرة على المجلدات الفعلية في نظام الملفات، مثل الإنشاء، الحذف، النقل، أو التحقق من وجود مجلد.

مثال كود

```
using System;
using System.IO;

public class FolderManagement
{
    public static void Main(string[] args)
    {
        string currentDirectory = Directory.GetCurrentDirectory();
        Console.WriteLine($"المجلد الحالي: {currentDirectory}");

        string newFolderPath = Path.Combine(currentDirectory, "MyNewFolder");

        if (!Directory.Exists(newFolderPath))
        {
            Directory.CreateDirectory(newFolderPath);
            Console.WriteLine($"تم إنشاء المجلد: {newFolderPath}");
        }
        else
    }
```

```
{
    Console.WriteLine($"المجلد موجود بالفعل : {newFolderPath}");
}
}
```

شرح الكود

يقوم الكود أولاً بالحصول على المسار الحالي للتطبيق باستخدام `Directory.GetCurrentDirectory()`. ثم، يستخدم `Path.Combine()` لدمج المسار الحالي مع اسم مجلد جديد ("MyNewFolder") بشكل صحيح. بعد ذلك، يتحقق `Directory.Exists()` مما إذا كان المجلد الجديد موجودًا بالفعل. إذا لم يكن موجودًا، يقوم `Directory.CreateDirectory()` بإنشاء المجلد.

- كلاس **Path** مفيد للتعامل مع سلاسل المسارات بشكل مستقل عن وجود الملفات أو المجلدات.
- كلاس **Directory** يسمح لك بإجراء عمليات فعلية (إنشاء، حذف، نقل) على المجلدات في نظام الملفات.
- دائمًا استخدم `Directory.Exists()` للتحقق من وجود مجلد قبل محاولة إجراء عمليات عليه لتجنب الأخطاء.

التعامل مع الملفات — StreamReader و StreamWriter

التعامل مع الملفات: StreamReader و StreamWriter

في لغة #C، تُعدّ الفئتان **StreamReader** و **StreamWriter** أدوات أساسية وقوية للتعامل مع الملفات النصية. تُمكنك هاتان الفئتان من قراءة البيانات النصية من الملفات أو كتابتها إليها بطريقة منظمة وفعالة، مما يجعلهما ضروريين لأي تطبيق يتطلب التفاعل مع بيانات مخزنة على القرص.

مفهوم StreamReader و StreamWriter

تُستخدم فئة **StreamWriter** لكتابة البيانات النصية، سطرًا بسطر أو كتلة، إلى ملف معين. في المقابل، تُستخدم فئة **StreamReader** لقراءة البيانات النصية من ملف موجود، مما يسمح لك باسترجاع المحتوى سطرًا بسطر أو دفعة واحدة. كلاهما يعملان مع تدفقات البيانات لتوفير واجهة سهلة للتعامل مع الملفات.

مثال عملي

```
using System;
using System.IO;

public class FileOperations
{
    public static void Main(string[] args)
    {
        string filePath = "myDocument.txt";
        string textToWrite = "هذا سطر جديد في الملف!\nC#!\nمرحباً بكم في عالم";

        // StreamWriter كتابة إلى الملف باستخدام
        using (StreamWriter sw = new StreamWriter(filePath))
        {
            sw.WriteLine(textToWrite);
        }
        Console.WriteLine("تمت الكتابة إلى الملف بنجاح");
    }
}
```

```
// StreamReader قراءة من الملف باستخدام
using (StreamReader sr = new StreamReader(filePath))
{
    string content = sr.ReadToEnd();
    Console.WriteLine("\nمحتوى الملف الذي تمت قراءته:");
    Console.WriteLine(content);
}
Console.WriteLine("تمت القراءة من الملف بنجاح");
}
```

شرح الكود

يقوم الكود أولاً بتعريف مسار ملف "myDocument.txt" ونص سيتم كتابته. يستخدم **StreamWriter** لإنشاء الملف وكتابة النص داخله، حيث تضمن جملة **using** إغلاق الكاتب وتفرغ المخزن المؤقت تلقائيًا. بعد ذلك، يفتح الكود نفس الملف باستخدام **StreamReader** ليقرأ كل محتواه بواسطة **ReadToEnd()** ثم يعرضه على شاشة الكونسول. تضمن **using** هنا أيضًا إغلاق القارئ بشكل صحيح بعد الانتهاء.

- **StreamWriter**: يُستخدم لكتابة النصوص والبيانات إلى الملفات.
- **StreamReader**: يُستخدم لقراءة النصوص والبيانات من الملفات.
- يُفضل دائمًا استخدام جملة **using** لضمان إغلاق الموارد (مثل الملفات) تلقائيًا وتجنب تسريبها.

التعامل مع الملفات — استخدام using statement

التعامل مع الملفات: استخدام using statement

عند العمل مع الملفات أو أي موارد أخرى تتطلب إغلاقًا وتطهيرًا بعد الاستخدام في C#, فإن جملة `using statement` هي أداة قوية وضرورية. إنها تضمن تحرير الموارد بشكل صحيح وتلقائي، حتى لو حدثت أخطاء.

لماذا نستخدم using statement؟

تُستخدم `using statement` لضمان استدعاء الدالة `Dispose()` على الكائنات التي تنفذ الواجهة `IDisposable`. هذا يعني أن الموارد مثل اتصالات قواعد البيانات أو تدفقات الملفات سيتم إغلاقها وتحريرها تلقائيًا بمجرد الانتهاء من استخدامها، مما يمنع تسرب الموارد.

مثال عملي

```
using System.IO;

public class FileHandler
{
    public void WriteToFile(string path, string content)
    {
        using (StreamWriter writer = new StreamWriter(path))
        {
            writer.WriteLine(content);
        }
    }
}
```

شرح الكود

في هذا المثال، نقوم بإنشاء كائن `StreamWriter` لكتابة محتوى إلى ملف. جملة `using` تضمن أن كائن `StreamWriter` سيتم إغلاقه وتحرير جميع الموارد المرتبطة به بشكل تلقائي بمجرد انتهاء كتلة

`using`، سواء تمت الكتابة بنجاح أم حدث خطأ أثناء العملية. هذا يلغي الحاجة إلى استدعاء `writer.Close()` أو `writer.Dispose()` يدويًا في كتلة `finally`.

- تضمن `using statement` تحرير الموارد (مثل الملفات) تلقائيًا فور الانتهاء منها.
- تساعد في منع تسرب الموارد وتقليل الأخطاء المرتبطة بإدارة الموارد يدويًا.
- تعمل مع أي كائن ينفذ واجهة `IDisposable`، مما يجعل الكود أنظف وأكثر أمانًا.

التعامل مع الملفات — التعامل مع JSON باستخدام System.Text.Json

التعامل مع JSON باستخدام System.Text.Json

يُعد JSON (JavaScript Object Notation) تنسيقًا شائعًا لتبادل البيانات، ويوفر #C دعمًا ممتازًا للتعامل معه عبر المكتبة المدمجة **System.Text.Json**. هذه المكتبة حديثة وفعالة لتحويل الكائنات إلى JSON والعكس.

المفهوم الأساسي: التسلسل وإلغاء التسلسل

يتضمن التعامل مع JSON عمليتين رئيسيتين: **التسلسل (Serialization)**، وهي تحويل كائن #C إلى نص JSON، و**إلغاء التسلسل (Deserialization)**، وهي تحويل نص JSON مرة أخرى إلى كائن #C من نوع محدد.

مثال عملي على الكود

```
using System.Text.Json;
using System;

public class Book
{
    public string Title { get; set; }
    public string Author { get; set; }
}

public class Program
{
    public static void Main(string[] args)
    {
        // JSON (Serialization) تحويل كائن إلى
        Book myBook = new Book { Title = "C# Basics", Author = "Jane Doe" };
        string jsonText = JsonSerializer.Serialize(myBook);
        Console.WriteLine(jsonText);
    }
}
```

```
// (Deserialization) إلى كائن JSON تحويل
Book? newBook = JsonSerializer.Deserialize<Book>(jsonText);
Console.WriteLine($"Title: {newBook?.Title}, Author: {newBook?.Author}");
}
```

شرح الكود

في هذا المثال، نقوم أولاً بتعريف فئة `Book` بسيطة. ثم، نستخدم `JsonSerializer.Serialize(myBook)` لتحويل كائن `myBook` إلى نص JSON. بعد ذلك، نستخدم `JsonSerializer.Deserialize<Book>(jsonText)` لتحويل نص JSON هذا مرة أخرى إلى كائن من نوع `Book`.

نقاط أساسية

- `System.Text.Json` هي مكتبة #C مدمجة وفعالة للتعامل مع JSON.
- استخدم `JsonSerializer.Serialize()` لتحويل كائنات #C إلى نصوص JSON.
- استخدم `JsonSerializer.Deserialize<T>()` لتحويل نصوص JSON إلى كائنات #C.

مفاهيم متقدمة — Multicast و Delegates

Delegates

Multicast Delegates و Delegates

في C#، تُعد Delegates مفهومًا قويًا يسمح لك بالتعامل مع الدوال (methods) كمتغيرات. هذا يفتح أبوابًا للبرمجة المرنة والتصميم المبني على الأحداث (event-driven programming). أما Multicast Delegates فتتيح لك جمع عدة دوال في delegate واحد لتنفيذها بالتسلسل.

ما هو Delegate؟

ال Delegate هو نوع يشير إلى دالة (method) ذات توقيع (signature) محدد (نوع الإرجاع ومعاملات الإدخال). يمكنك التفكير فيه كـ "مؤشر دالة آمن النوع" (type-safe function pointer). يسمح لك بتمرير الدوال كمعاملات، وتخزينها في متغيرات، واستدعائها لاحقًا.

Multicast Delegates

عندما تُضيف عدة دوال إلى نفس ال Delegate باستخدام العامل `+=`، يصبح هذا ال Delegate Multicast Delegate. عند استدعاء هذا ال Delegate، فإنه يقوم بتنفيذ جميع الدوال المضافة إليه بالترتيب الذي أُضيفت به.

مثال برمجي

```
using System;

public delegate void SimpleDelegate(int num);

public class Processor
{
    public void ShowNumber(int x)
    {
        Console.WriteLine($"الرقم الأصلي: {x}");
    }
}
```

```

    }

    public void SquareNumber(int x)
    {
        Console.WriteLine($"مربع الرقم: {x * x}");
    }
}

public class Program
{
    public static void Main(string[] args)
    {
        Processor p = new Processor();

        // إسناد الدالة الأولى Delegate إنشاء
        SimpleDelegate del = p.ShowNumber;

        // Multicast Delegate إضافة الدالة الثانية، ليصبح
        del += p.SquareNumber;

        Console.WriteLine("بالقيمة 5 Multicast Delegate استدعاء");
        del(5); // سيتم استدعاء كلتا الدالتين
    }
}

```

شرح الكود

يعرّف الكود أولاً **SimpleDelegate** كنوع يمكنه الإشارة إلى أي دالة (method) لا تُرجع قيمة وتأخذ عددًا صحيحًا واحدًا. ثم لدينا فئة **Processor** تحتوي على دالتين (ShowNumber و SquareNumber) تتوافقان مع توقيع الـ Delegate. في الدالة **Main**، نقوم بإنشاء كائن من **Processor**. بعد ذلك، نُنشئ مثيلًا من الـ Delegate ونُسند إليه الدالة **ShowNumber**. باستخدام العامل **+=**، نضيف الدالة **SquareNumber** إلى نفس الـ Delegate، مما يحوله إلى Multicast Delegate. عندما نستدعي **del(5)**، يتم تنفيذ كلتا الدالتين (ShowNumber ثم SquareNumber) بالتسلسل، كلتاهما تستقبل القيمة 5.

خلاصة

- الـ **Delegate** يسمح لك بالتعامل مع الدوال كمتغيرات، مما يزيد من مرونة الكود.

- ال **Multicast Delegate** يمكنه الاحتفاظ بأكثر من دالة واحدة، ويقوم بتشغيلها جميعًا عند استدعائه.
- توقيع ال Delegate (نوع الإرجاع والمعاملات) يجب أن يتطابق مع توقيع الدوال التي يشير إليها.

مفاهيم متقدمة — Event Handlers و Events

الأحداث (Events) ومعالجات الأحداث (Event Handlers) في C#

في البرمجة، الأحداث هي طريقة للكائنات لإرسال إشعارات عندما يحدث شيء مهم، ومعالجات الأحداث هي الدوال التي تستجيب لهذه الإشعارات. هذا الدرس يشرح كيف يمكنك هذه الآلية من بناء تطبيقات مرنة ومتراصة.

ما هي الأحداث (Events)؟

الحدث هو إشارة يرسلها كائن (يُسمى "المرسل") لإخبار الكائنات الأخرى (تُسمى "المستمعين") بحدوث إجراء معين أو تغيير في حالته. معالج الحدث هو ببساطة دالة يتم استدعاؤها تلقائيًا عندما يتم رفع الحدث للاستجابة له.

مثال توضيحي (Code Example)

```
using System;

// لتوقيع معالج الحدث (Delegate) تعريف مفوض 1.
public delegate void StatusUpdateHandler(string statusMessage);

public class Notifier
{
    // باستخدام المفوض (Event) تعريف حدث 2.
    public event StatusUpdateHandler OnStatusChanged;

    public void ChangeStatus(string newStatus)
    {
        Console.WriteLine($"Notifier: تغيير الحالة إلى '{newStatus}'");
        // رفع الحدث 3. (Raise Event)
        // يقوم باستدعاء جميع الدوال المشتركة في الحدث
        OnStatusChanged?.Invoke(newStatus);
    }
}
```

```

public class Listener
{
    // 4. معالج الحدث (Event Handler)
    public void HandleStatusChange(string status)
    {
        Console.WriteLine($"Listener: استلم تحديث الحالة : {status}");
    }
}

public class Program
{
    public static void Main(string[] args)
    {
        Notifier notifier = new Notifier();
        Listener listener = new Listener();

        // 5. ربط معالج الحدث بالحدث (الاشتراك)
        notifier.OnStatusChanged += listener.HandleStatusChange;

        notifier.ChangeStatus("بدأ العمل");
        notifier.ChangeStatus("انتهى العمل");

        // يمكن إلغاء الاشتراك أيضًا
        notifier.OnStatusChanged -= listener.HandleStatusChange;
    }
}

```

شرح الكود

يبدأ الكود بتعريف **المفوض (Delegate)** `StatusUpdateHandler`، والذي يحدد توقيع أي دالة يمكن أن تعمل كمعالج حدث. بعد ذلك، نقوم بتعريف **الحدث (Event)** `OnStatusChanged` داخل الكائن `Notifier`. عندما يتم استدعاء الدالة `ChangeStatus`، يتم **رفع الحدث** `Invoke.OnStatusChanged()`، مما يستدعي بدوره جميع الدوال المشتركة. الكائن `Listener` يحتوي على **معالج الحدث** `HandleStatusChange` الذي يتم ربطه بالحدث باستخدام العامل `+=` في الدالة `Main`، وهكذا يستجيب الكائن `Listener` لتغيرات الحالة من الكائن `Notifier`.

- الأحداث تسمح للكائنات بإبلاغ كائنات أخرى بحدوث شيء دون الحاجة إلى معرفة تفاصيل تلك الكائنات.

- يستخدم **المفوض (Delegate)** لتحديد توقيع الدالة التي ستعمل كمعالج للحدث.

- يمكن للكائنات **الاشتراك** في الأحداث (باستخدام **=+**) و**إلغاء الاشتراك** منها (باستخدام **=-**) للتحكم في كيفية الاستجابة للأحداث.

مفاهيم متقدمة — Lambda Expressions

تعبيرات لامدا (Lambda Expressions)

تعبيرات لامدا في #C هي طريقة موجزة لكتابة دوال مجهولة (anonymous functions) لا تتطلب تعريفاً رسمياً. إنها تجعل الكود أقصر وأكثر قابلية للقراءة، وتستخدم بشكل واسع مع LINQ والبرمجة الوظيفية.

المفهوم الأساسي

تسمح تعبيرات لامدا بكتابة كود تنفيذي "مضمن" (inline) كمعامل لدالة أخرى. تتكون من جزء المدخلات والجزء التنفيذي، مفصولين بالعامل `<=>` (يُقرأ "يذهب إلى" أو "يؤول إلى").

مثال برمجي

```
using System;
using System.Collections.Generic;
using System.Linq;

public class LambdaDemo
{
    public static void Main(string[] args)
    {
        List<int> numbers = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8 };

        // استخدام تعبير لامدا لتصفية الأرقام الزوجية
        List<int> evenNumbers = numbers.Where(n => n % 2 == 0).ToList();

        Console.WriteLine("الأرقام الزوجية هي:");
        foreach (int num in evenNumbers)
        {
            Console.WriteLine(num);
        }
    }
}
```

شرح الكود

في هذا المثال، لدينا قائمة من الأرقام. استخدمنا تعبير `n % 2 == 0` مع دالة `where` لتصفية الأرقام. هنا، `n` يمثل كل عنصر في القائمة، والتعبير `n % 2 == 0` هو الشرط الذي يحدد ما إذا كان الرقم زوجيًا، ليتم تضمينه في القائمة الجديدة `evenNumbers`.

- تعتبر **تعبيرات لامدا** طريقة مختصرة لتعريف الدوال المجهولة في سطر واحد.
- تستخدم العامل `<=` لفصل معاملات الدالة عن نصها البرمجي.
- تزيد من **وضوح الكود وتركيزه**، وهي أساسية عند التعامل مع استعلامات LINQ.

مفاهيم متقدمة — Extension Methods

أساليب التوسيع (Extension Methods)

أساليب التوسيع (Extension Methods) في #C تتيح لك إضافة وظائف جديدة لأنواع موجودة بالفعل دون الحاجة لتعديل الكود الأصلي لتلك الأنواع أو اشتقاق أنواع جديدة منها. هذا مفيد جدًا لتوسيع مكتبات أو فئات لا تملك صلاحية تعديلها.

مفهوم رئيسي

الفكرة هي "تمديد" أو "توسيع" فئة (Class) أو واجهة (Interface) موجودة بإضافة أساليب جديدة (Methods) إليها، كما لو كانت جزءًا أصليًا منها. يتم تعريفها كـ أساليب ثابتة (static methods) داخل فئات ثابتة (static classes)، وتُستخدم الكلمة المفتاحية `this` في أول معامل لها.

مثال برمجي

```
using System;

public static class StringExtensions
{
    public static string CapitalizeFirstLetter(this string input)
    {
        if (string.IsNullOrEmpty(input))
            return input;
        return char.ToUpper(input[0]) + input.Substring(1);
    }
}

public class Program
{
    public static void Main(string[] args)
    {
        string message = "hello world";
        string capitalizedMessage = message.CapitalizeFirstLetter();
        Console.WriteLine(capitalizedMessage); // Output: Hello world
    }
}
```

```
}  
}
```

شرح الكود

في هذا المثال، قمنا بإنشاء فئة ثابتة اسمها `StringExtensions` بداخلها أسلوب توسيع اسمه `CapitalizeFirstLetter` مخصص للفئة `string`. لاحظ استخدام الكلمة المفتاحية `this` قبل المعامل الأول `string input`، مما يشير إلى أن هذا الأسلوب هو أسلوب توسيع. يمكننا الآن استدعاء هذا الأسلوب مباشرة على أي كائن من نوع `string`، كما لو كان جزءًا أصليًا منه.

- تضيف وظائف جديدة لأنواع موجودة دون تعديل الكود الأصلي.
- تستخدم الكلمة المفتاحية `this` في أول معامل لتعريف الأسلوب ك أسلوب توسيع.
- تساعد في تحسين قابلية قراءة الكود وتنظيمه عن طريق إضافة وظائف ذات صلة.

مفاهيم متقدمة — Anonymous Types

الأنواع المجهولة (Anonymous Types)

تُعد **الأنواع المجهولة** في #C ميزة قوية تمكنك من إنشاء كائنات بدون الحاجة إلى تعريف فئة (Class) بشكل صريح. إنها مفيدة جدًا عند الحاجة إلى تجميع مجموعة من الخصائص في كائن واحد بشكل مؤقت، خاصة عند العمل مع استعلامات LINQ.

المفهوم الرئيسي: الأنواع المجهولة

الأنواع المجهولة هي أنواع كائنات يتم إنشاؤها في وقت التشغيل بواسطة المترجم. تتميز بأن خصائصها تكون للقراءة فقط (read-only) ويتم تعريفها ضمنيًا باستخدام الكلمة المفتاحية `var`. لا يمكن إعادة تسمية أو تغيير نوع خصائصها بعد إنشائها.

مثال الكود

```
var person = new { Name = "علي", Age = 30 };
Console.WriteLine($"الاسم : {person.Name}");
Console.WriteLine($"العمر : {person.Age}");

var product = new { Id = 101, Name = "حاسوب محمول", Price = 1200.50m };
Console.WriteLine($"المنتج : {product.Name}, السعر : {product.Price}");
```

شرح الكود

في هذا المثال، قمنا بإنشاء كائنين من نوعين مجهولين: `person` و `product`. يحتوي الكائن `person` على خاصيتين هما `Name` و `Age`، بينما يحتوي الكائن `product` على `Id`، `Name` و `Price`. يتم الوصول إلى خصائص هذه الكائنات وطباعة قيمها مباشرة باستخدام نقطة (.)، تمامًا كما لو كانت كائنات من فئات معروفة.

- تسمح بإنشاء كائنات لتجميع البيانات بشكل مؤقت دون تعريف فئة محددة.
- يتم تعريف خصائصها ضمنيًا وتكون للقراءة فقط.

- تستخدم بشكل واسع في استعلامات LINQ لتشكيل نتائج الاستعلام.

البرمجة غير المتزامنة (Asynchronous Programming) — مقدمة إلى `await` و `async`

مقدمة إلى `await` و `async`

تُعد البرمجة غير المتزامنة (Asynchronous Programming) جزءًا أساسيًا لبناء تطبيقات سريعة الاستجابة وفعّالة، خاصة في C#. في هذا الدرس، سنتعرف على الكلمات المفتاحية `await` و `async` التي تُبسّط كتابة الكود غير المتزامن.

مفهوم `await` و `async`

تُمكننا `await` و `async` من تنفيذ مهام تستغرق وقتًا طويلاً دون تجميد واجهة المستخدم أو حجب خيط التنفيذ الرئيسي. عند استخدام `async` لتحديد دالة، فإنها تُشير إلى أن هذه الدالة يمكن أن تحتوي على نقاط انتظار. بينما تُستخدم `await` لانتظار اكتمال مهمة معينة، مع تحرير خيط التنفيذ لأداء مهام أخرى في تلك الأثناء، مما يعزز استجابة التطبيق.

مثال برمجي

```
using System;
using System.Threading.Tasks;

public class Program
{
    public static async Task Main(string[] args)
    {
        Console.WriteLine("بدء المهمة الرئيسية");
        await PerformLongRunningTask();
        Console.WriteLine("انتهاء المهمة الرئيسية");
    }

    public static async Task PerformLongRunningTask()
    {
        Console.WriteLine("...جاري تنفيذ مهمة غير متزامنة");
        await Task.Delay(3000); // محاكاة لعملية تستغرق 3 ثوانٍ
    }
}
```

```
        Console.WriteLine("المهمة غير المتزامنة اكتملت");  
    }  
}
```

شرح الكود

في هذا المثال، تُحدد الدالة `PerformLongRunningTask` كـ `async`، مما يسمح لها باستخدام `await` عند الوصول إلى `Task.Delay(3000)` داخل هذه الدالة، يتم تعليق تنفيذ `PerformLongRunningTask` مؤقتًا لمدة 3 ثواني، ويعود التحكم إلى الدالة `Main`. هذا يسمح لـ `Main` بتنفيذ أي عمل آخر أو تحرير خيط التنفيذ. بمجرد انتهاء التأخير، تستأنف `PerformLongRunningTask` تنفيذها من حيث توقفت. تُظهر رسائل الطباعة كيفية تدفق التنفيذ.

- `async`: تحدد دالة يمكنها تنفيذ عمليات غير متزامنة وتحتوي على نقاط انتظار.
- `await`: تُستخدم لانتظار اكتمال مهمة غير متزامنة دون حجب خيط التنفيذ.
- **تحسين الاستجابة**: تُحسن البرمجة غير المتزامنة بشكل كبير من استجابة التطبيق، خاصة عند التعامل مع عمليات I/O أو المهام الطويلة.

البرمجة غير المتزامنة (Asynchronous Programming) — Tasks

المهام (Tasks)

في البرمجة غير المتزامنة (Asynchronous Programming) بـ C#، تعد **المهام (Tasks)** عنصرًا أساسيًا. إنها تمثل عملية يمكن تشغيلها بشكل مستقل عن التدفق الرئيسي للبرنامج، مما يسمح لتطبيقك بالبقاء سريع الاستجابة ودون تجميد.

المفهوم الرئيسي

كائن **Task** هو تمثيل لعملية واحدة قد تكون قد اكتملت أو لم تكتمل بعد، وقد ترجع نتيجة أو لا ترجع. باستخدام المهام، يمكنك تنفيذ التعليمات البرمجية التي تستغرق وقتًا طويلاً دون حظر خيط التنفيذ (thread) الرئيسي، مما يحافظ على استجابة واجهة المستخدم.

مثال على الكود

```
using System;
using System.Threading.Tasks;

public class Program
{
    public static async Task Main(string[] args)
    {
        Console.WriteLine("بدء العملية غير المتزامنة");
        await PerformAsyncTask();
        Console.WriteLine("العملية غير المتزامنة انتهت");
    }

    public static async Task PerformAsyncTask()
    {
        Console.WriteLine("بدء مهمة انتظار");
        await Task.Delay(2000); // انتظار لمدة ثانيتين دون حظر
        Console.WriteLine("مهمة انتظار انتهت");
    }
}
```

```
}  
}
```

شرح الكود

في هذا المثال، لدينا دالتان غير متزامنتين: `Main` و `PerformAsyncTask`. عند استدعاء `PerformAsyncTask`، تبدأ عملية انتظار وهمية باستخدام `Task.Delay` لمدة ثانيتين. كلمة المفتاح `await` تجعل البرنامج ينتظر انتهاء هذه المهمة دون تجميد البرنامج بأكمله، مما يسمح للتدفقات الأخرى بالاستمرار. عندما تنتهي المهمة، يستأنف التنفيذ من حيث توقف.

النقاط الرئيسية

- `Task` هي كائنات تمثل عمليات يمكن تشغيلها بشكل غير متزامن.
- كلمة المفتاح `await` تسمح لك بالانتظار حتى تكتمل مهمة دون حظر خيط التنفيذ الرئيسي.
- البرمجة بالمهام **تحسن من استجابة التطبيق** بشكل كبير، خاصة في واجهات المستخدم الرسومية.

البرمجة غير المتزامنة (Asynchronous Programming) — إدارة الأخطاء في Async

إدارة الأخطاء في البرمجة غير المتزامنة (Async)

تُعد البرمجة غير المتزامنة جزءًا أساسيًا في #C لتطوير تطبيقات سريعة الاستجابة. ولكن، مثل أي عملية برمجية، يمكن أن تحدث أخطاء. فهم كيفية catch هذه الأخطاء ومعالجتها بشكل صحيح في السياقات غير المتزامنة أمر بالغ الأهمية لإنشاء تطبيقات مستقرة وموثوقة.

مفهوم رئيسي: استخدام try-catch مع await

في #C، يمكنك استخدام كتل `try-catch` القياسية للتعامل مع الاستثناءات (الأخطاء) التي تحدث داخل المهام غير المتزامنة. عندما تستخدم `await` على مهمة قد تفشل، فإن الاستثناء الذي يتم إطلاقه من تلك المهمة سيتم إعادة إطلاقه (re-thrown) على السياق الذي ينتظرها، مما يجعله قابلاً للاصطياد بواسطة كتلة `catch` كما هو الحال مع التعليمات البرمجية المتزامنة.

مثال كود

```
public class AsyncErrorHandling
{
    private static async Task SimulateFailingOperation()
    {
        await Task.Delay(100); // Simulate some async work
        throw new InvalidOperationException("فشلت العملية غير المتزامنة");
    }

    public static async Task RunExample()
    {
        Console.WriteLine("بدء محاولة تنفيذ عملية غير متزامنة");
        try
        {
            string result = await SimulateFailingOperation();
            Console.WriteLine("العملية اكتملت بنجاح: " + result); // This line will not be reached
        }
        catch (InvalidOperationException ex)
        {
            Console.WriteLine($"حدث خطأ: {ex.Message}");
        }
    }
}
```

```

    }
    catch (InvalidOperationException ex)
    {
        Console.WriteLine($"تم التقاط خطأ محدد: {ex.Message}");
    }
    catch (Exception ex)
    {
        Console.WriteLine($"تم التقاط خطأ عام غير متوقع: {ex.Message}")
    }
    Console.WriteLine("انتهت عملية التعامل مع الأخطاء.");
}
}

```

شرح الكود

في هذا المثال، لدينا دالة `SimulateFailingOperation` غير متزامنة تحاكي عملاً ثم تُطلق استثناءً من نوع `InvalidOperationException`. الدالة `RunExample` تستدعي هذه العملية داخل كتلة `try`. عندما تُطلق `SimulateFailingOperation` الاستثناء، فإن عبارة `await` تُعيد إطلاق هذا الاستثناء، مما يسمح لكتلة `catch (InvalidOperationException ex)` بالتقاطه والتعامل معه بشكل خاص، ثم تنتقل التعليمات البرمجية لتنفيذ ما بعدها.

خلاصات رئيسية

- **استخدام `try-catch`:** قم بتضمين عبارات `await` داخل كتل `try-catch` للتعامل مع الاستثناءات بشكل أنيق.
- **إعادة إطلاق الاستثناءات:** الاستثناءات التي تحدث داخل مهمة غير متزامنة تُعاد إطلاقها عند استخدام `await` عليها، مما يجعلها قابلة للاصطياد.
- **تعزيز الاستقرار:** تضمن إدارة الأخطاء الجيدة في البرمجة غير المتزامنة استقرار التطبيق وتوفير تجربة مستخدم أفضل، حتى عند حدوث مشاكل.

البرمجة غير المتزامنة (Asynchronous)

Parallel Programming Basics — (Programming

أساسيات البرمجة المتوازية (Parallel Programming Basics)

تساعدنا البرمجة المتوازية (Parallel Programming) على تنفيذ أجزاء متعددة من البرنامج في نفس الوقت. هذا مهم جداً لتسريع العمليات التي تتطلب الكثير من موارد المعالجة (CPU-bound operations)، مستغلةً قوة المعالجات الحديثة ذات الأنوية المتعددة.

ما هي البرمجة المتوازية؟

البرمجة المتوازية هي أسلوب لتنظيم العمل بحيث يتم تقسيم مهمة كبيرة إلى مهام أصغر يمكن تنفيذها **بشكل متزامن** عبر عدة أنوية للمعالج. في C#، توفر مكتبة المهام المتوازية (Task Parallel Library - TPL) أدوات قوية لتبسيط هذه العملية.

مثال عملي

```
using System;
using System.Diagnostics;
using System.Threading.Tasks;

public class Program
{
    public static void Main()
    {
        Console.WriteLine("بدء عملية حسابية متوازية");
        Stopwatch stopwatch = new Stopwatch();
        stopwatch.Start();

        long totalSum = 0;
        Parallel.For(0, 1000000, i =>
        {
            // عملية بسيطة تتطلب وقتاً
            long result = i * i;
        });
    }
}
```

```

        Interlocked.Add(ref totalSum, result); // إضافة آمنة للخيط
    });

    stopwatch.Stop();
    Console.WriteLine($"المجموع الكلي: {totalSum}");
    Console.WriteLine($"الوقت المستغرق (متوازي): {stopwatch.ElapsedMilli
}
}

```

شرح الكود

يقوم هذا الكود بتشغيل عملية حسابية بسيطة (ضرب الأرقام في نفسها) لمليون مرة. بدلاً من استخدام حلقة `for` عادية، نستخدم `Parallel.For`. هذا يجعل النظام يقوم بتقسيم هذه المليون عملية على الأنوية المتاحة في معالجك، مما يسرع من إنجاز المهمة بشكل كبير مقارنةً بالتنفيذ التسلسلي. `Interlocked.Add` يُستخدم هنا لضمان إضافة آمنة للناتج الكلي من كل مهمة متوازية.

- البرمجة المتوازية تتيح تنفيذ أجزاء من الكود في نفس الوقت لتسريع الأداء.
- تستفيد من الأنوية المتعددة للمعالج لتقسيم عبء العمل.
- في #C، توفر مكتبة TPL (مثل `Parallel.For`) أدوات سهلة لتطبيق البرمجة المتوازية.

التعامل مع قواعد البيانات — مقدمة إلى ADO.NET

مقدمة إلى ADO.NET

تُعد ADO.NET جزءاً أساسياً من .NET Framework، وتوفر مجموعة قوية من الفئات والخدمات التي تتيح لتطبيقات #C الاتصال بقواعد البيانات، استعراض البيانات، تعديلها، وحفظ التغييرات. إنها البوابة التي تربط تطبيقك بعالم البيانات.

المكونات الأساسية لـ ADO.NET

تنقسم ADO.NET بشكل عام إلى مكونات متصلة (Connected Layer) ومكونات غير متصلة (Disconnected Layer). المكونات المتصلة مثل **SqlConnection** و **SqlCommand** تسمح بالاتصال المباشر بقاعدة البيانات وتنفيذ الأوامر. أما المكونات غير المتصلة مثل **DataSet** و **DataAdapter**، فتتيح لك العمل مع البيانات بشكل مستقل عن قاعدة البيانات بعد جلبها.

مثال على الكود

```
using System;
using System.Data.SqlClient;

public class AdonetIntro
{
    public static void Main(string[] args)
    {
        string connectionString = "Data Source=localhost;Initial Catalog=Sam

        try
        {
            using (SqlConnection connection = new SqlConnection(connectionString))
            {
                connection.Open();
                Console.WriteLine("تم الاتصال بقاعدة البيانات بنجاح");
            }
        }
    }
}
```

```

    }
    catch (SqlException ex)
    {
        Console.WriteLine("حدث خطأ أثناء الاتصال " + ex.Message);
    }
    catch (Exception ex)
    {
        Console.WriteLine("خطأ عام " + ex.Message);
    }
}
}

```

شرح الكود

يبدأ الكود بتحديد سلسلة الاتصال (connection string) التي تحتوي على معلومات قاعدة البيانات. ثم يقوم بإنشاء كائن `SqlConnection` باستخدام هذه السلسلة. العبارة `using` تضمن أن الاتصال سيُغلق تلقائياً بمجرد الانتهاء من استخدامه، حتى لو حدث خطأ. بعد ذلك، يتم فتح الاتصال باستخدام `connection.Open()`، ويتم عرض رسالة نجاح أو خطأ حسب نتيجة العملية.

- **ADO.NET** هي تقنية #C للتعامل مع قواعد البيانات، وتتيح الاتصال وتنفيذ العمليات على البيانات.
- تتضمن مكونات للاتصال المباشر (مثل `SqlConnection`) وأخرى للعمل مع البيانات دون اتصال دائم (مثل `DataSet`).
- من الضروري دائماً فتح الاتصال وإغلاقه بشكل صحيح باستخدام عبارة `using` لضمان إدارة الموارد بكفاءة.

التعامل مع قواعد البيانات — تنفيذ أوامر SQL الأساسية

تنفيذ أوامر SQL الأساسية في #C

تعد القدرة على التفاعل مع قواعد البيانات جزءًا أساسيًا من تطوير أي تطبيق. في هذا الدرس، سنتعلم كيف يمكن لتطبيقات #C الاتصال بقاعدة بيانات (مثل SQL Server) وتنفيذ أوامر SQL الأساسية مثل **INSERT**، **UPDATE**، و**DELETE** لتعديل البيانات، و**SELECT** لاسترجاعها.

الاتصال بقاعدة البيانات وتنفيذ الأوامر

للتفاعل مع قاعدة البيانات من #C، نستخدم كائنات مثل `SqlConnection` لإدارة الاتصال، و `SqlCommand` لتحديد وتنفيذ أمر SQL. يعتمد تنفيذ الأمر على نوعه، حيث نستخدم `ExecuteNonQuery` للأوامر التي لا تعيد بيانات (مثل الإضافة والتعديل والحذف).

مثال عملي: إضافة سجل جديد

```
using System;
using System.Data.SqlClient;

public class SqlExecutor
{
    public static void Main(string[] args)
    {
        string connectionString = "Data Source=serverName;Initial Catalog=databaseName;User Id=username;Password=password;";
        string sqlCommandText = "INSERT INTO Students (Name, Age) VALUES ('2020-01-01', 20)";

        using (SqlConnection connection = new SqlConnection(connectionString))
        {
            try
            {
                connection.Open();
                using (SqlCommand command = new SqlCommand(sqlCommandText, connection))
                {
                    int rowsAffected = command.ExecuteNonQuery();
                }
            }
            catch { }
        }
    }
}
```

```

        Console.WriteLine($"سجل بنجاح {rowsAffected} تم إضافة");
    }
}
catch (SqlException ex)
{
    Console.WriteLine("حدث خطأ عند إضافة البيانات" + ex.Message);
}
}
}
}

```

شرح الكود

يبدأ الكود بتحديد سلسلة الاتصال `connectionString` التي تخبر التطبيق بكيفية الوصول إلى قاعدة البيانات. ثم نحدد أمر SQL المراد تنفيذه (هنا، لإضافة طالب جديد). يستخدم الكود `SqlConnection` لإنشاء اتصال و `SqlCommand` لتحديد الأمر وتنفيذه. يتم فتح الاتصال ثم استدعاء `ExecuteNonQuery()` الذي ينفذ الأمر ويعيد عدد الصفوف المتأثرة. يتم التعامل مع أي أخطاء محتملة باستخدام `try-catch`.

نقاط أساسية

- `SqlConnection`: يُستخدم لإنشاء وإدارة الاتصال بقاعدة البيانات.
- `SqlCommand`: يُستخدم لإنشاء وتحديد أمر SQL المراد تنفيذه.
- `ExecuteNonQuery()`: طريقة لتنفيذ أوامر SQL التي لا تعيد مجموعات نتائج (مثل INSERT, UPDATE, DELETE).

التعامل مع قواعد البيانات — مقدمة إلى Entity Framework Core

مقدمة إلى Entity Framework Core

Entity Framework Core (EF Core) هو إطار عمل (ORM) (Object-Relational Mapper) شائع جداً في #C، يسهل التعامل مع قواعد البيانات بشكل كبير دون الحاجة لكتابة استعلامات SQL يدوياً. إنه يسمح لك بالتفاعل مع البيانات باستخدام كائنات #C المألوفة لديك.

ما هو Entity Framework Core (EF Core)؟

EF Core هو تقنية تمكن المطورين من التفاعل مع البيانات في قواعد البيانات العلائقية باستخدام كائنات #C بدلاً من أوامر SQL المباشرة. يقوم بتحويل استدعاءات الكود البرمجي إلى استعلامات SQL، ثم يعكس نتائج قاعدة البيانات إلى كائنات #C لتسهيل العمل عليها.

مثال عملي

```
using Microsoft.EntityFrameworkCore;
using System.Collections.Generic;

// 1. تعريف الكيان (Entity)
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Age { get; set; }
}

// 2. تعريف سياق قاعدة البيانات (DbContext)
public class SchoolContext : DbContext
{
    public DbSet<Student> Students { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBui
    {
```

```
// للاختبار استخدام قاعدة بيانات SQLite
optionsBuilder.UseSqlite("Data Source=school.db");
}
}
```

شرح الكود

في هذا المثال، بدأنا بتعريف فئة **Student** التي تمثل جدولاً للطلاب في قاعدة البيانات، مع خصائص مطابقة لأعمدة الجدول. ثم أنشأنا فئة **SchoolContext** التي ترث من **DbContext**، وهي نقطة الدخول الرئيسية لـ EF Core. الخاصية **Students** من نوع **DbSet<Student>** تخبر EF Core أن لدينا مجموعة من كائنات الطلاب (جدول طلاب). وأخيراً، تحدد الدالة **OnConfiguring** نوع قاعدة البيانات التي سنتصل بها، وهنا استخدمنا SQLite لأغراض بسيطة.

نقاط أساسية

- **تبسيط التعامل مع البيانات:** EF Core يحول كائنات #C إلى بيانات قواعد بيانات والعكس، مما يقلل الحاجة لكتابة SQL يدوياً.
- **ORM (Object-Relational Mapper):** هو وسيط يربط بين عالم البرمجة الشيئية (الكائنات) وعالم قواعد البيانات العلائقية (الجدول).
- **DbContext و DbSet:** هما المكونان الأساسيان للتفاعل؛ DbContext يمثل جلسة اتصال بقاعدة البيانات، و DbSet يمثل مجموعة من الكائنات (جدول).

التعامل مع قواعد البيانات — إنشاء النماذج والعلاقات (Models & Relationships)

إنشاء النماذج والعلاقات (Models & Relationships)

في #C، تُعد النماذج (Models) والعلاقات (Relationships) أساسًا للتعامل المنظم مع قواعد البيانات. تساعدنا هذه المفاهيم على تمثيل هيكل قاعدة البيانات في تطبيقنا، مما يسهل قراءة البيانات وكتابتها وإدارتها.

ما هي النماذج (Models)؟

النموذج هو ببساطة فئة (Class) في #C تمثل جدولًا معينًا في قاعدة البيانات. خصائص (Properties) هذه الفئة تتوافق مع أعمدة الجدول، مما يوفر طريقة سهلة وموجهة للكائنات للتفاعل مع البيانات.

ما هي العلاقات (Relationships)؟

العلاقات تحدد كيف ترتبط النماذج (الجدول) المختلفة ببعضها البعض. أشهر أنواع العلاقات هي: واحد لواحد (One-to-One)، واحد لمتعدد (One-to-Many)، و متعدد لمتعدد (Many-to-Many). هذه العلاقات ضرورية لربط البيانات ذات الصلة ببعضها.

مثال على نموذج وعلاقة

```
public class Category
{
    public int Id { get; set; }
    public string Name { get; set; }
    public List<Product> Products { get; set; } = new List<Product>();
}

public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
```

```
public decimal Price { get; set; }
public int CategoryId { get; set; } // مفتاح خارجي
public Category Category { get; set; }
}
```

شرح الكود

في هذا المثال، لدينا نموذج **Category** يمثل جدول الفئات، ونموذج **Product** يمثل جدول المنتجات. الفئة **Category** تحتوي على خاصية **Products** وهي قائمة من المنتجات، مما يعكس علاقة "واحد لمتعدد" (كل فئة تحتوي على العديد من المنتجات). خاصية **CategoryId** في نموذج **Product** هي مفتاح خارجي يربطه بالفئة الأم **Category**.

- **النماذج هي تمثيلات #C لجدول قواعد البيانات:** كل نموذج (فئة) يعكس هيكل جدول بيانات.
- **العلاقات تربط النماذج ببعضها:** تسمح بربط البيانات المتصلة عبر الجداول المختلفة.
- **المفاتيح الخارجية تُمكن العلاقات:** تُستخدم لإنشاء الروابط بين النماذج (الجدول) في قاعدة البيانات.

التعامل مع قواعد البيانات — Migration وإدارة قاعدة البيانات

Migration وإدارة قاعدة البيانات

عند تطوير تطبيقات #C التي تتفاعل مع قواعد البيانات، وخاصة باستخدام Entity Framework Core، يصبح إدارة تغييرات هيكل قاعدة البيانات (Schema) مع مرور الوقت تحديًا. توفر لنا آلية الـ **Migration** حلاً قويًا ومنظمًا لهذه المشكلة.

مفهوم الـ Migration

الـ Migration في #C هي خاصية ضمن Entity Framework Core تسمح لك بتتبع وإدارة التغييرات التي تحدث على نموذج بيانات تطبيقك (Code-First Model) وتطبيقها على قاعدة البيانات بشكل تدريجي. تتيح لك هذه الآلية إضافة أو تعديل أو حذف الجداول والأعمدة والفهارس دون الحاجة لإعادة إنشاء قاعدة البيانات يدويًا، مما يحافظ على البيانات الموجودة.

مثال الكود

```
Add-Migration InitialCreate -Context ApplicationDbContext
```

شرح الكود

يقوم هذا الأمر بإنشاء ملف Migration جديد باسم **InitialCreate** في مشروعك. هذا الملف سيحتوي على الكود اللازم لإنشاء الهيكل الأولي لقاعدة البيانات (الجداول والحقول) بناءً على نماذج البيانات (Models) المعرّفة في تطبيقك، ضمن سياق قاعدة البيانات المحدد **ApplicationDbContext**. بعد إنشاء الـ Migration، ستحتاج إلى تطبيقها على قاعدة البيانات باستخدام أمر **Update-Database**.

- تتبع التغييرات التي تطرأ على هيكل قاعدة البيانات بشكل منظم وموثق.
- تطبيق تحديثات قاعدة البيانات تدريجيًا دون فقدان البيانات الموجودة.

- الحفاظ على تزامن دائم بين نماذج البيانات في الكود وهيكل قاعدة البيانات الفعلي.

بناء تطبيقات C# متنوعة — تطبيقات Console

تطبيقات Console

تعد تطبيقات الكونسول (Console Applications) نقطة البداية المثالية لتعلم برمجة C#. إنها برامج تعمل في بيئة نصية بحتة، مثل موجه الأوامر (Command Prompt) أو الطرفية (Terminal)، وتتفاعل مع المستخدم عن طريق إدخال وإخراج النصوص فقط. تُستخدم لتعلم الأساسيات السريعة واختبار الأكواد.

المفهوم الأساسي

يعتمد تطبيق الكونسول بشكل أساسي على التفاعل مع الشاشة السوداء (أو شاشة الطرفية)؛ حيث يقوم بعرض المعلومات للمستخدم عبر النصوص، ويستقبل المدخلات منه أيضاً كتعليمات نصية. لا تحتوي هذه التطبيقات على واجهة مستخدم رسومية (GUI)، مما يجعلها بسيطة ومركزة على منطق البرنامج.

مثال كود

```
using System;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("!مرحباً بك في تطبيق الكونسول");
        Console.Write("الرجاء إدخال اسمك: ");
        string userName = Console.ReadLine();
        Console.WriteLine($"{userName}! أهلاً وسهلاً بك يا");
        Console.WriteLine("...اضغط على أي مفتاح للمتابعة");
        Console.ReadKey();
    }
}
```

شرح الكود

يبدأ هذا الكود بعرض رسالة ترحيب على الشاشة باستخدام `Console.WriteLine()`. ثم يطلب من المستخدم إدخال اسمه باستخدام `Console.Write()` ويعرض المؤشر للانتظار. يتم قراءة الإدخال من المستخدم وحفظه في المتغير `userName` باستخدام `Console.ReadLine()`. بعد ذلك، يعرض رسالة ترحيب مخصصة تتضمن الاسم الذي تم إدخاله. أخيرًا، `Console.ReadKey()` يحافظ على نافذة الكونسول مفتوحة حتى يضغط المستخدم على أي مفتاح.

- تطبيقات الكونسول هي الأساس لتعلم #C والتفاعل النصي المباشر.
- تستخدم `Console.WriteLine()` لعرض النصوص و `Console.ReadLine()` لقراءة مدخلات المستخدم.
- مثالية لتطوير الأدوات البسيطة، تجربة الأكواد، وفهم سير عمل البرنامج الأساسي.

بناء تطبيقات #C متنوعة — تطبيقات Windows Forms

تطبيقات Windows Forms

تُعد تطبيقات Windows Forms جزءًا أساسيًا في لغة #C لبناء برامج سطح المكتب ذات واجهة المستخدم الرسومية (GUI). تتيح لك هذه التقنية إنشاء تطبيقات تفاعلية بسهولة، حيث يمكنك تصميم الواجهة مرئيًا باستخدام أدوات السحب والإفلات، ثم إضافة المنطق البرمجي للتحكم في سلوك التطبيق.

المفهوم الأساسي لتطبيقات Windows Forms

تعتمد Windows Forms على فكرة تصميم الواجهة المرئية عن طريق إضافة مكونات (Controls) مثل الأزرار ومربعات النصوص والقوائم إلى نموذج (Form). بعد ذلك، يمكنك كتابة أكواد #C للاستجابة لأحداث المستخدم، مثل النقر على زر أو إدخال نص، لتجعل التطبيق يقوم بمهام محددة.

مثال كودي بسيط

```
private void buttonClickEvent_Click(object sender, EventArgs e)
{
    // عند النقر على الزر، يتم عرض مربع رسالة
    MessageBox.Show("أهلاً بك في تطبيقنا الأول باستخدام Windows Forms!");
}
```

شرح الكود

يمثل هذا الكود دالة لمعالجة حدث النقر على زر معين في تطبيق Windows Forms. عندما يقوم المستخدم بالنقر على هذا الزر، يتم استدعاء هذه الدالة. داخلها، تقوم التعليمة `MessageBox.Show`

بعرض نافذة منبثقة صغيرة تحتوي على نص ترحيبي، لإعلام المستخدم بنجاح العملية أو تقديم معلومة بسيطة.

- **بناء واجهات رسومية:** تُستخدم لإنشاء برامج سطح المكتب بواجهات تفاعلية ومرئية.
- **مكونات جاهزة:** توفر مكتبة غنية بالمكونات (أزرار، مربعات نص، قوائم) لتصميم الواجهات بسرعة.
- **البرمجة الموجهة بالأحداث:** تعتمد على كتابة أكواد للاستجابة لأفعال المستخدم (مثل النقر أو الإدخال).

بناء تطبيقات #C متنوعة — مقدمة إلى WPF

مقدمة إلى WPF

في هذا الدرس، سنبدأ رحلتنا في عالم بناء تطبيقات سطح المكتب الغنية باستخدام #C ومكتبة WPF. ستقدم هذه المقدمة فهماً أساسياً لـ WPF، وكيف يمكنك من إنشاء واجهات مستخدم جذابة وتفاعلية لتطبيقات Windows.

ما هو WPF؟

WPF (Windows Presentation Foundation) هي إطار عمل قوي ضمن .NET يسمح للمطورين بإنشاء تطبيقات سطح مكتب تفاعلية وغنية بصرياً لنظام التشغيل Windows. إنه يجمع بين واجهة المستخدم والرسومات والوسائط والمستندات في نموذج برمجة واحد، مستخدماً لغة XAML لتصميم الواجهة و #C للمنطق الخلفي.

مثال على كود XAML بسيط

```
<Window x:Class="WpfApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="الأول WPF تطبيق" Height="200" Width="300">
    <Grid>
        <TextBlock Text="أهلاً بك في WPF!"
                   FontSize="24"
                   HorizontalAlignment="Center"
                   VerticalAlignment="Center"/>
    </Grid>
</Window>
```

شرح الكود

يمثل الكود أعلاه نافذة تطبيق WPF بسيطة. العنصر **<Window>** هو الحاوية الرئيسية للتطبيق ويحدد حجم النافذة وعنوانها. داخله، نستخدم **<Grid>** لترتيب العناصر، وهو أحد لوحات التخطيط الشائعة في

WPF. يظهر **<TextBlock>** نصًا بسيطًا "أهلاً بك في WPF!" في منتصف النافذة، مع تحديد حجم الخط ومحاذاته.

- **WPF** هو إطار عمل لبناء تطبيقات سطح مكتب غنية بصريًا في C#.
- يستخدم **XAML** لتصميم واجهة المستخدم وC# للمنطق البرمجي.
- يوفر **WPF** قدرات قوية للرسومات والوسائط وواجهات المستخدم التفاعلية الحديثة.

بناء تطبيقات C# متنوعة — بناء APIs باستخدام ASP.NET Core

بناء APIs باستخدام ASP.NET Core

تُعد واجهات برمجة التطبيقات (APIs) جسوراً أساسية لتبادل البيانات بين مختلف التطبيقات. باستخدام ASP.NET Core، يمكن للمطورين بناء APIs قوية وفعالة تدعم التواصل بين الواجهات الأمامية (مثل تطبيقات الويب والجوال) والخلفيات، مما يفتح الأبواب لتطبيقات C# متنوعة ومتكاملة.

مفهوم ال API في ASP.NET Core

ال API (Application Programming Interface) في ASP.NET Core هي مجموعة من نقاط النهاية (endpoints) التي تسمح لتطبيقات مختلفة بالتفاعل مع بعضها البعض. هي تستخدم بروتوكول HTTP لإرسال واستقبال البيانات، عادة بصيغة JSON، مما يتيح بناء أنظمة موزعة ومرنة.

مثال الكود

```
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class GreetingController : ControllerBase
{
    [HttpGet]
    public string GetGreeting()
    {
        return "API! مرحباً بكم في";
    }
}
```

شرح الكود

يوضح هذا الكود كيفية إنشاء متحكم (Controller) بسيط في ASP.NET Core API. السمة

[ApiController] تحدد أن الفئة هي متحكم API، بينما تحدد [Route("api/[controller]")]

المسار الأساسي لنقاط النهاية (مثلاً: api/Greeting/). السمة [HttpGet] تشير إلى أن الدالة

GetGreeting() ستستجيب لطلبات HTTP GET، وتقوم بإرجاع رسالة نصية بسيطة.

- **سهولة البناء:** ASP.NET Core يوفر أدوات ومكونات جاهزة لبناء APIs بسرعة وفعالية.
- **تبادل البيانات:** APIs تسمح بتبادل البيانات (غالباً بصيغة JSON) بين تطبيقات الواجهة الأمامية والخلفية.
- **المتحكمات (Controllers):** هي الوحدات الأساسية التي تنظم نقاط النهاية وتتعامل مع طلبات HTTP المختلفة.

بناء تطبيقات #C متنوعة — مقدمة إلى Dependency Injection

مقدمة إلى Dependency Injection

Dependency Injection (DI) هو نمط تصميم أساسي في بناء تطبيقات #C الحديثة، يهدف إلى جعل الكود أكثر مرونة، قابلية للاختبار، وسهولة في الصيانة. يساعدنا هذا المفهوم على فصل المكونات عن بعضها، مما يقلل من الترابط (Coupling) ويزيد من قابلية التوسع.

ما هو Dependency Injection؟

ببساطة، Dependency Injection هو مبدأ لتزويد الكائن بالاعتمادات (Dependencies) التي يحتاجها من الخارج، بدلاً من أن يقوم الكائن بإنشاء هذه الاعتمادات بنفسه. الاعتماد هو أي كائن أو خدمة تحتاجها فئة أخرى لأداء وظيفتها.

مثال عملي

```
using System;

// 1. تحديد العقد (Interface) الواجهة
public interface ILogger
{
    void Log(string message);
}

// 2. تطبيق الواجهة (Implementation)
public class ConsoleLogger : ILogger
{
    public void Log(string message)
    {
        Console.WriteLine($"[Console Log]: {message}");
    }
}
```

```
// 3. (Constructor Injection) الفئة التي تحتاج الاعتماد وتلقاه عبر البناء
public class ReportGenerator
{
    private readonly ILogger _logger;

    public ReportGenerator(ILogger logger)
    {
        _logger = logger;
    }

    public void GenerateReport()
    {
        _logger.Log("بدء توليد التقرير");
        // منطق توليد التقرير هنا
        _logger.Log("تم الانتهاء من توليد التقرير");
    }
}

// 4. استخدام الفئات وتمرير الاعتماد
public class Program
{
    public static void Main(string[] args)
    {
        // إنشاء الاعتماد
        ILogger myLogger = new ConsoleLogger();

        // (Dependency Inject) عبر البناء ReportGenerator تمرير الاعتماد إلى
        ReportGenerator generator = new ReportGenerator(myLogger);
        generator.GenerateReport();
    }
}
```

شرح الكود

في هذا المثال، لدينا واجهة `ILogger` وفئة `ConsoleLogger` التي تطبق هذه الواجهة. الفئة `ReportGenerator` تحتاج إلى خدمة تسجيل الأحداث (Logging)، ولكنها لا تقوم بإنشاء `ConsoleLogger` بنفسها. بدلاً من ذلك، يتم تمرير كائن يطبق `ILogger` (في حالتنا `ConsoleLogger`) إلى مُنشئ `ReportGenerator`. هذا ما يسمى بـ "حقن الاعتماد" (Dependency Injection)، حيث تُزود الفئة باعتمادها من خارجها، مما يجعل `ReportGenerator` مستقلة عن كيفية تنفيذ عملية التسجيل.

- **تحسين قابلية الاختبار:** يسهل استبدال الاعتمادات بتطبيقات وهمية (Mocks) أثناء الاختبارات، دون الحاجة لتعديل الفئة التي يتم اختبارها.
- **تقليل الترابط (Coupling):** الفئات تعتمد على الواجهات (Interfaces) بدلاً من تطبيقات محددة، مما يقلل من تماسك المكونات ببعضها.
- **مرونة أكبر:** يمكن تغيير تطبيق اعتماد معين (مثل تبديل `ConsoleLogger` بـ `FileLogger`) دون تعديل الفئات التي تستخدم هذا الاعتماد.